Roma Tre

INGEGNERIA
CIVILE, INFORMATICA E DELLE TECNOLOGIE AERONAUTICHE

Information Visualization

Implicit Representations of Trees

M. Patrignani

 copyright ©2025 m.patrignani

Copyright notice

- All the pages/slides in this presentation, including but not limited to, images, photos, animations, videos, sounds, music, and text (hereby referred to as “material”) are protected by copyright
- This material, with the exception of some multimedia elements licensed by other organizations, is property of the authors and/or organizations appearing in the first slide
- This material, or its parts, can be reproduced and used for didactical purposes within universities and schools, provided that this happens for non-profit purposes
- Any other use is prohibited, unless explicitly authorized by the authors on the basis of an explicit agreement
- This copyright notice must always be redistributed together with the material, or its portions

 copyright ©2025 m.patrignani

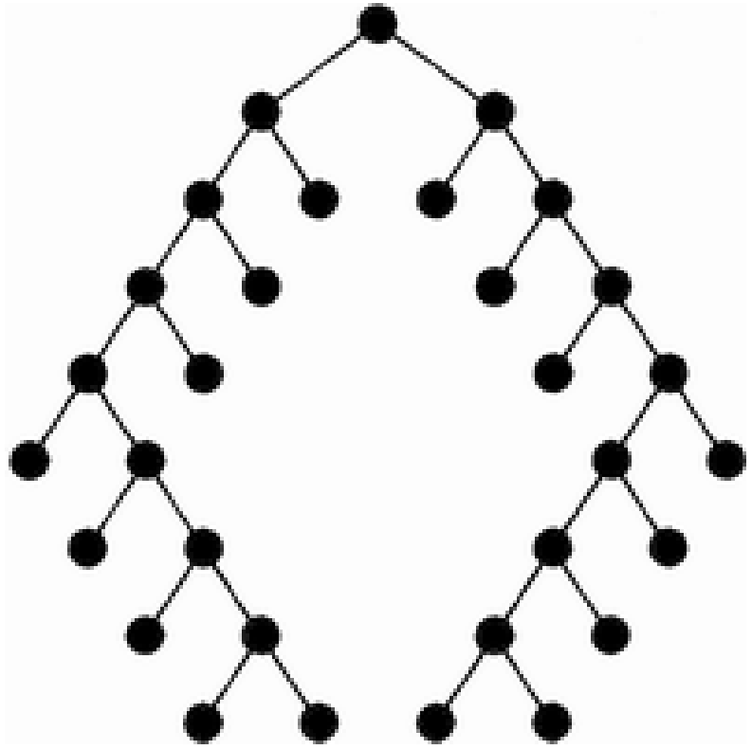
Overview

- Limitations of node-link representations of trees
- Implicit tree representations
  - treemaps and nested treemaps
  - variants of treemaps
    - cushion treemaps, cluster treemaps, squarified treemaps, ordered treemaps, quantum treemaps, voronoi treemaps
  - circular treemaps

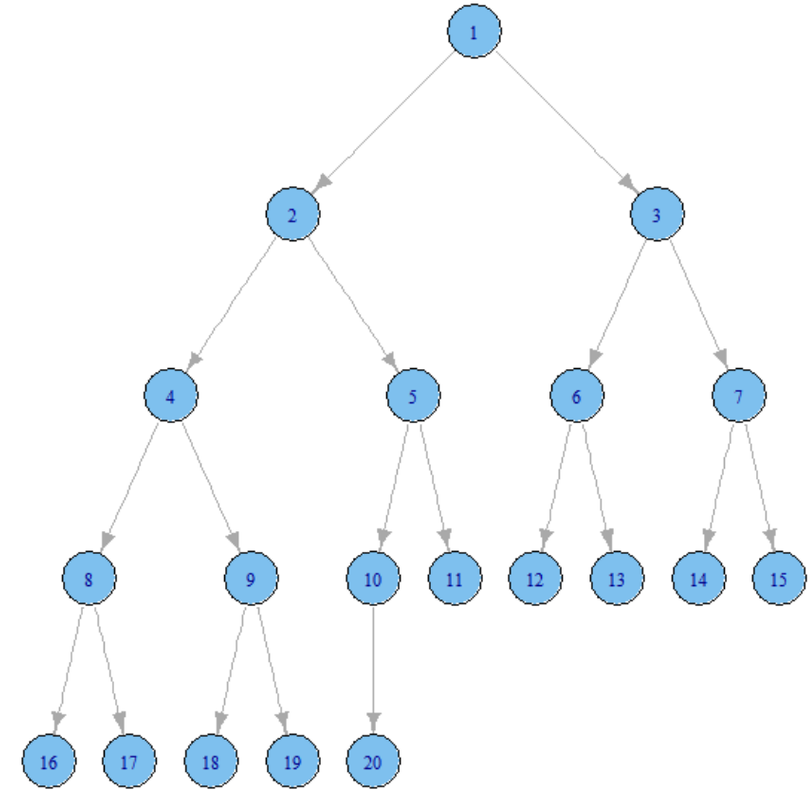
 copyright ©2025 m.patrignani

Limitations of node-link representations

- Let us consider the algorithm by [Reingold & Tilford 81]



From [Reingold & Tilford 81]

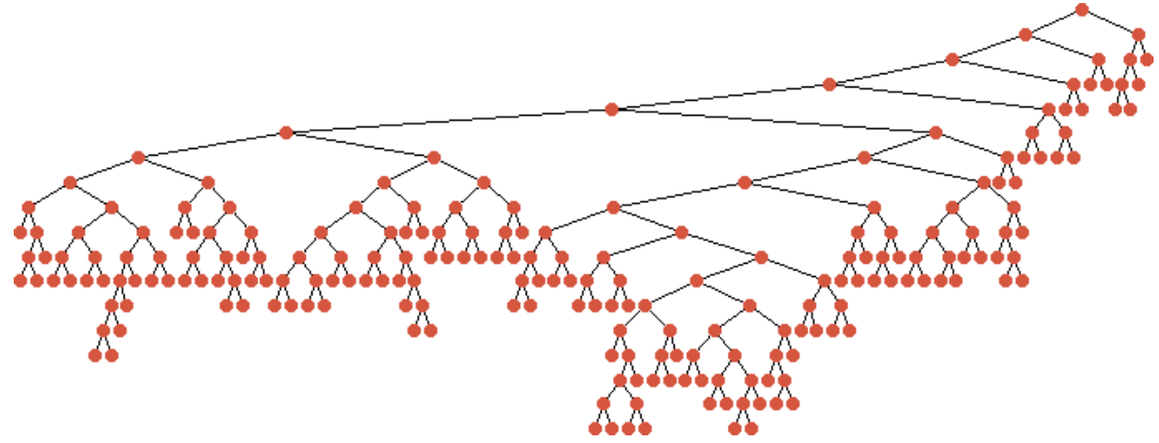


From http://stackoverflow.com/questions/18270370

 copyright ©2025 m.patrignani

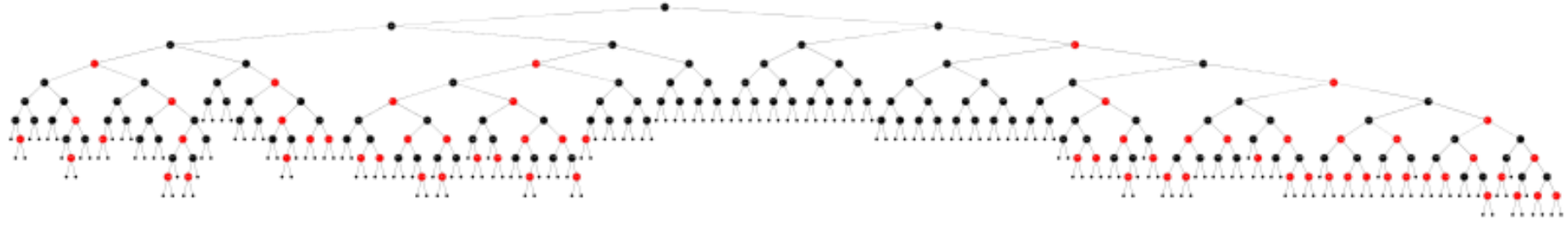
Limitations of node-link representations

- A Branch and Cut tree with 217 nodes drawn with [Walker 90]



From http://www.informatik.uni-koeln.de/ls_juenger/research/vbctool/

- A red-black binary tree drawn with [Reingold & Tilford 81]



From http://cartesianproduct.wordpress.com/tag/reingold-tilford-algorithm/

 copyright ©2025 m.patrignani

Limitations of node-link representations

- The main limitation stems from a poor use of the space
- The drawing tends to grow horizontally
  - the higher part of the drawing is very sparse
  - the lower part of the drawing is very dense
- It is difficult to encode variables for the data cases
  - for example, labels, shapes, colors, …

copyright ©2025 m.patrignani

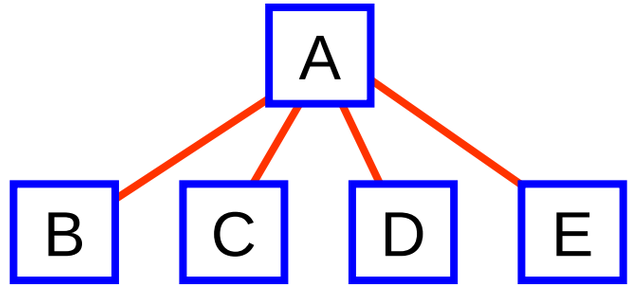
Implicit representations of trees

- Classifications [Schulz et al. 11]
  - by edge representation
  - by node representation
  - by dimensionality
  - by layout method

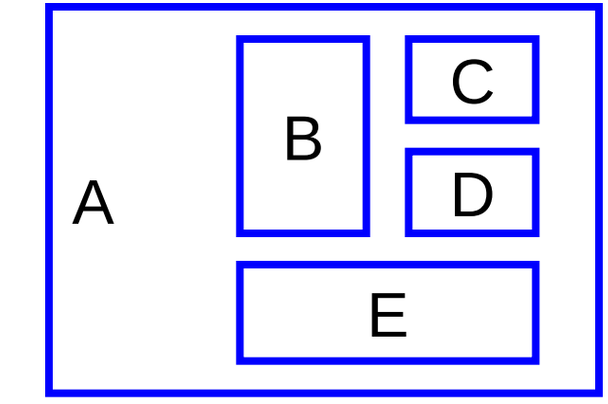
 copyright ©2025 m.patrignani

Implicit representations of trees

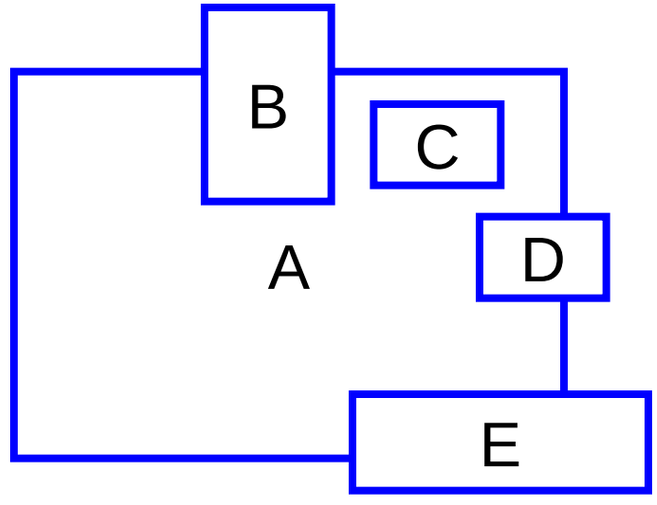
- Edge representations



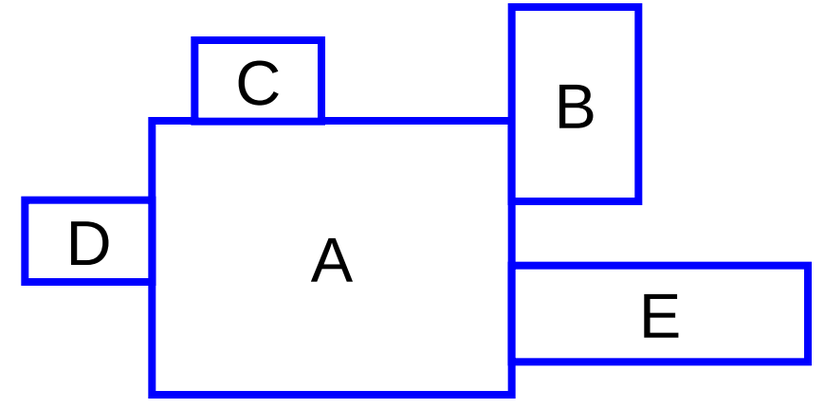
(explicit) node-link representation



implicit layout by inclusion
(space filling)



implicit layout by overlap

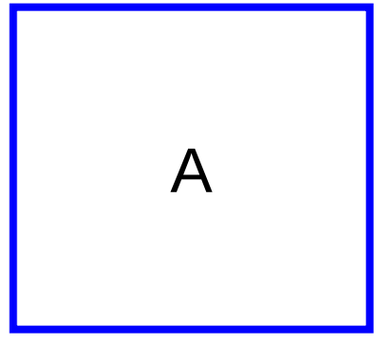


implicit layout by contact

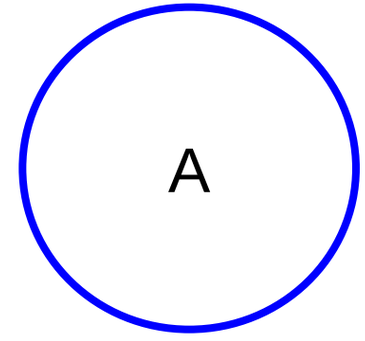
copyright ©2025 m.patrignani

Implicit representations of trees

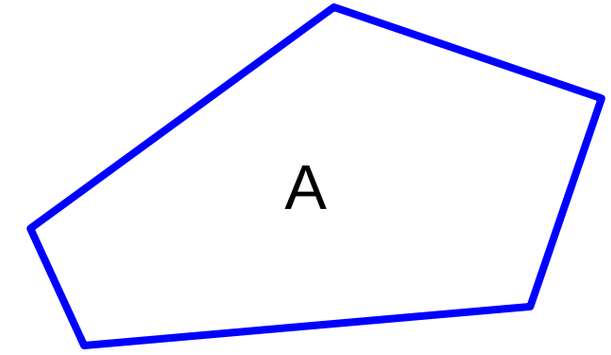
- Node representations



Squared nodes



Circle nodes



Nodes of arbitrary shape

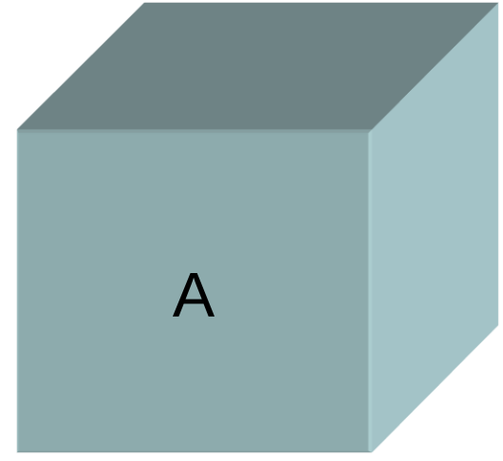
 copyright ©2025 m.patrignani

Implicit representations of trees

- Dimensionality



Two-dimensional



Three-dimensional

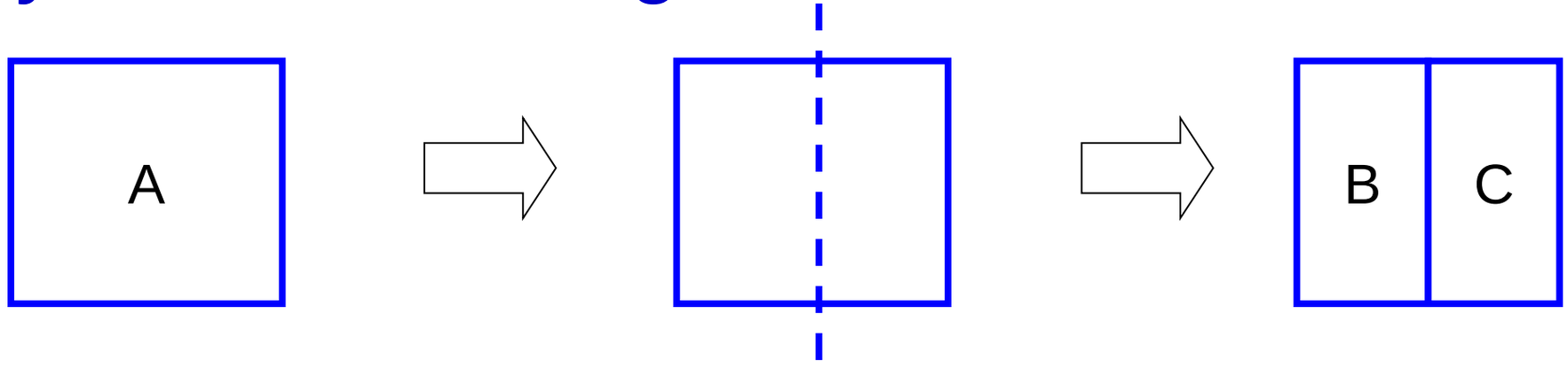


Two-and-a-half-dimensional

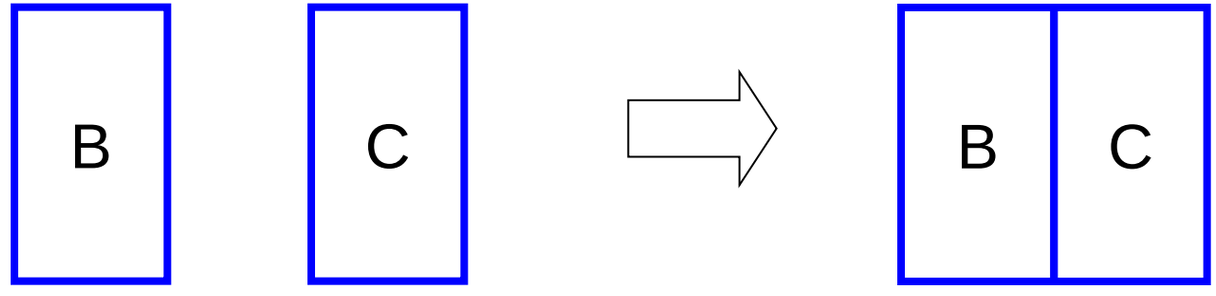
 copyright ©2025 m.patrignani

Implicit representations of trees

- Layout methodologies



Subdivision (top down)



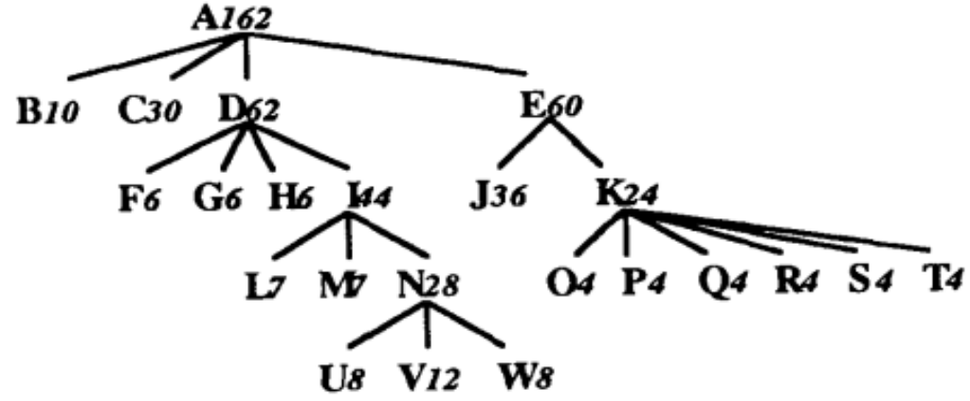
Packing (bottom up)

 copyright ©2025 m.patrignani

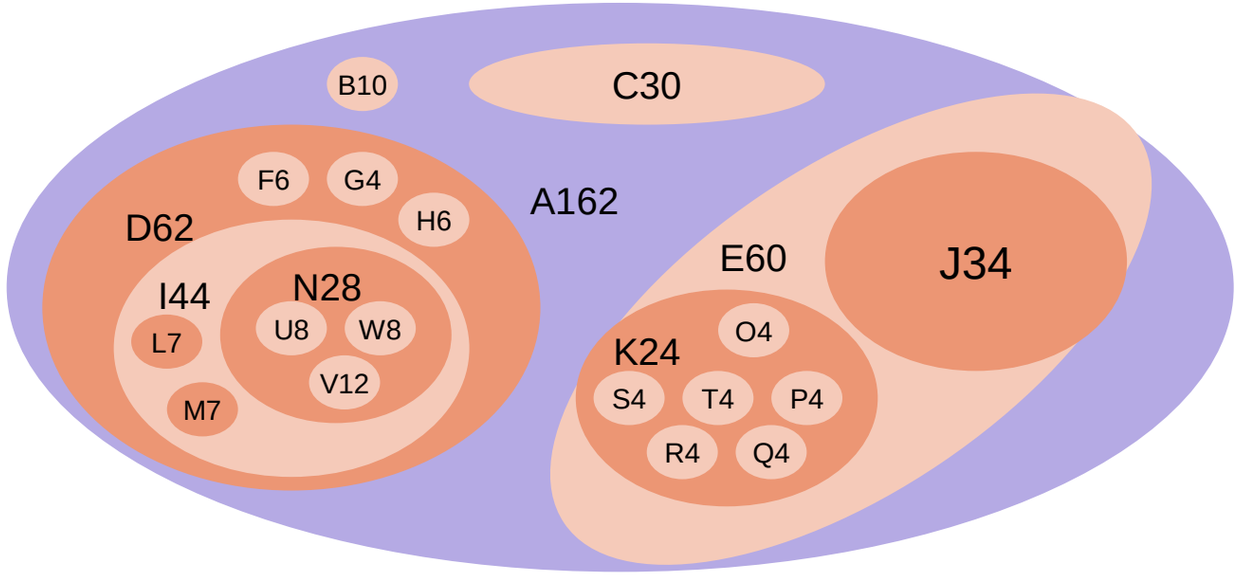
Space-filling approach

- Fill the space by nesting children inside parents

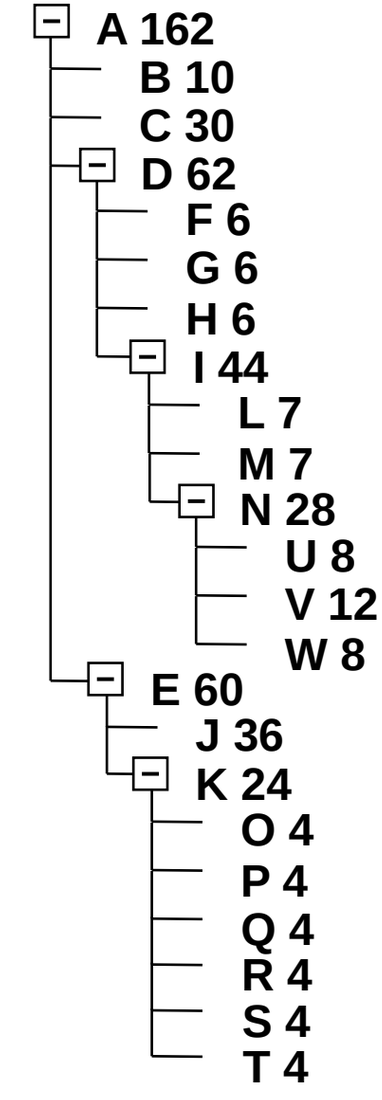
Node-link



Venn diagram



outline drawing

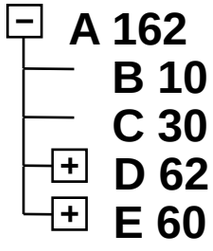


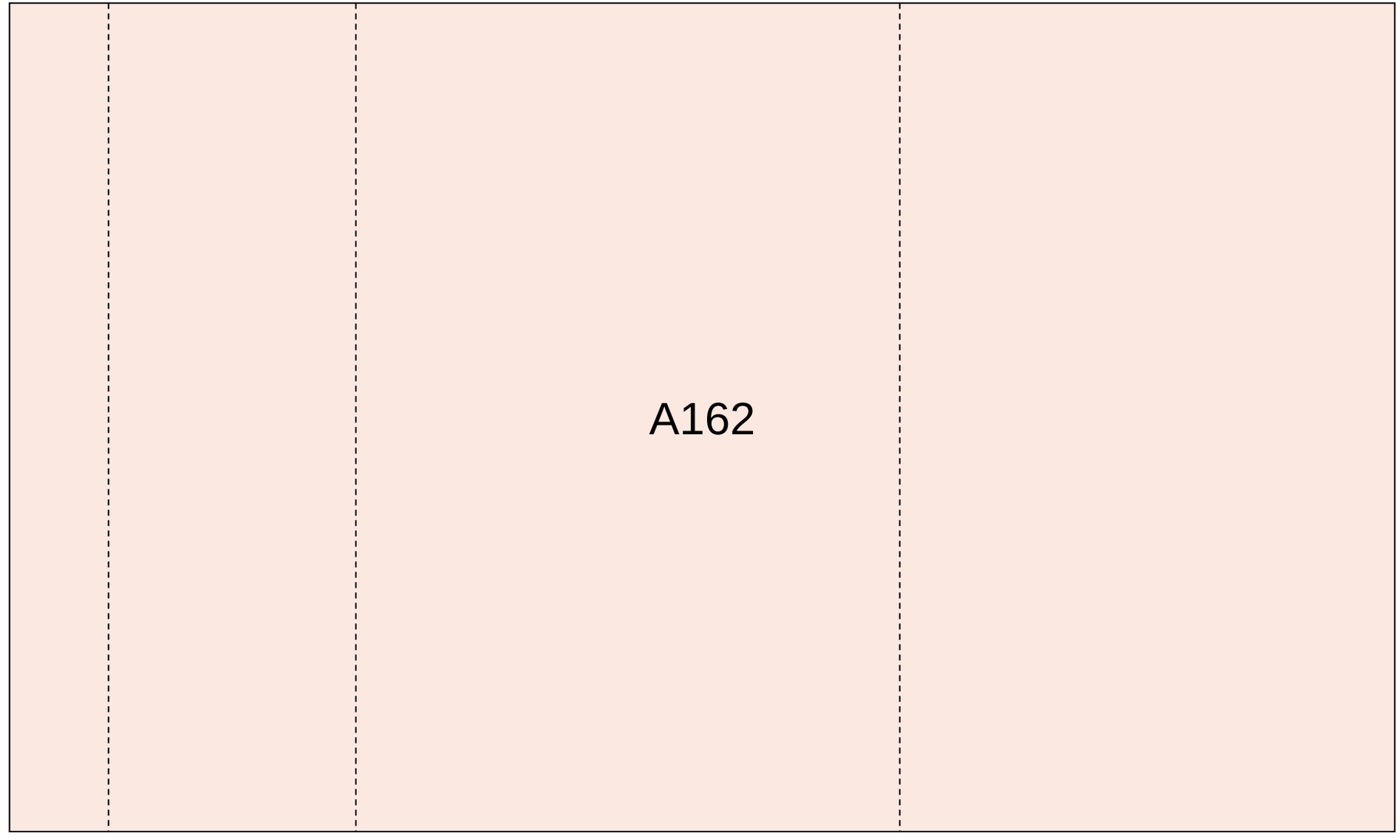
Treemaps

- Space-filling representation developed by [Johnson & Shneiderman 91]
  - rectangular shapes represent nodes
  - children are drawn inside their parent
  - alternate horizontal and vertical slicing at each successive level
    - called “slice-and-dice”
  - use area and color to encode other variable of data cases

 copyright ©2025 m.patrignani

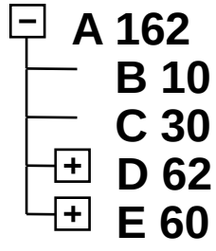
Slice-and-dice example





 copyright ©2025 m.patrignani

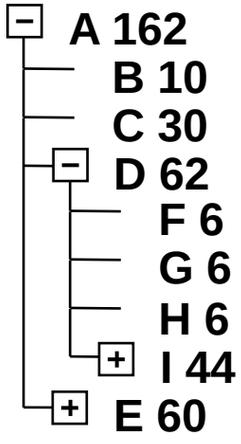
Slice-and-dice example

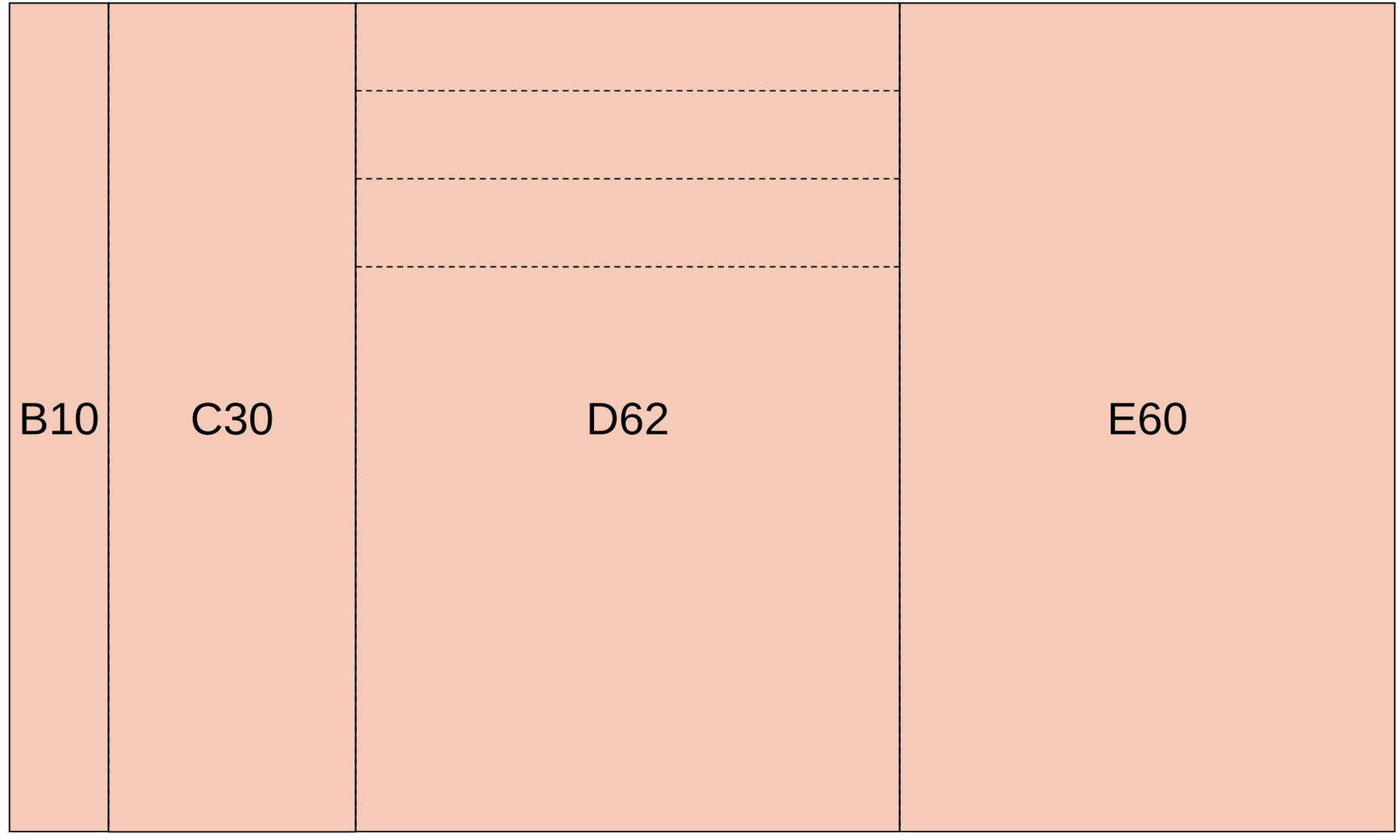




 copyright ©2025 m.patrignani

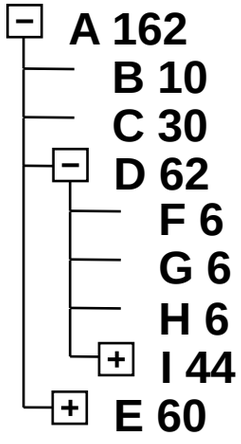
Slice-and-dice example

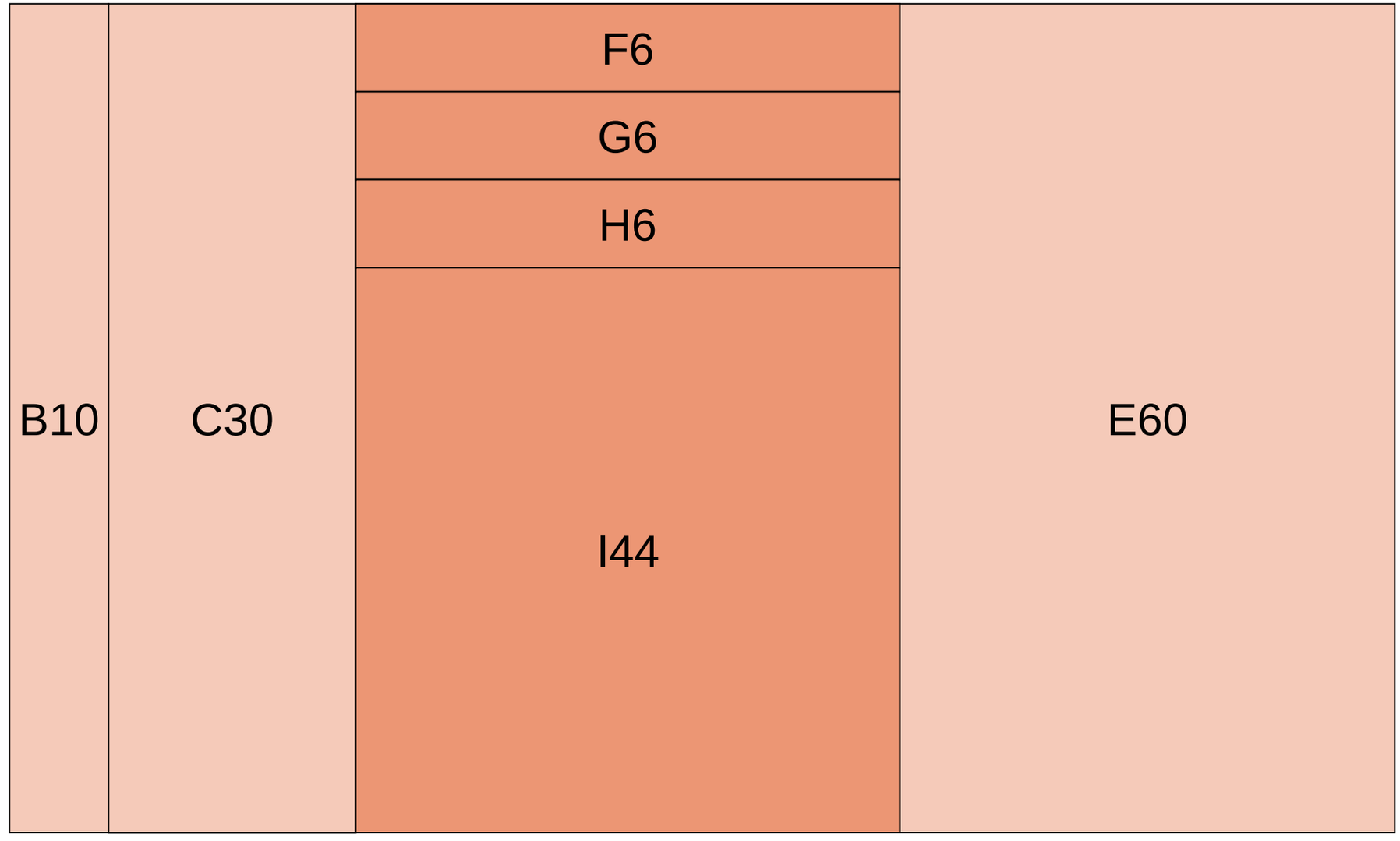




 copyright ©2025 m.patrignani

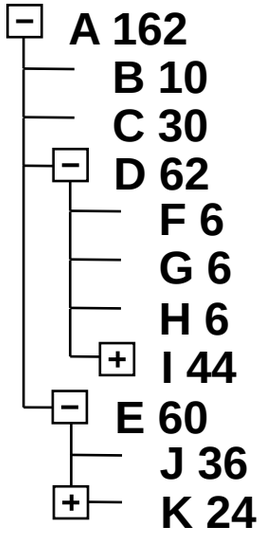
Slice-and-dice example

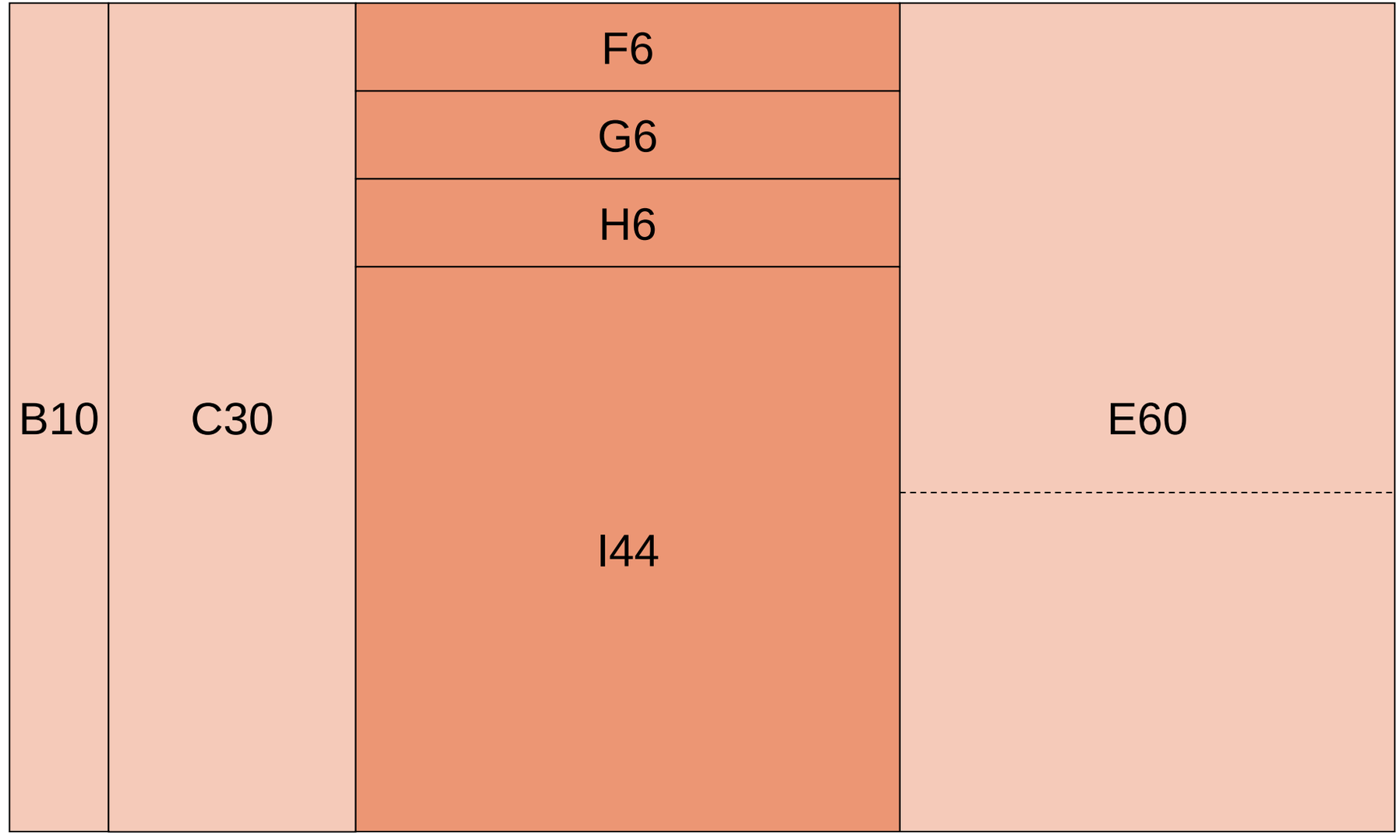




 copyright ©2025 m.patrignani

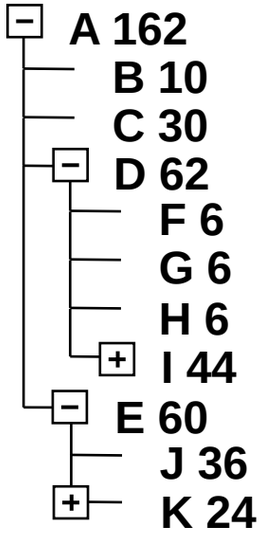
Slice-and-dice example

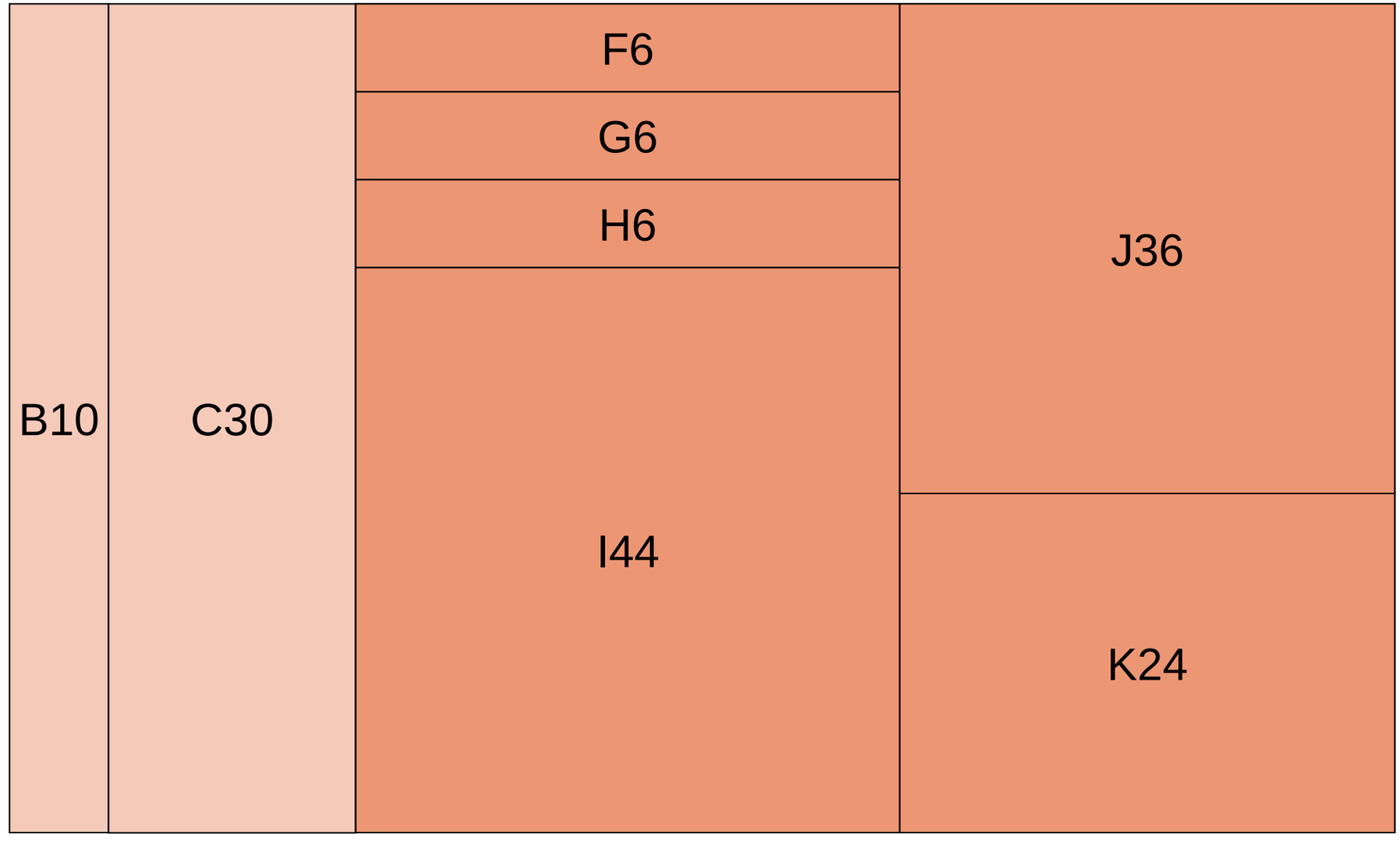




 copyright ©2025 m.patrignani

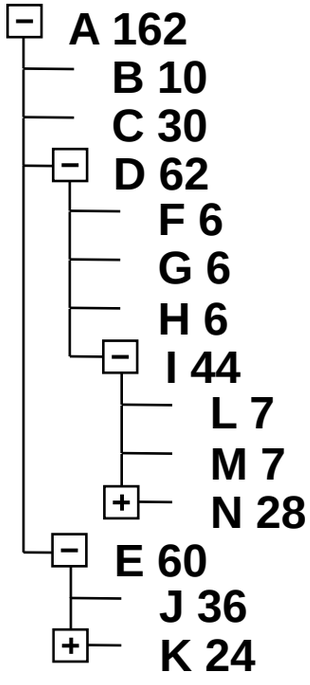
Slice-and-dice example

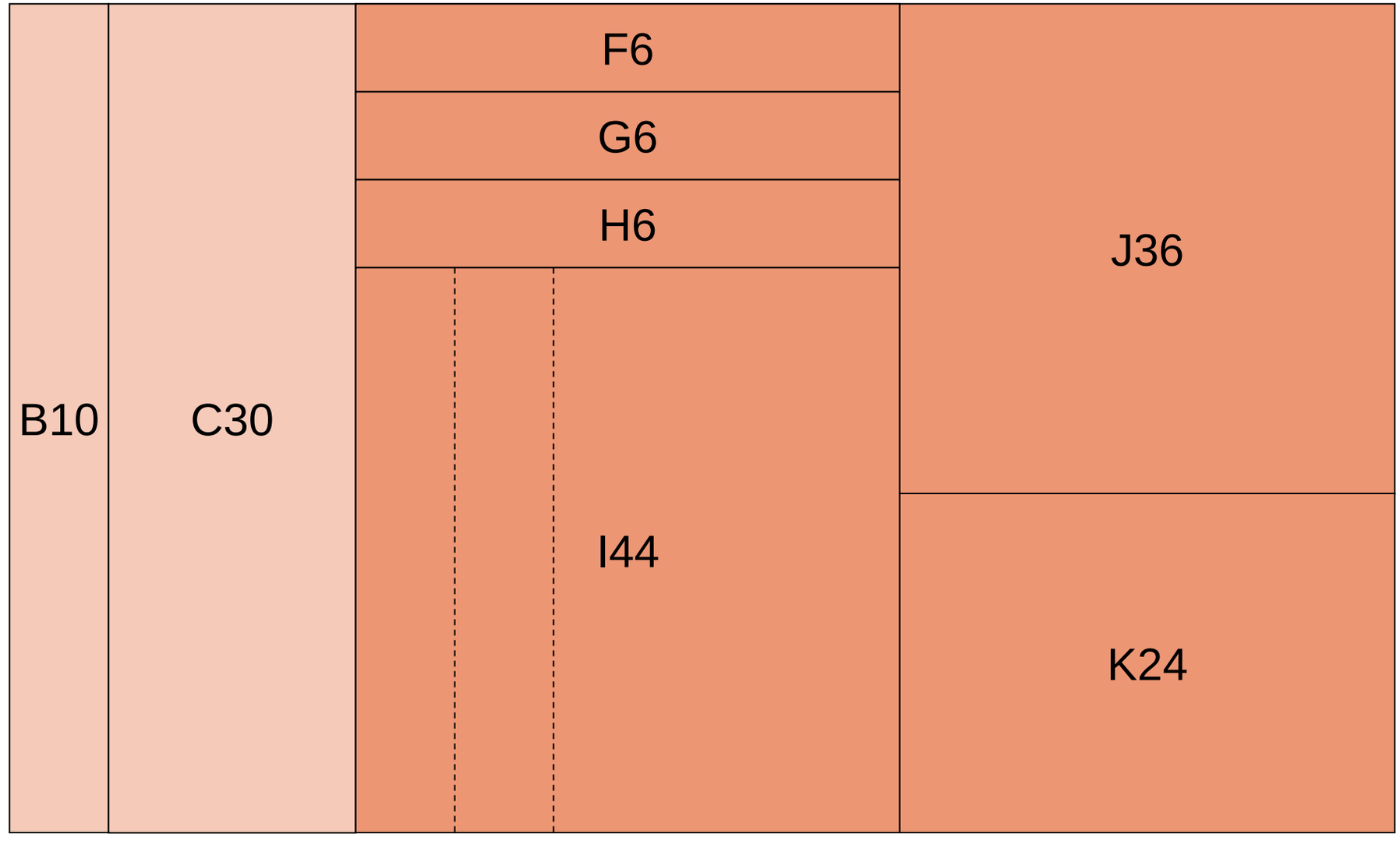




 copyright ©2025 m.patrignani

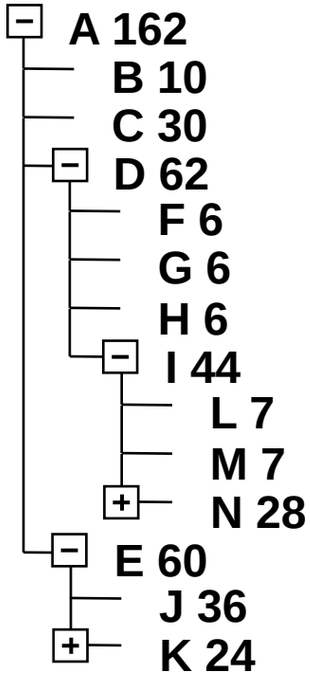
Slice-and-dice example

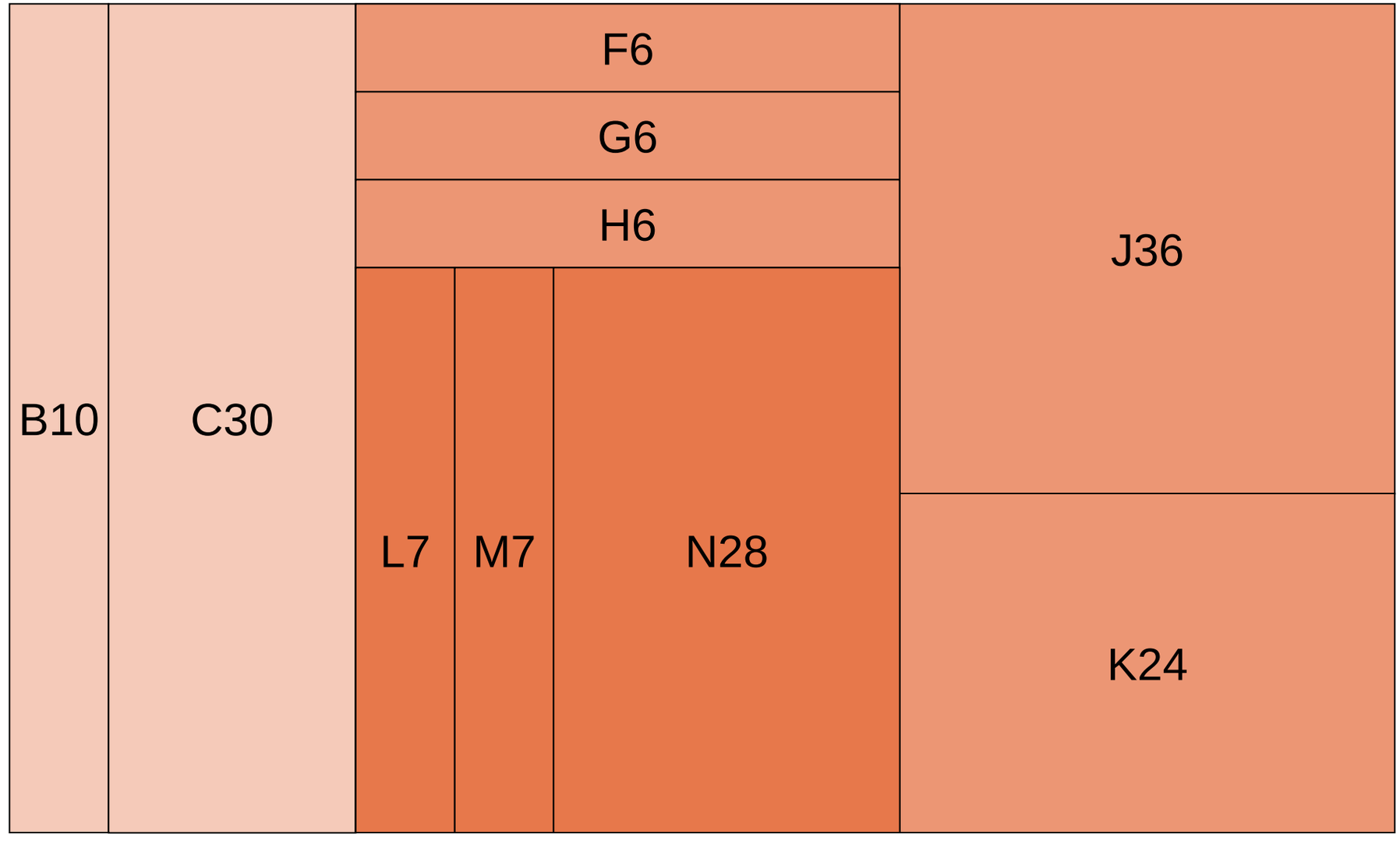




 copyright ©2025 m.patrignani

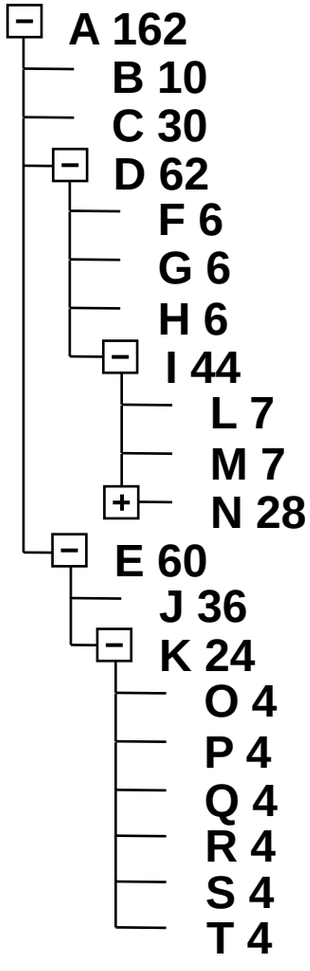
Slice-and-dice example

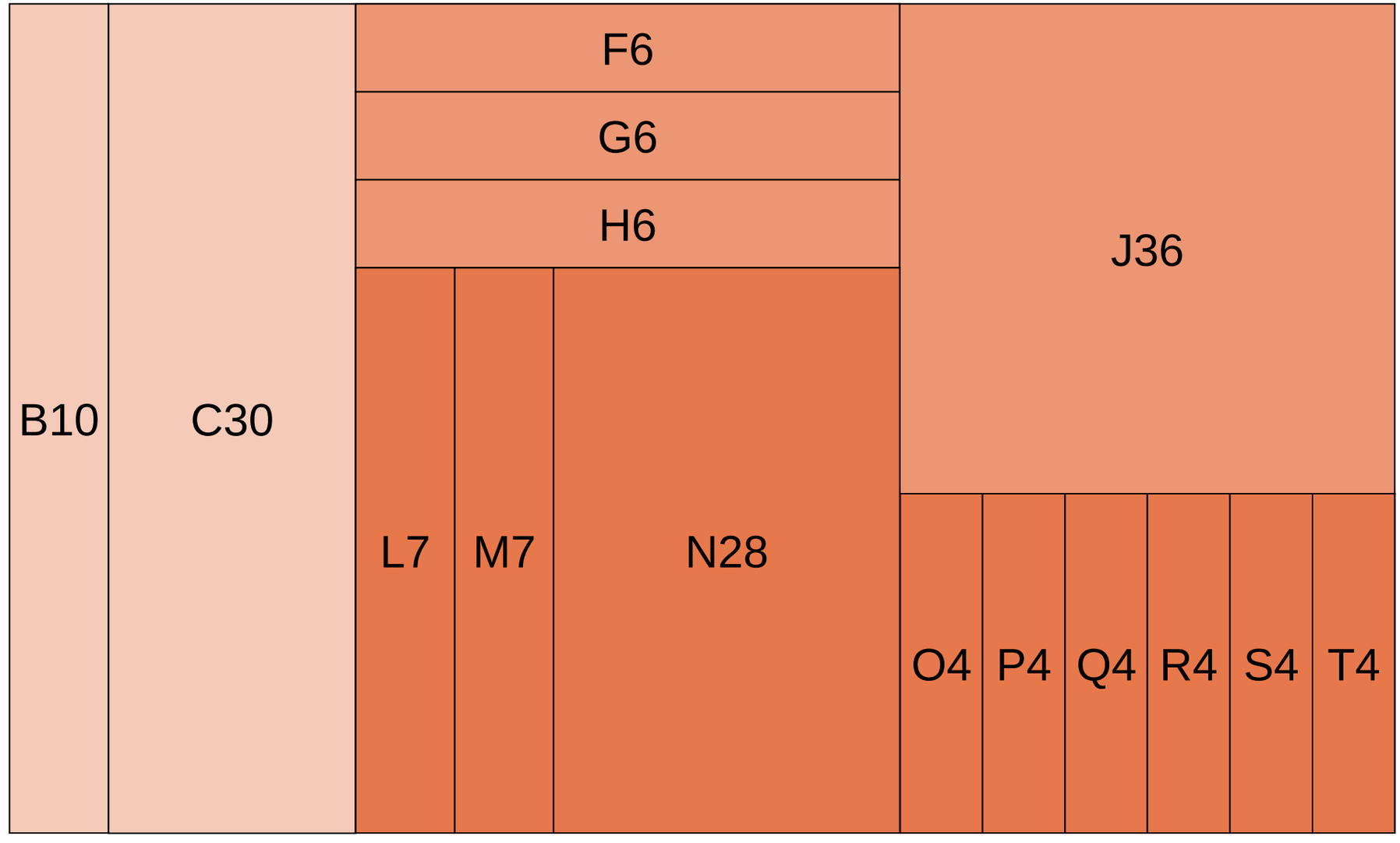




 copyright ©2025 m.patrignani

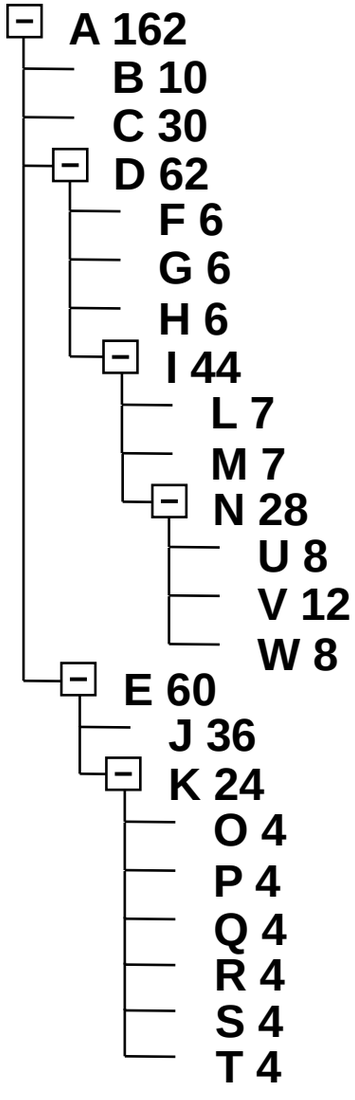
Slice-and-dice example

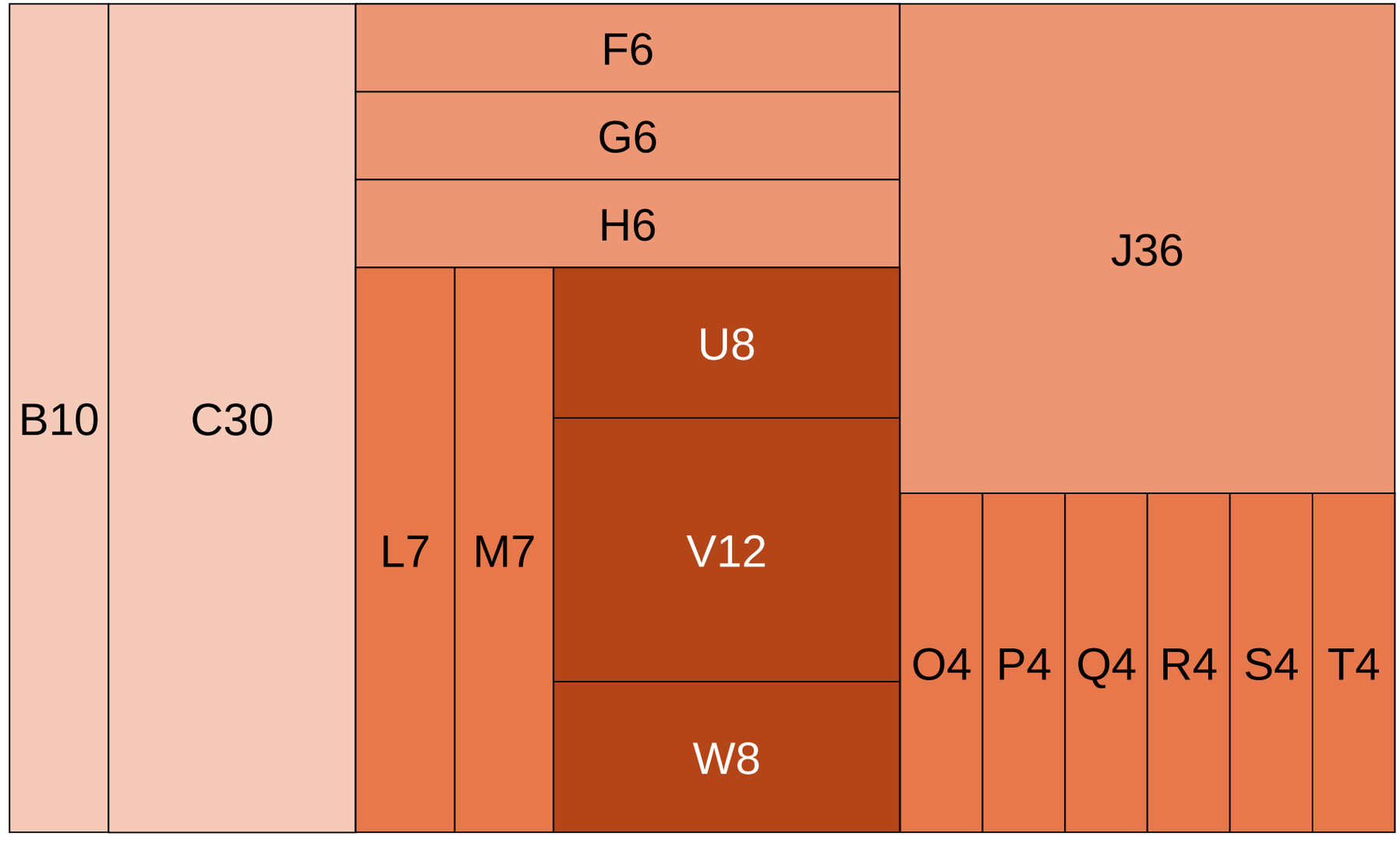




copyright ©2025 m.patrignani

Slice-and-dice example





 copyright ©2025 m.patrignani

Treemap algorithm

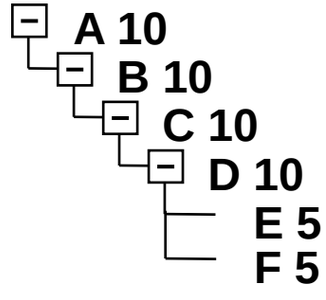
```
Treemap_Draw
/* Input: subtree, available rectangular area,
          orientation (horiz/vert) */
/* Purpose: fills the available area with a treemap
            drawing of the subtree */

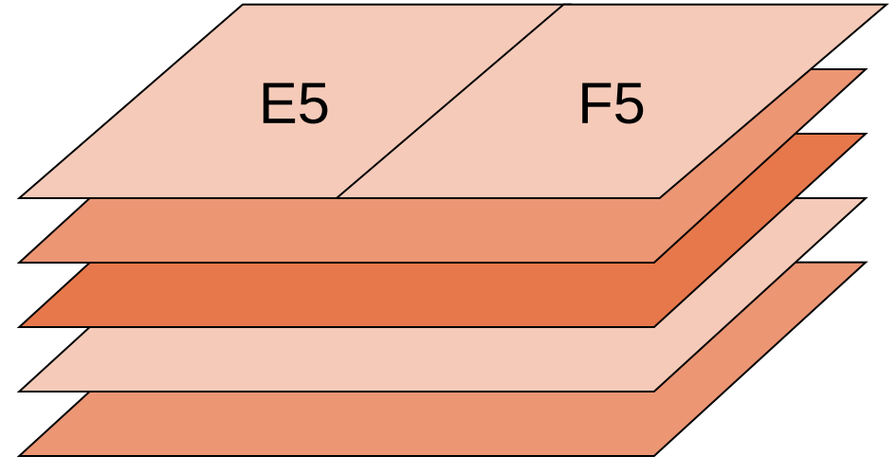
   If there are no children
      Draw rectangle in the available area with
         suitable color, label, etc.
   Else
      Subdivide available area among children based
         on their size and current orientation
      Make recursive call on subtrees swapping
         horiz/vert orientation
```

 copyright ©2025 m.patrignani

Treemaps limitations

- Only the leaves are visible
  - labels, colors, and borders of internal nodes are blotted
- Chains of degree-one nodes give an ambiguous representation

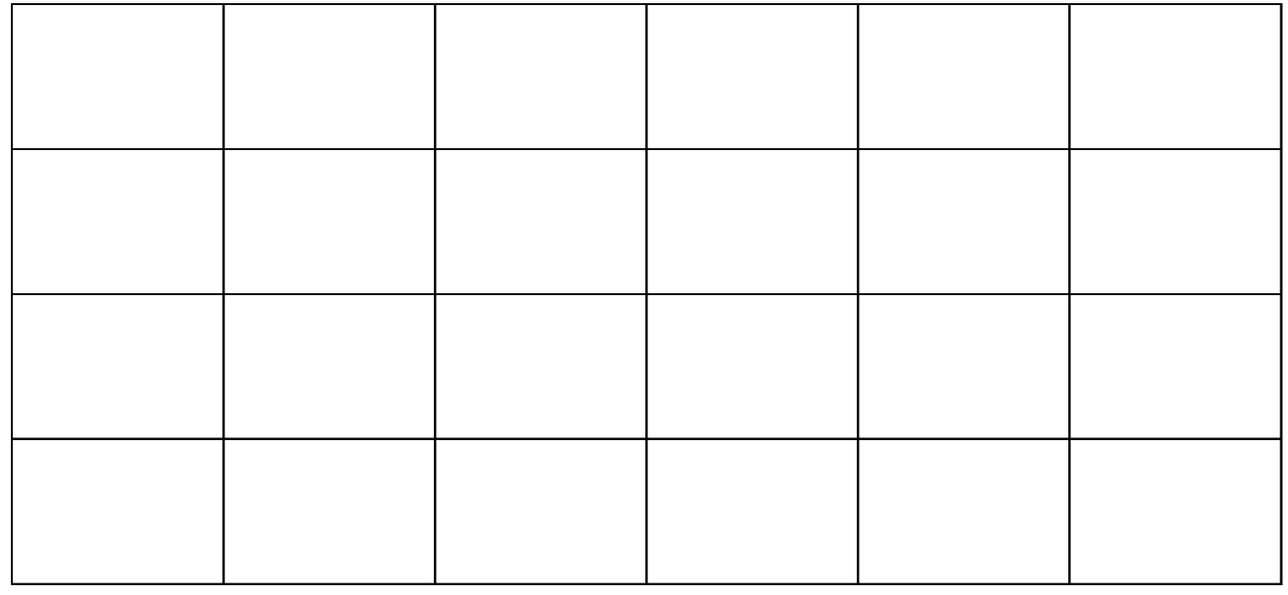




 copyright ©2025 m.patrignani

Border detection problem

- It is sometimes difficult to detect borders
- This is especially true when the tree is balanced
  - each child of the root has the same size and the same number of children



 copyright ©2025 m.patrignani

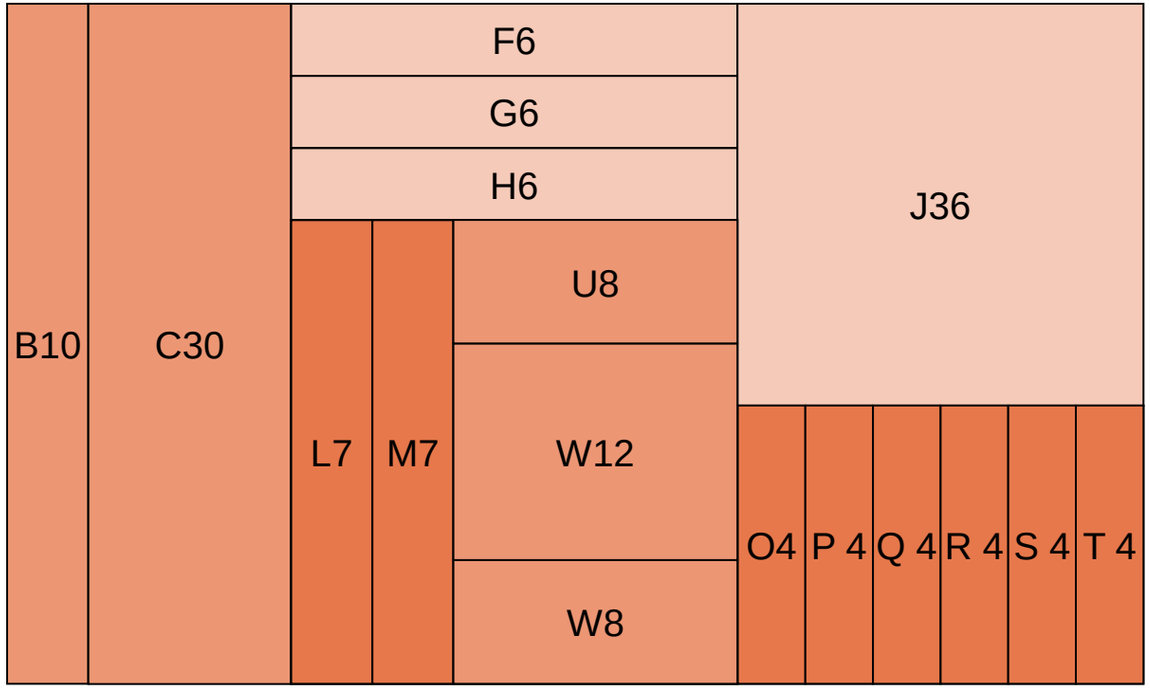
High degree vertices problem

- Many small files in some directories
- Interaction
  - show only a blackened area that invites a closer examination by zooming

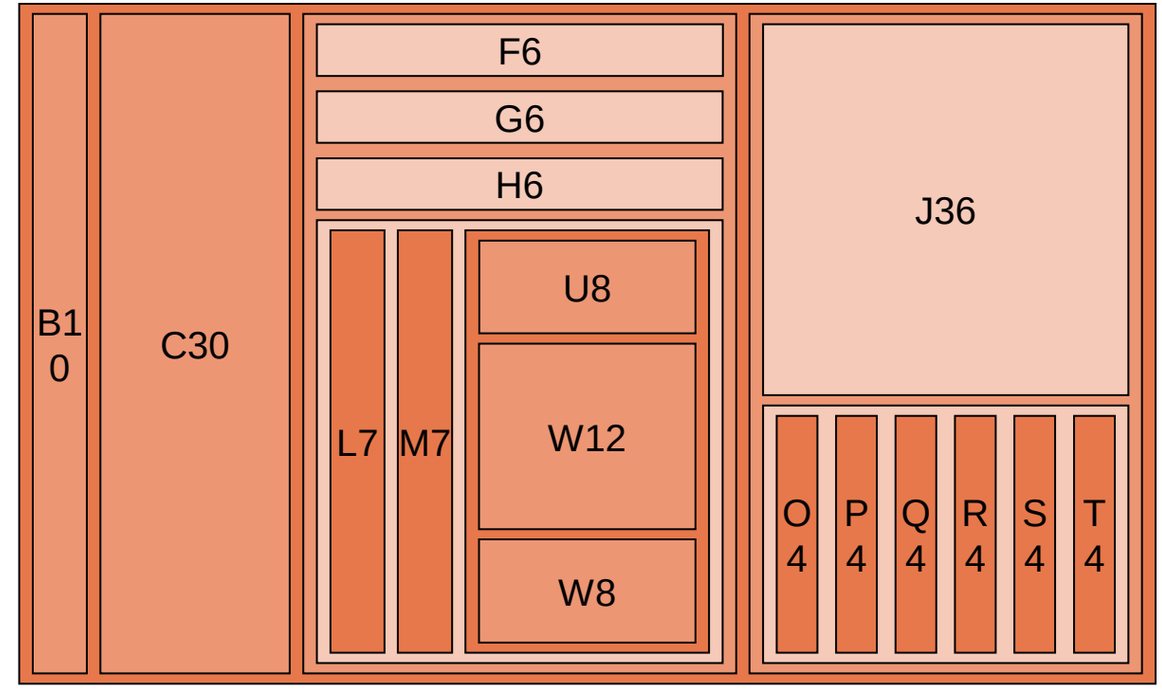
 copyright ©2025 m.patrignani

Nested treemaps

- Leave the border of each rectangle visible
  - The nesting could be large enough to host labels



Treemap

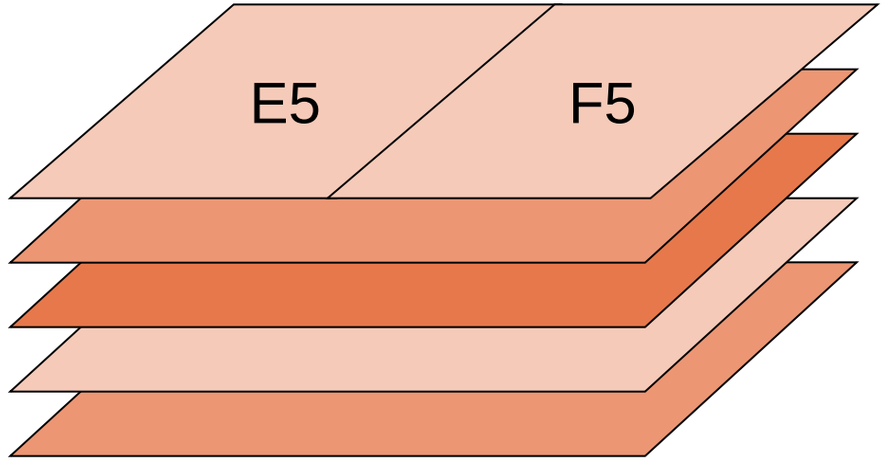


Nested treemap

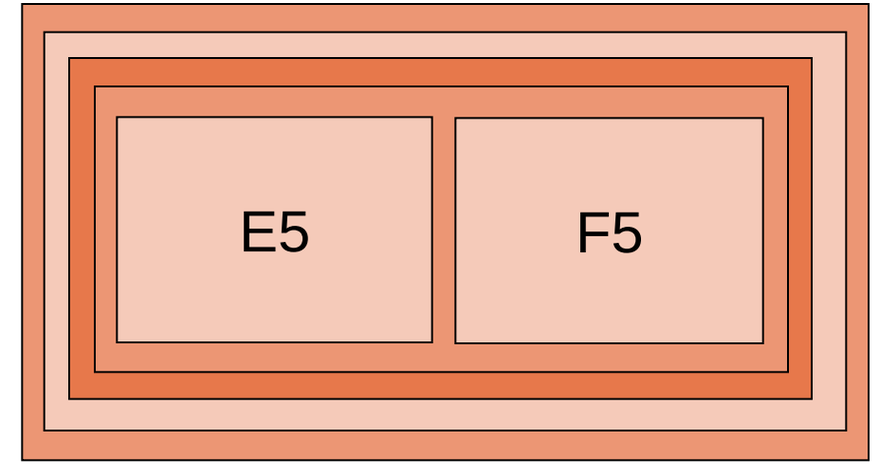
copyright ©2025 m.patrignani

Nested treemaps are not ambiguous

- When you have chains of degree-one nodes



Tree structure

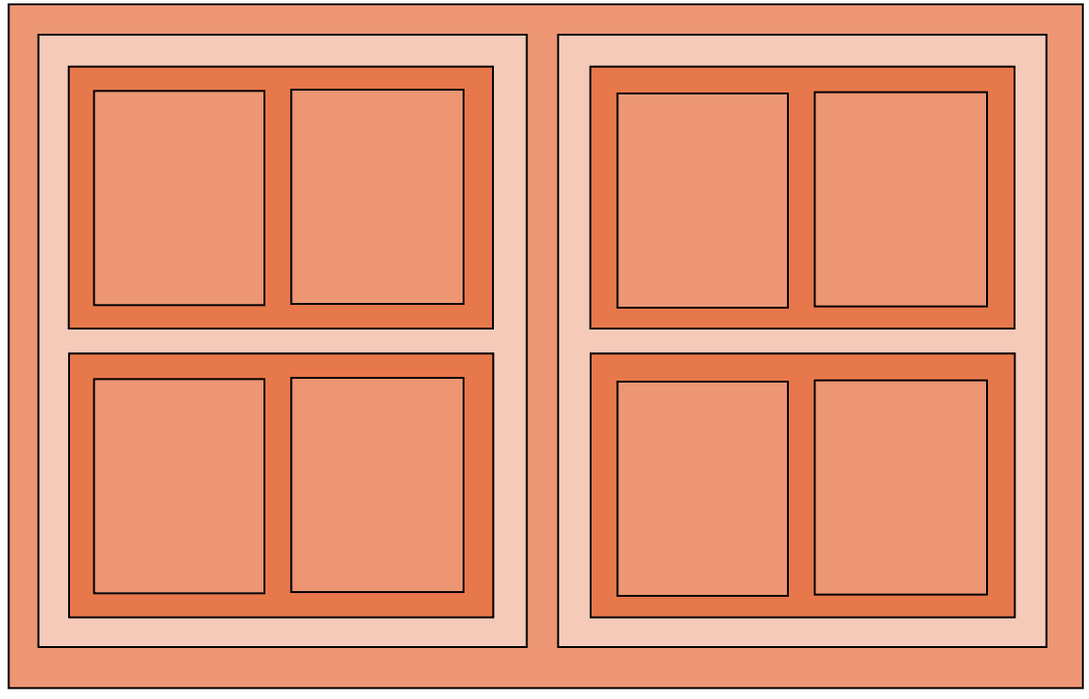


Nested treemap

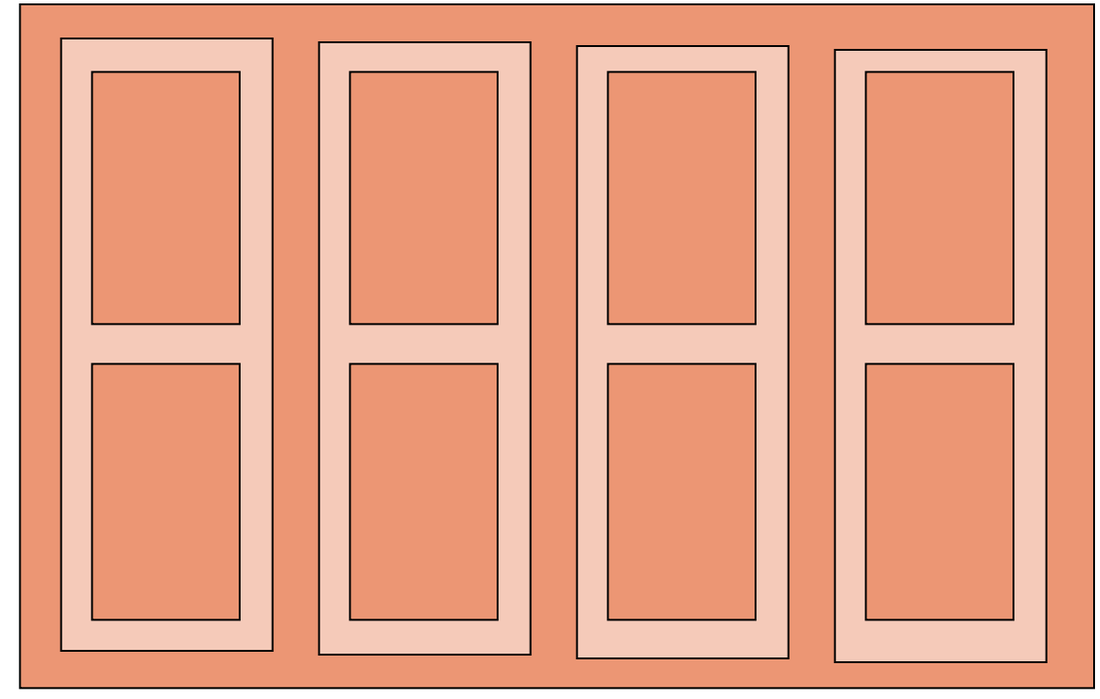
 copyright ©2025 m.patrignani

Nested treemaps are not ambiguous

- When you have balanced trees



A balanced tree



Another balanced tree

copyright ©2025 m.patrignani

Nested treemap algorithm

```
Nested_Treemap_Draw (knowing a feasible borderwidth)
/* Input: subtree, available area, borderwidth, orientation */
/* Purpose:    fill the available area with a nested
               treemap drawing of the subtree */

    If there are no children
        Draw rectangle in available area
    Else
        Reduce the available width and height by 2 *
    borderwidth    /* border of current node */
        If orientation == horizontal
            Reduce the available width by
            borderwidth * (number of children -1)
        Else    /* orientation == vertical */
            Reduce the available height by
            borderwidth * (number of children -1)
        Subdivide available area among children
        Make recursive call on subtrees
```

 copyright ©2025 m.patrignani

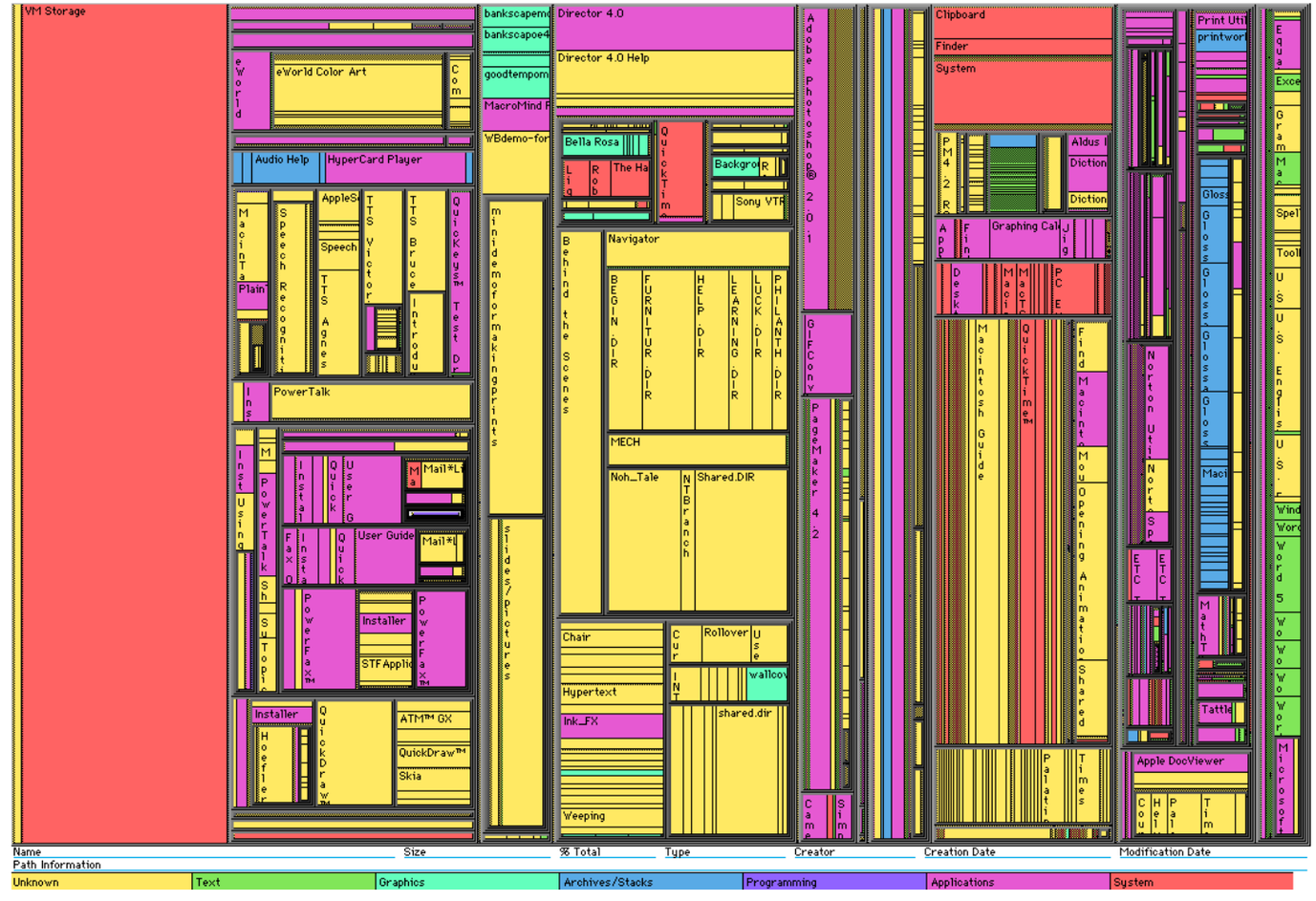
If borderwidth is not know *a priori*

- Modify the algorithm leaving borderwidth as a parameter
  - the available area is a function of borderwidth
  - at each recursive call you either subtract borderwidth or divide by a constant the current width and heigth
  - the available width and heigth are linear functions of borderwidth
    - `width = a - b * borderwidth`
    - `height = c - d * borderwidth`
- Compute for each rectangle the largest possible borderwidth that still allows a non-zero area
- Use the smallest of such borderwidths for all rectangles

 copyright ©2025 m.patrignani

Treeviz application

- File system exploration tool
  - size of rectangles is proportional to the file size

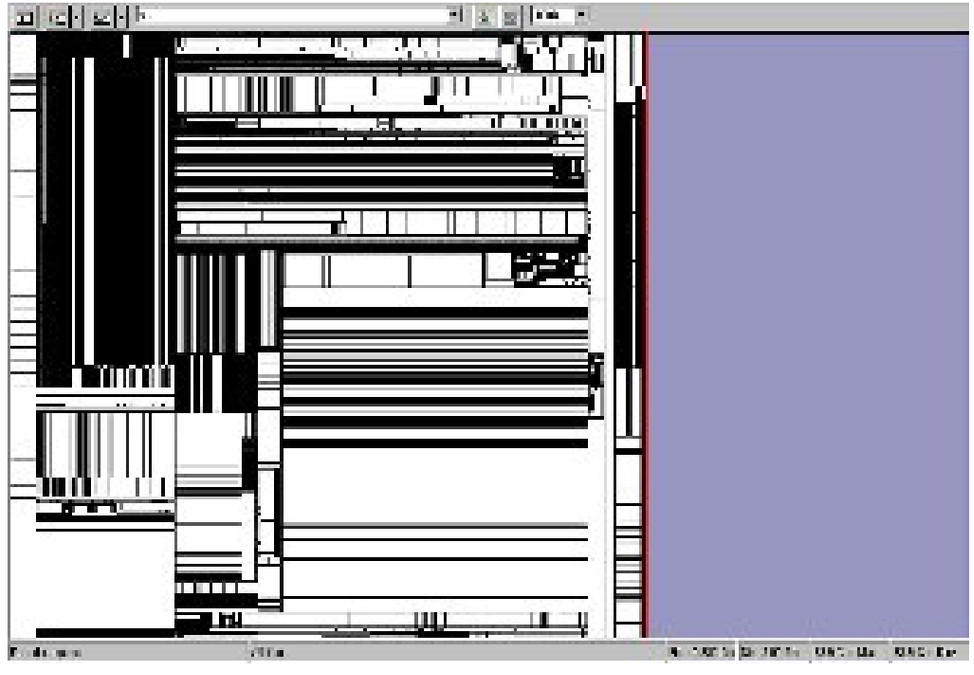


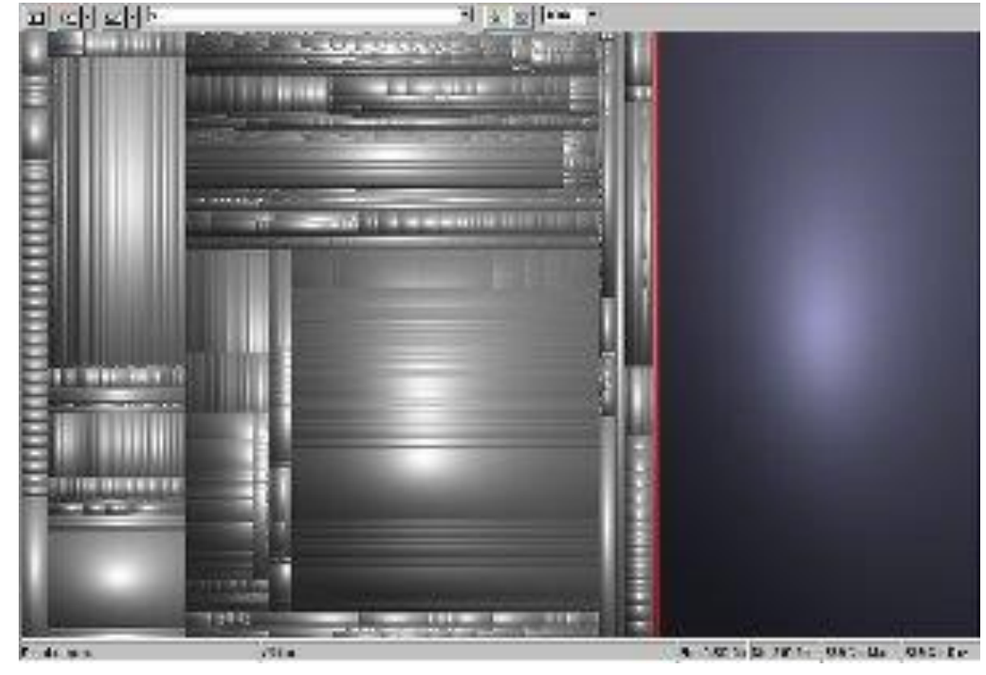
http://www.cs.umd.edu/hcil/pubs/treeviz.shtml

copyright ©2025 m.patrignani

Cushion treemaps

- Introduced in [van Wijk & van de Wetering 99]
- Apply texture to the rectangles that make them appear like little, shiny cushions
  - lines are no longer necessary to separate nodes
  - separation at difference height of the hierarchy can be perceived

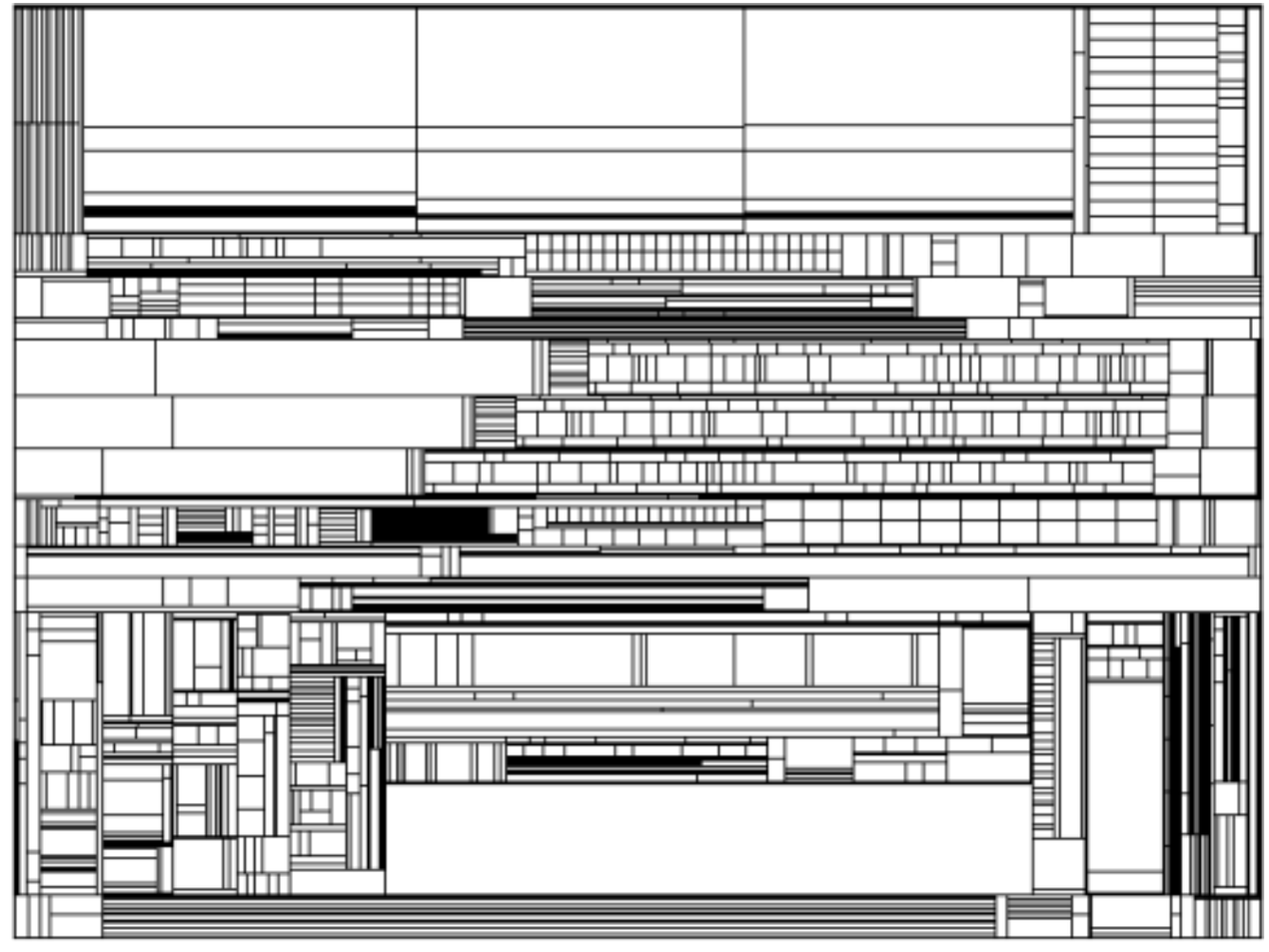




copyright ©2025 m.patrignani

Examples

- File system with 1400 files

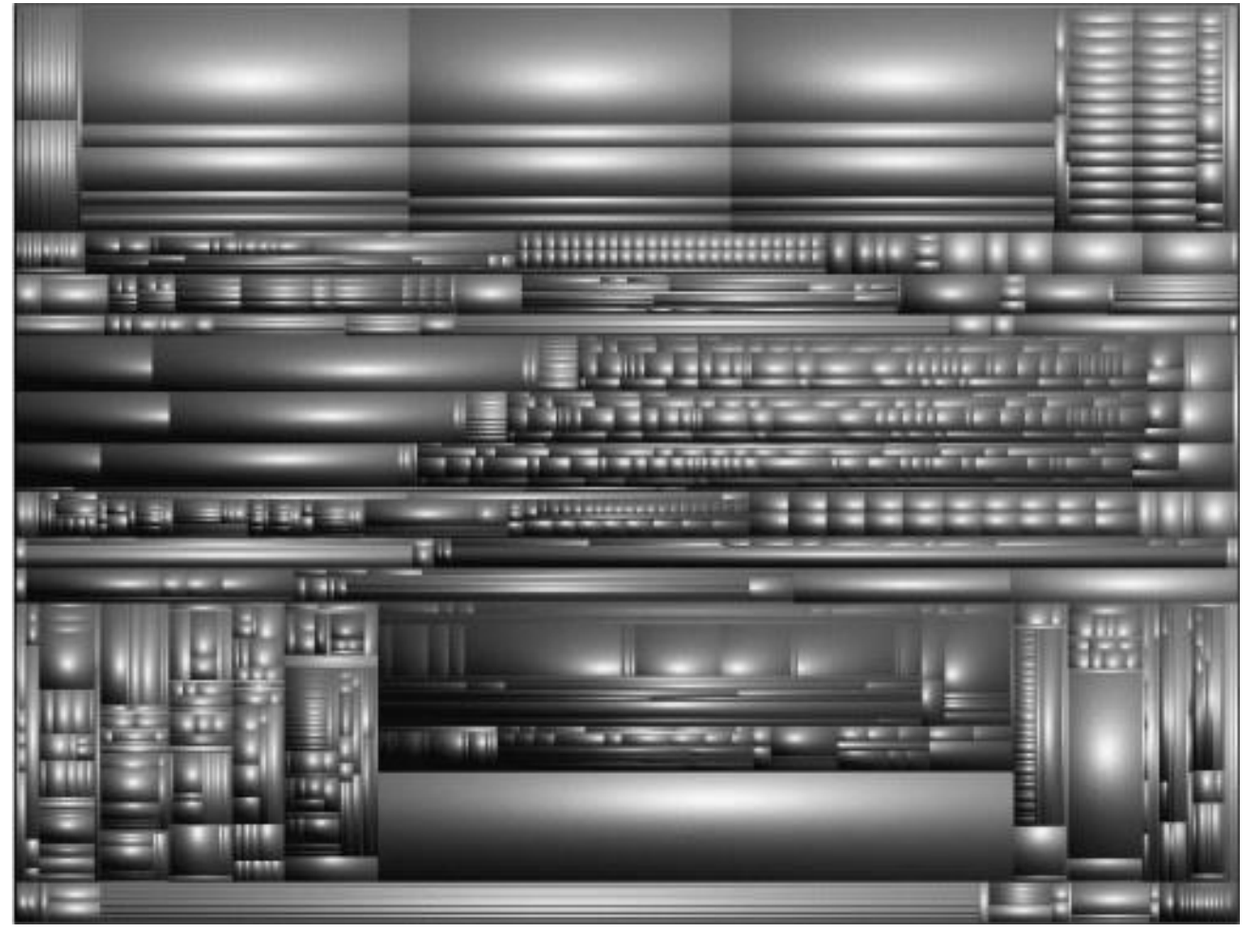


Traditional slice-and-dice

 copyright ©2025 m.patrignani

Examples

- File system with 1400 files

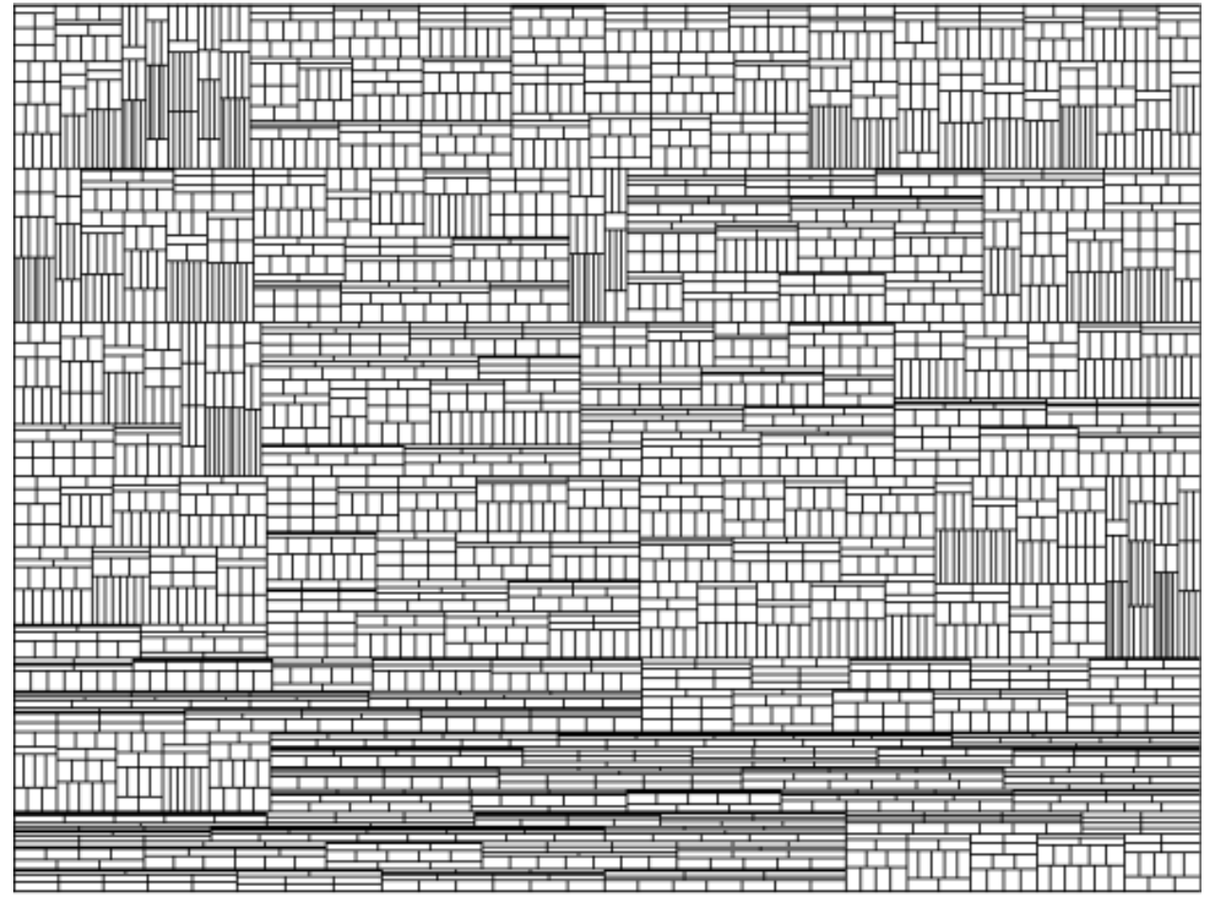


Cushion treemaps

 copyright ©2025 m.patrignani

Examples

- Artificial organization chart (six levels and 3060 employees)

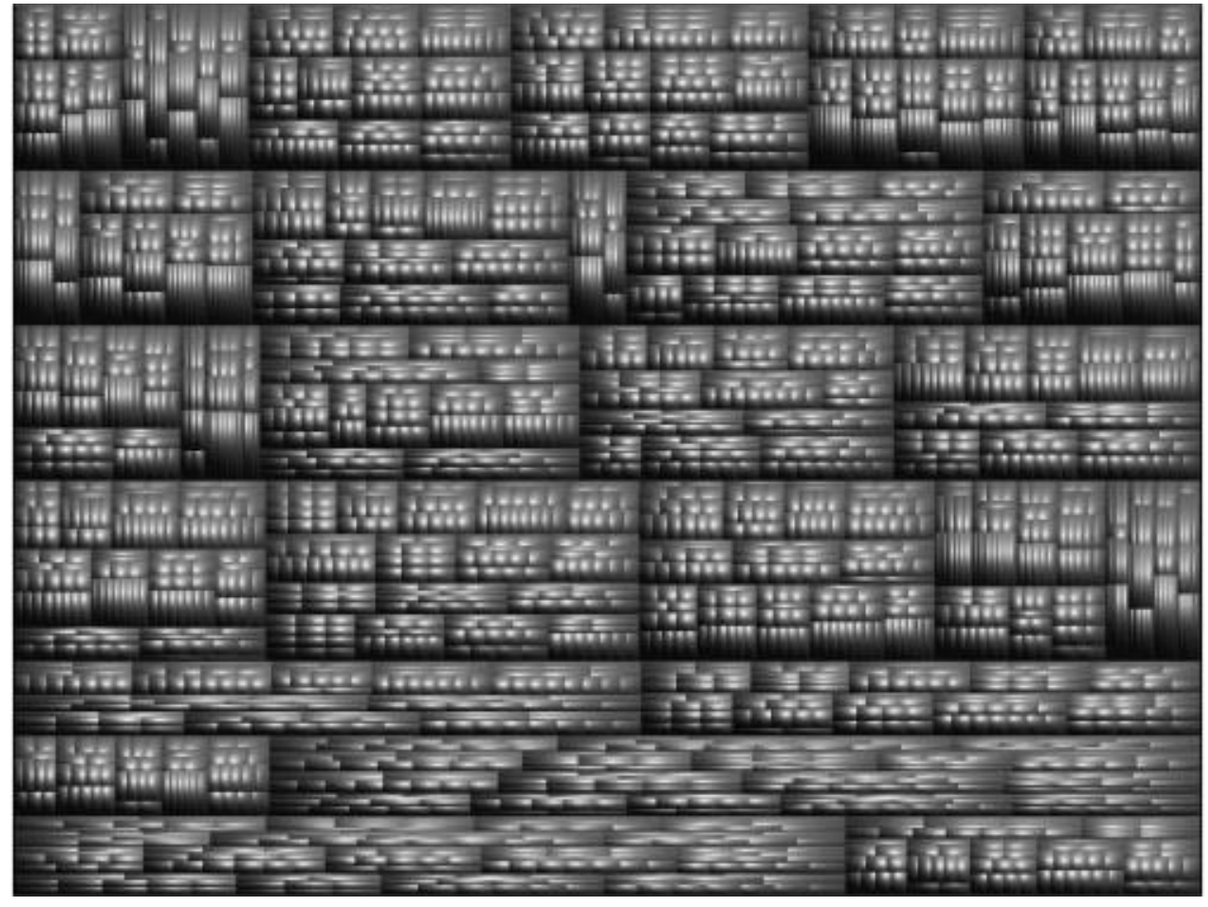


Traditional slice-and-dice

 copyright ©2025 m.patrignani

Examples

- Artificial organization chart (six levels and 3060 employees)

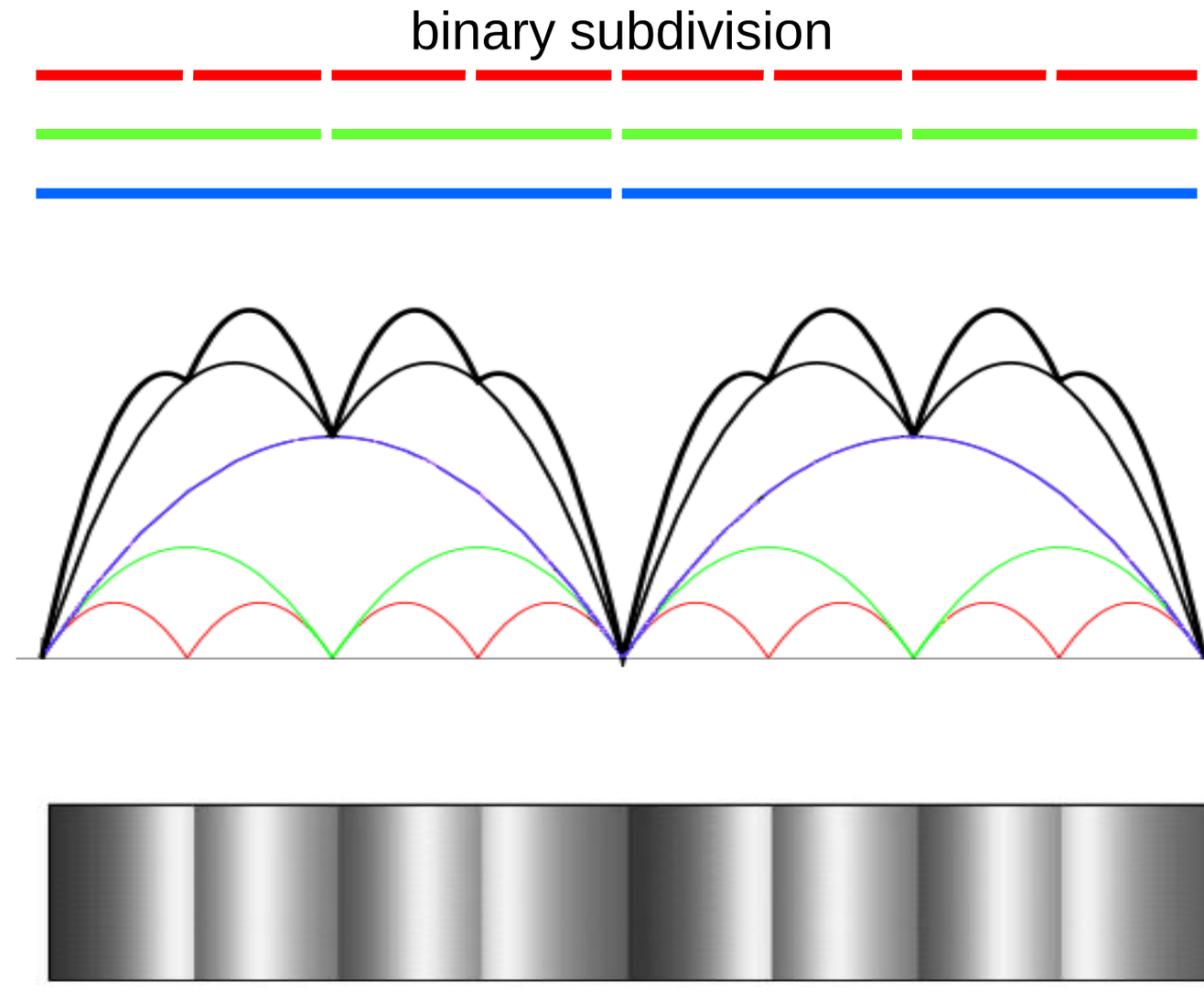


Cushion treemaps

 copyright ©2025 m.patrignani

Cushion treemaps algorithm

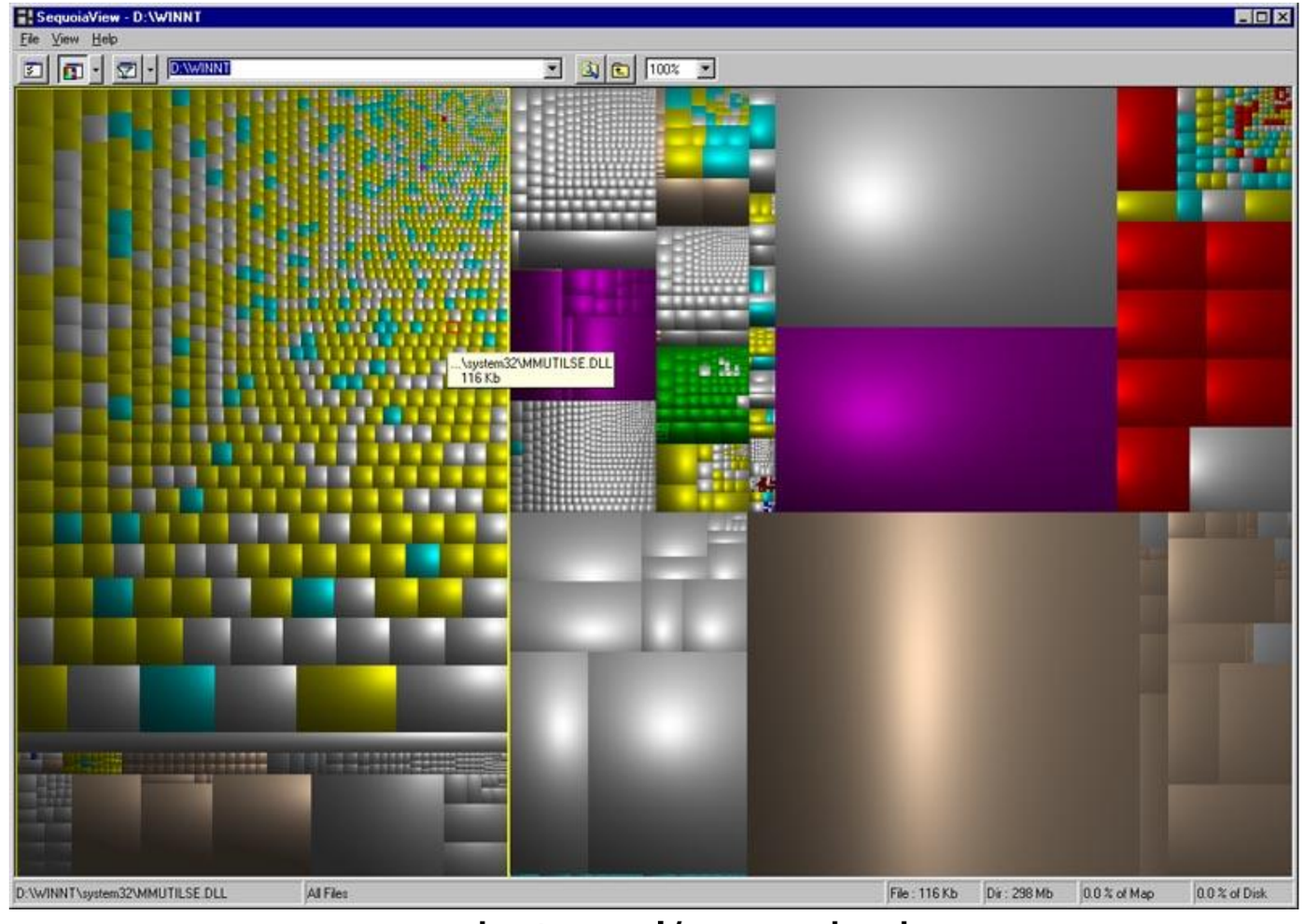
- Consider the unidimensional version of the problem
  - replace each segment with a bump
  - do this recursively
  - sum up all bumps
- Generalize to 2D



 copyright ©2025 m.patrignani

SequoiaView

- File system exploration tool (Windows)

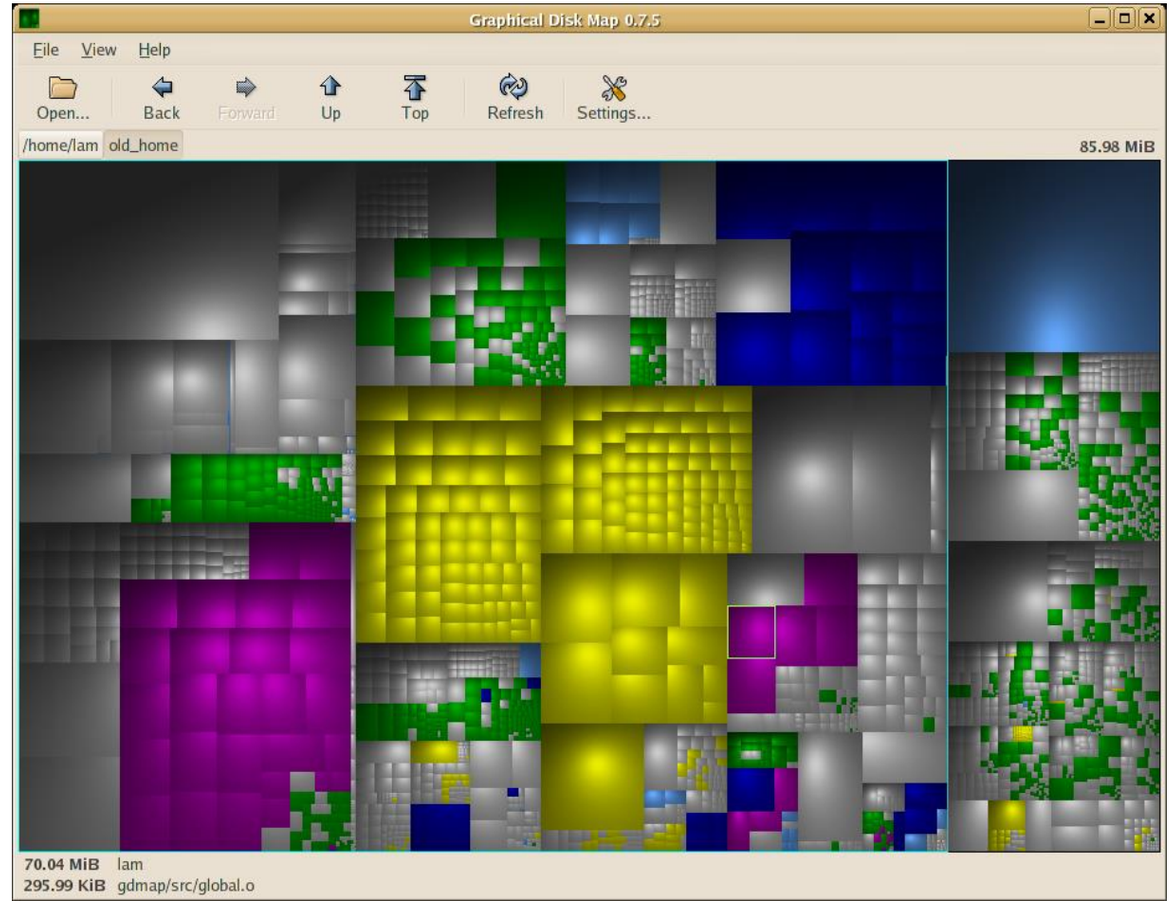


www.win.tue.nl/sequoiaview

 copyright ©2025 m.patrignani

GDMap

- File system explorer (Linux)

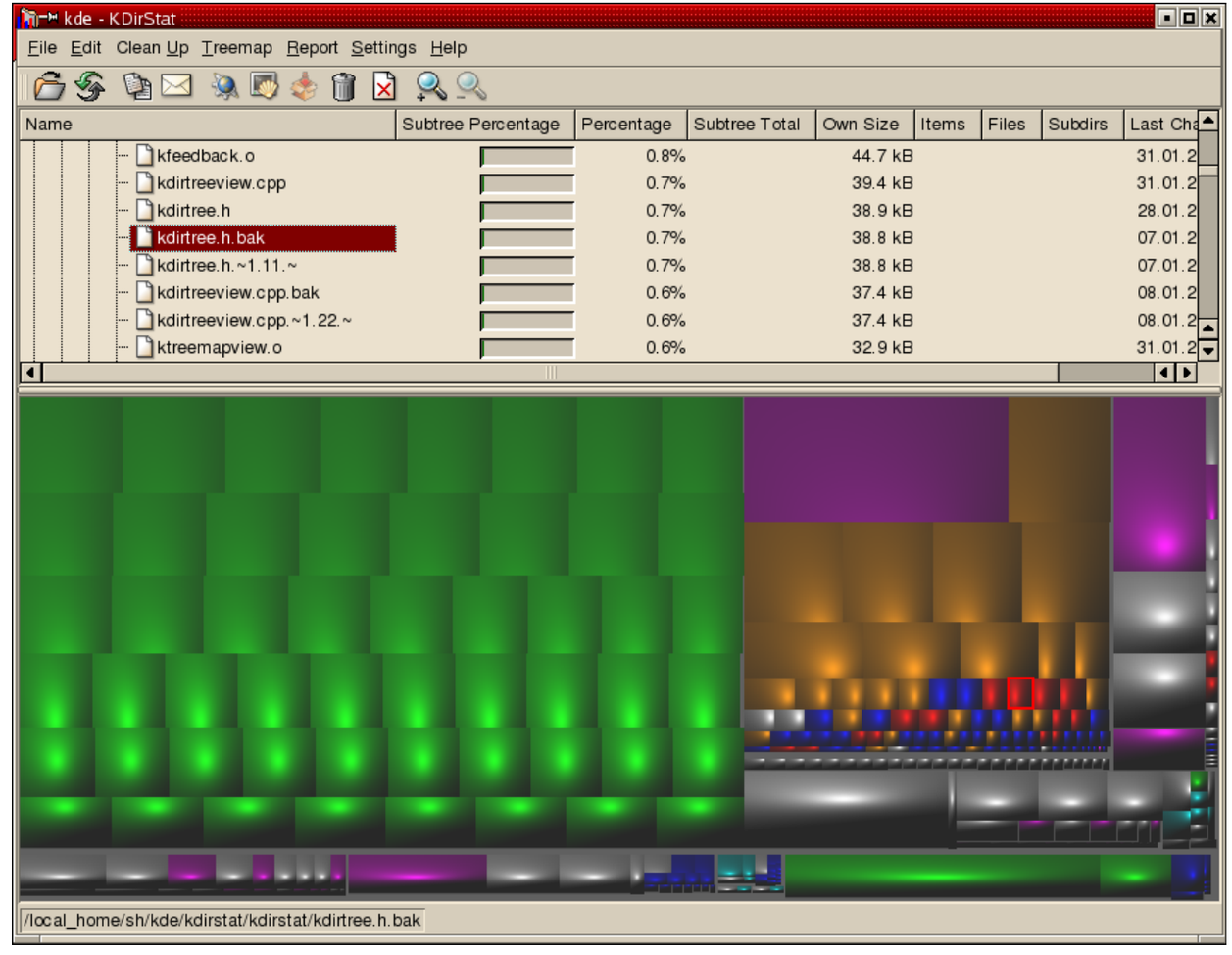


http://gdmap.sourceforge.net/

 copyright ©2025 m.patrignani

Kdirstat

- File system explorer (Linux)

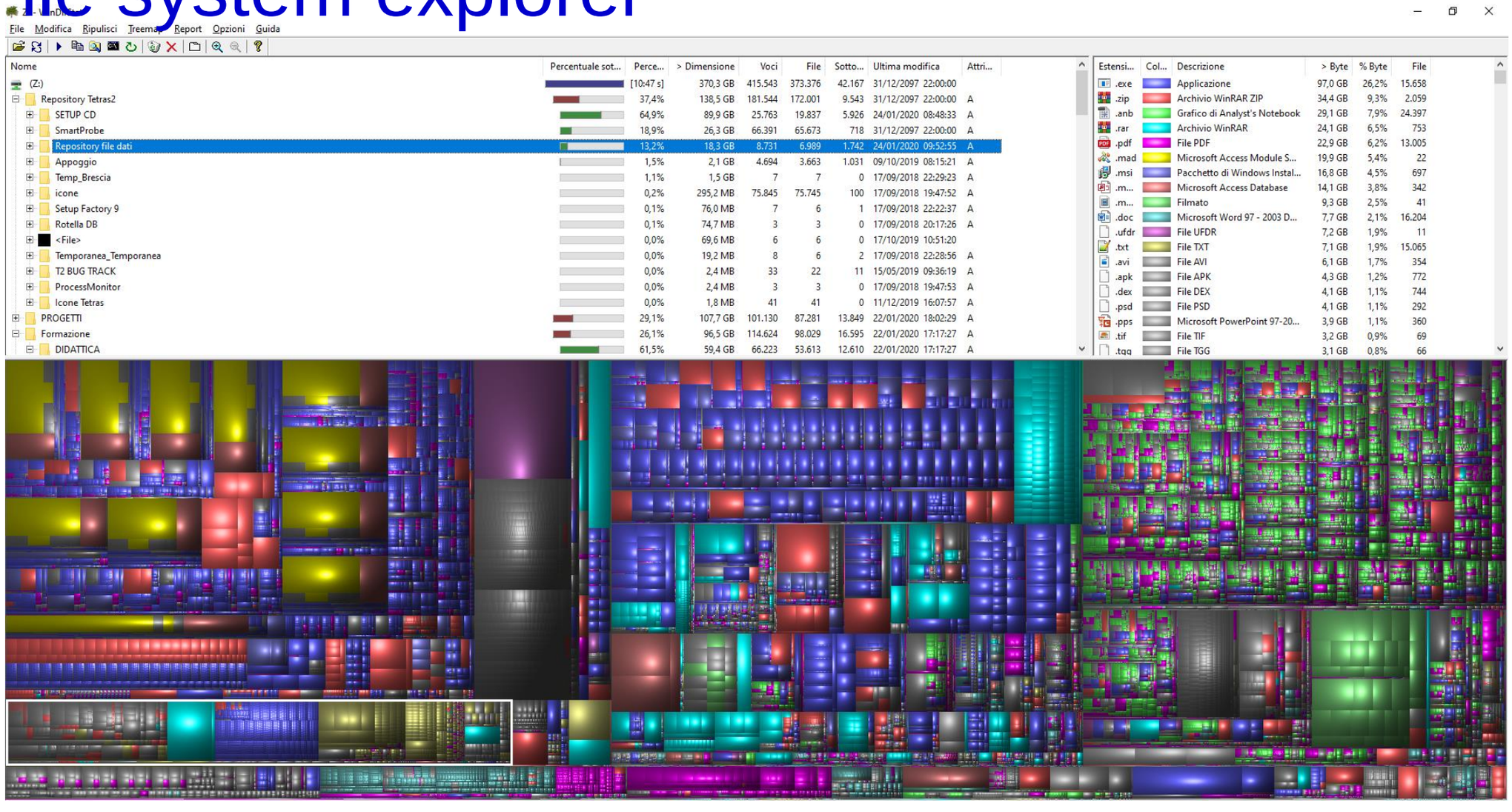


http://kdirstat.sourceforge.net/kdirstat.html

 copyright ©2025 m.patrignani

WinDirStat

- File system explorer



 copyright ©2025 m.patrignani

Aspect ratio problem

- Thin rectangles are visually unappealing

unappealing thin rectangle

- Visually comparing long narrow rectangles to squarish ones is problematic

which is bigger?

which is bigger?

- Interaction
  - hovering over the rectangles may produce the exact area value on the bottom of the display
- Change the layout algorithm?

 copyright ©2025 m.patrignani

Minimum aspect ratio treemap

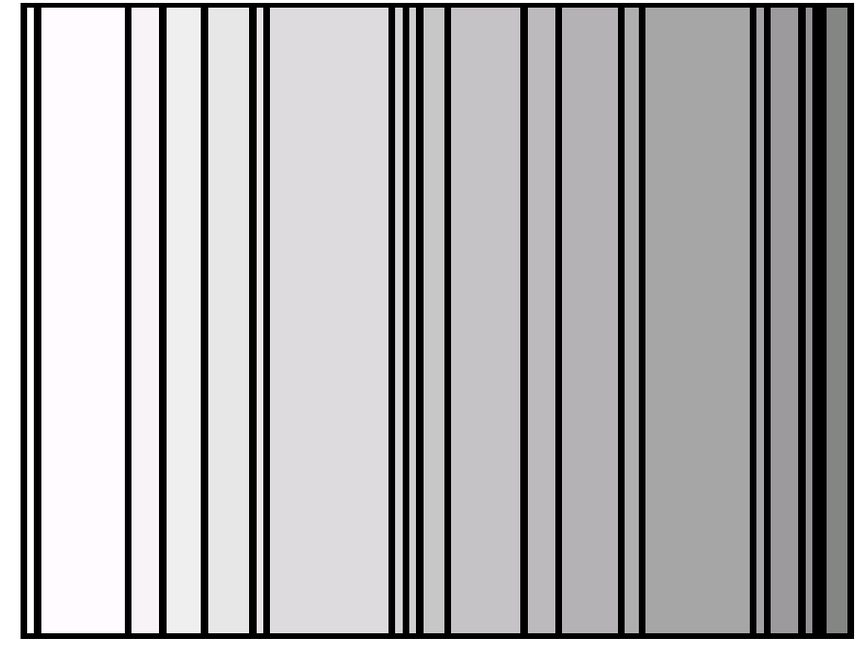
Input	A tree T and a rectangle R
Output	A treemap representation of T into R
Target	Reduce the value of the worst aspect ratio in the representation

- Shown to be strongly NP-hard
  - [van der Weele 2010]
  - remains NP-hard even if the tree is a star
  - remains NP-hard even if R is a square

copyright ©2025 m.patrignani

Slice-and-dice algorithm

- The original treemap algorithm splits the available rectangle into smaller rectangles
- The order of the rectangles is preserved but the rectangles may be very thin
  - the shade of grey represents the order of the children

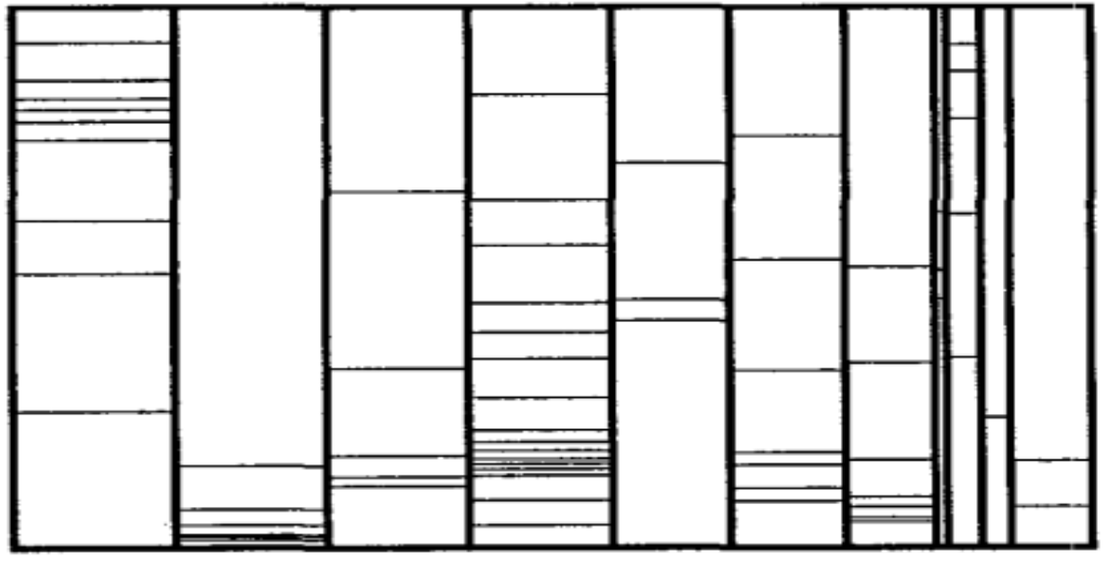


Picture from
[Shneiderman & Wattenberg 01]

 copyright ©2025 m.patrignani

Cluster treemaps

- Introduced in [Wattenberg 99]
  - algorithm not documented
- Employs both vertical and horizontal partitions at each level of hierarchy
- Obtains a better aspect-ratio
- However, the order of the children is not preserved



Slice-and-dice

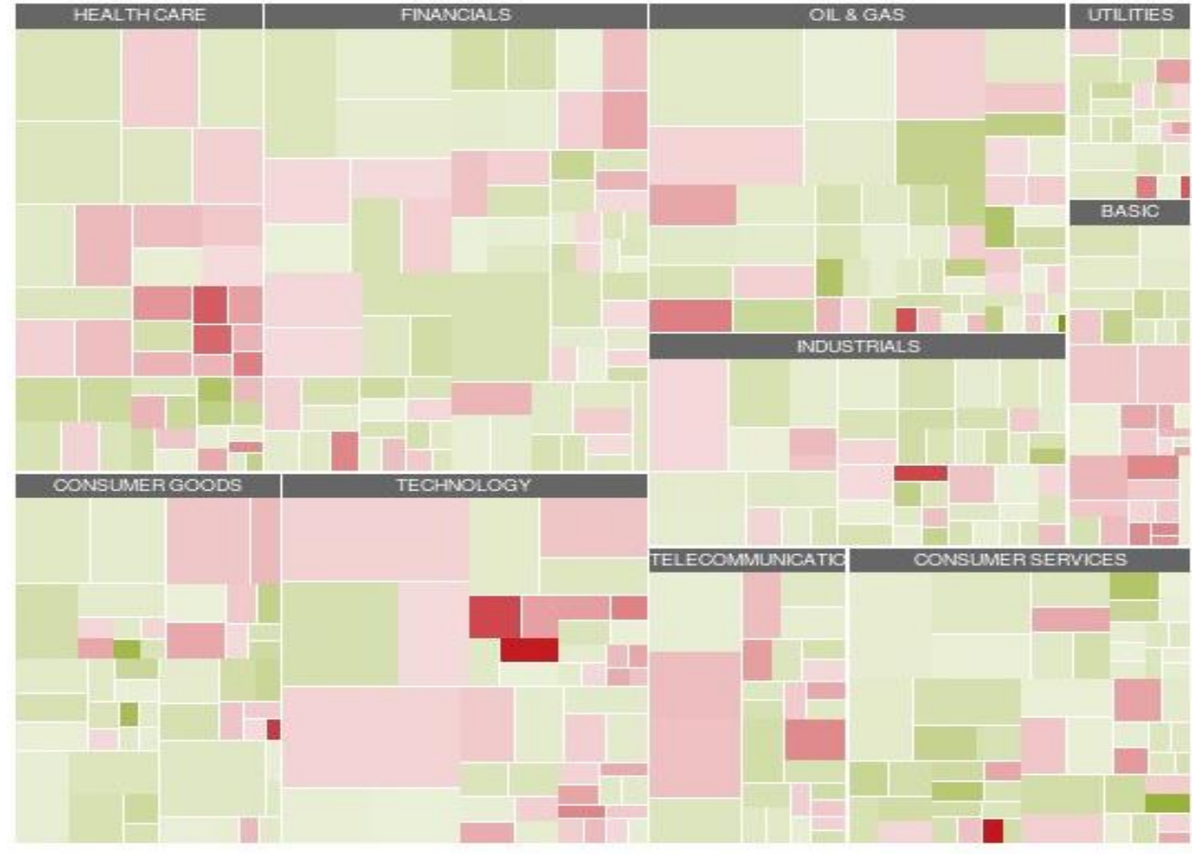


Cluster layout

copyright ©2025 m.patrignani

Map of the market

- Illustrates stock movements
- Avoids bad aspect ratios
- Deals with a shallow hierarchy

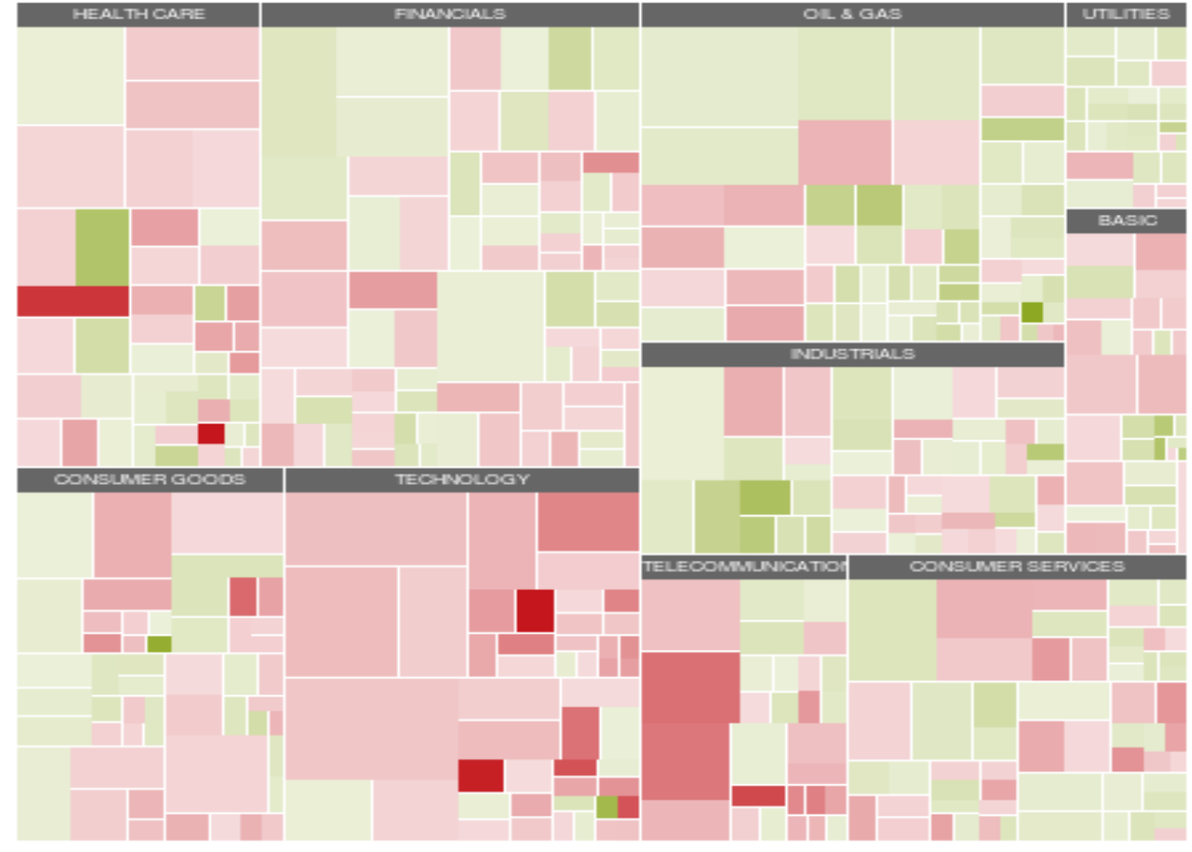


http://www.marketwatch.com/tools/stockresearch/marketmap
(no longer available)

copyright ©2025 m.patrignani

Map of the market

- Illustrates stock movements
- Avoids bad aspect ratios
- Deals with a shallow hierarchy
  - picture of a bad day

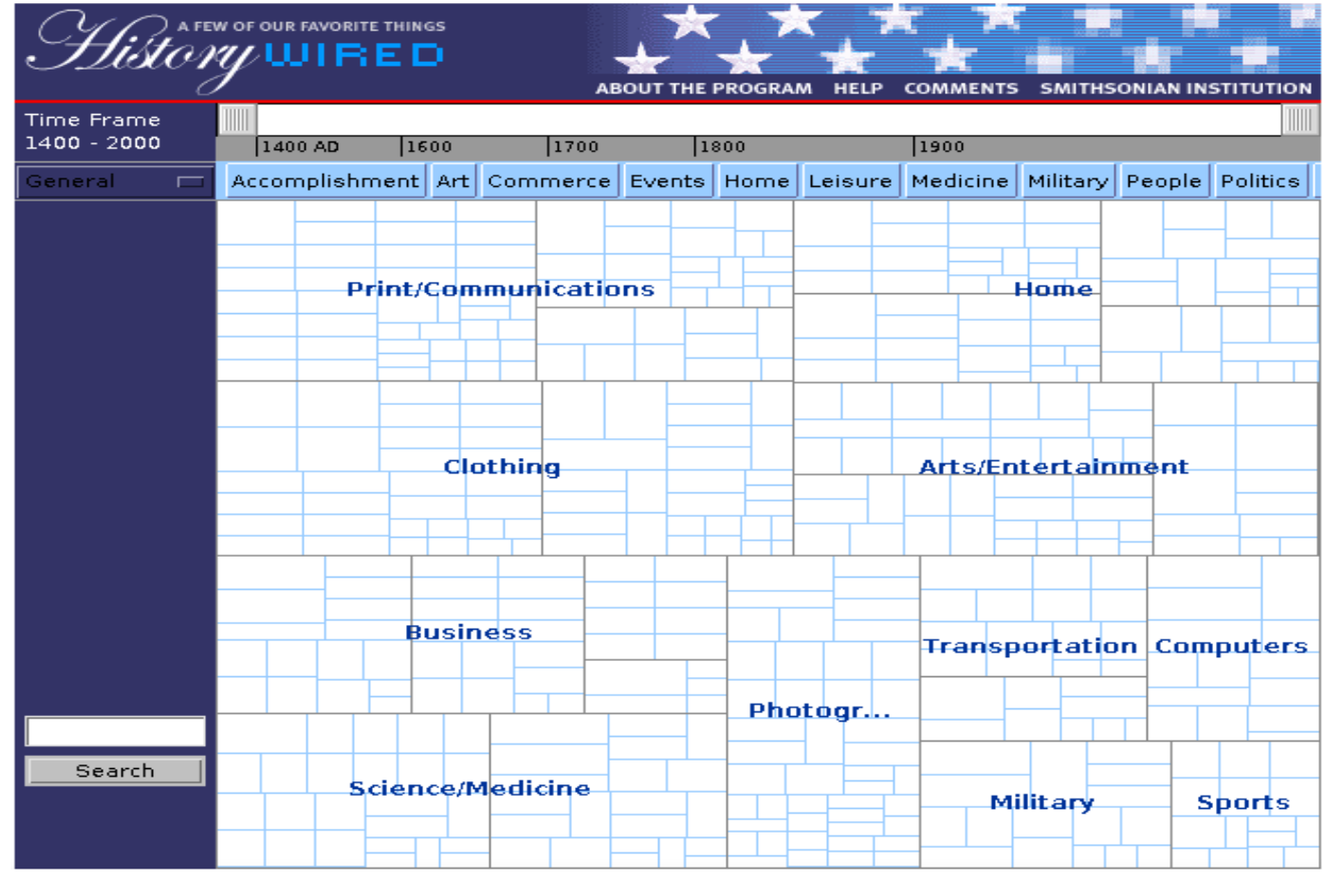


http://www.marketwatch.com/tools/stockresearch/marketmap
(no longer available)

copyright ©2025 m.patrignani

Map of the market

- Virtual tour of the National Museum of American History
  - Also uses the clustered treemap algorithm in [Wattenberg 99]



http://historywired.si.edu/ (no longer available)

copyright ©2025 m.patrignani

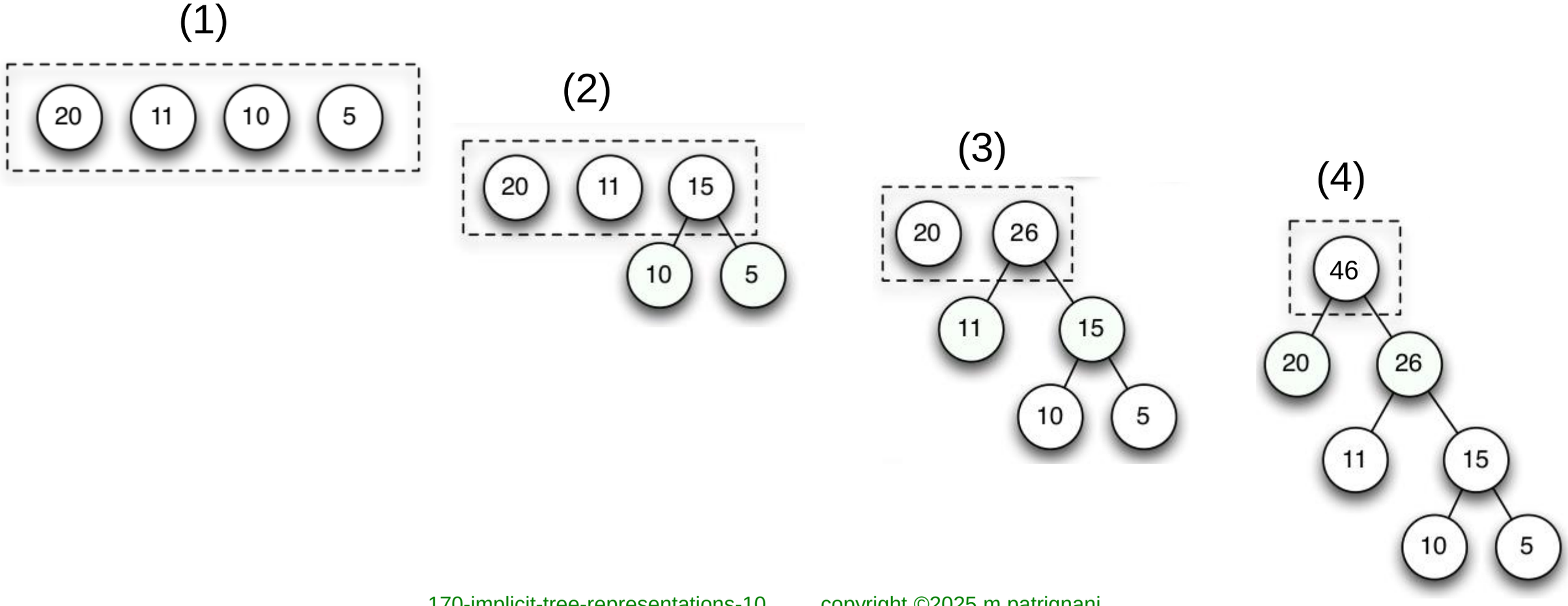
Binary treemaps

- The algorithm for drawing cluster treemaps has never been documented
- However cluster treemaps seem to be similar to a binary treemaps
- We will see the binary treemap algorithm on a living example
  - taken from [Heeringa & Jannen 16]
- Suppose the current node has four children of weight 20, 11, 10, and 5

 copyright ©2025 m.patrignani

For each parent, construct a binary tree

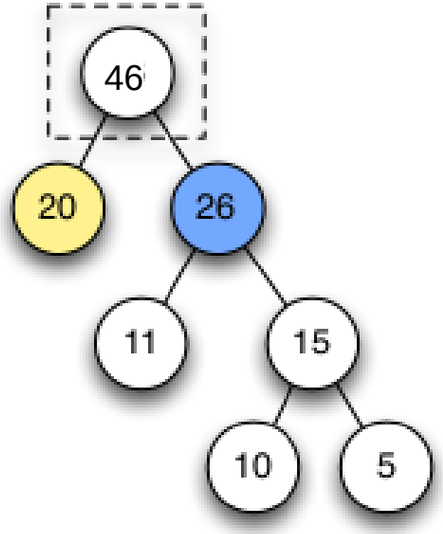
- Recursively join the subtrees with smaller weight

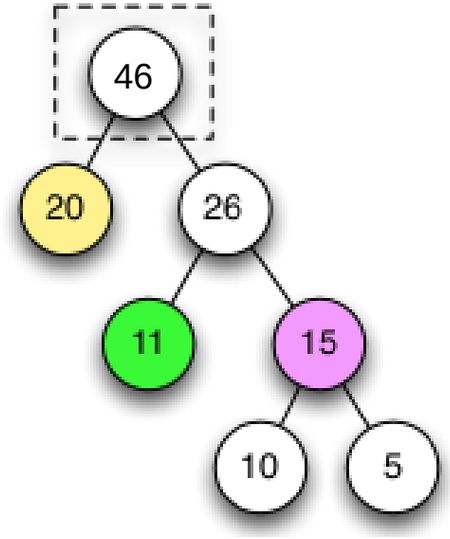


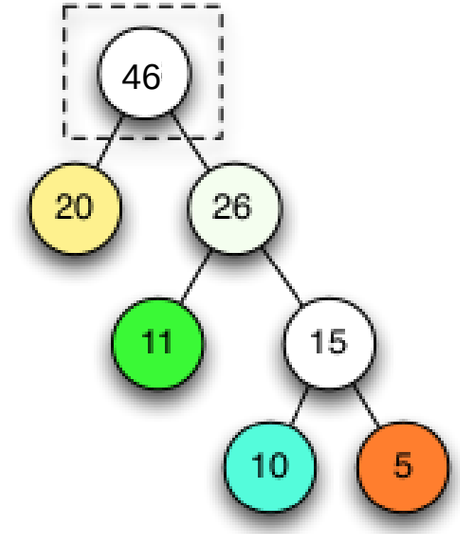
 copyright ©2025 m.patrignani

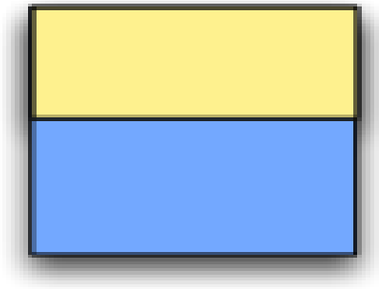
Drawing phase

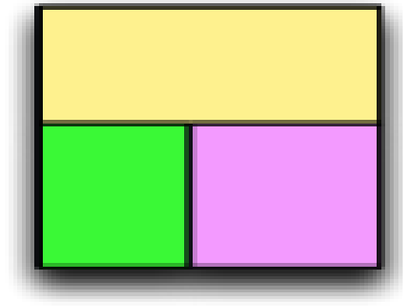
- Descend the binary tree alternating horizontal and vertical splits of the current area

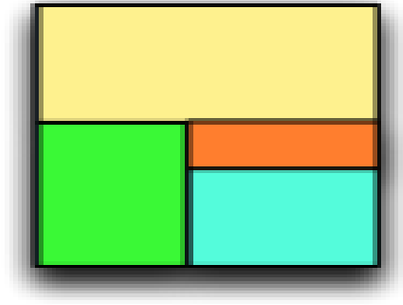








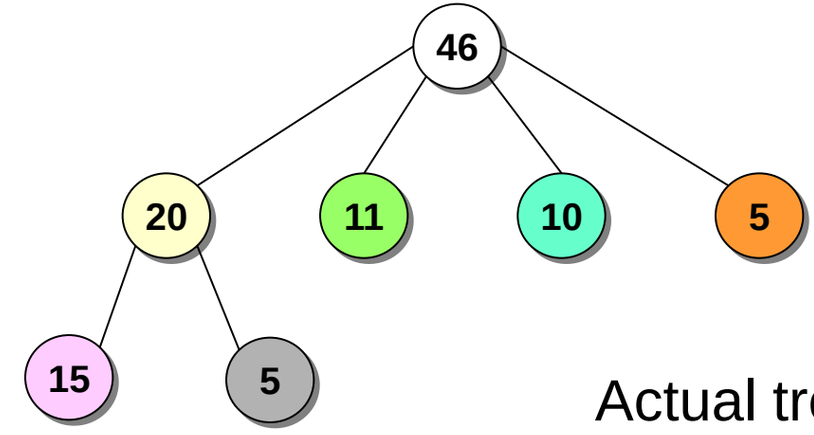




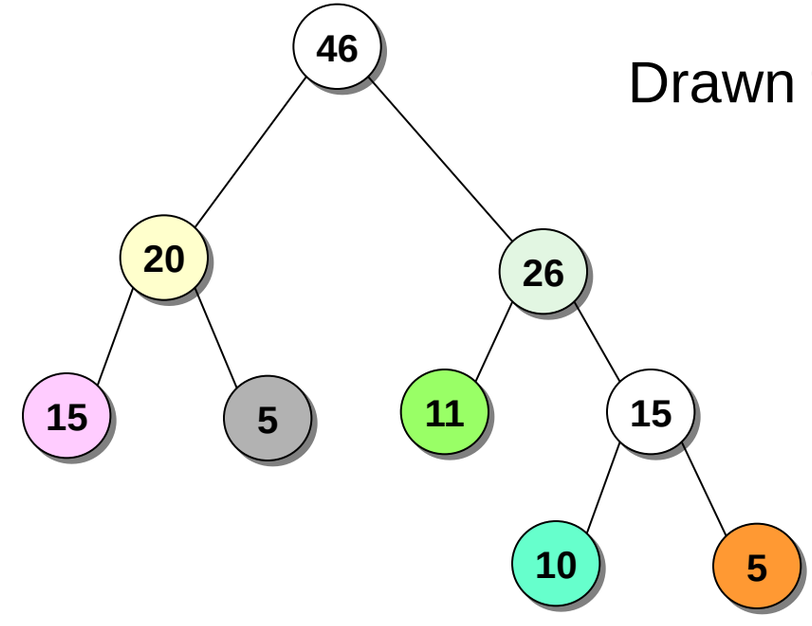
 copyright ©2025 m.patrignani

Binary treemaps ambiguities

- Binary treemaps are ambiguous when the tree has several generations of nodes
  - nested treemaps are recommanded



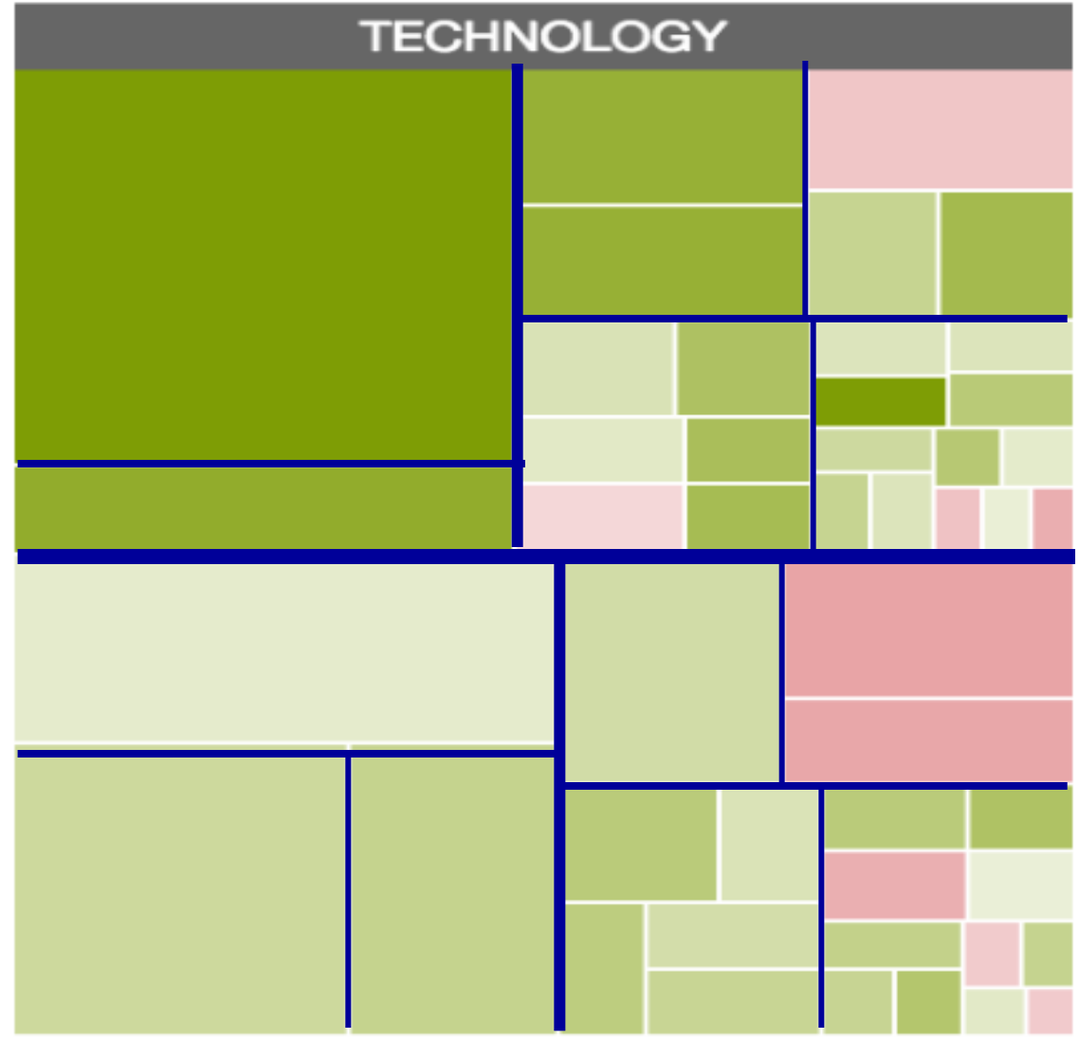
Actual tree



Drawn tree

copyright ©2025 m.patrignani

Examples of binary treemaps

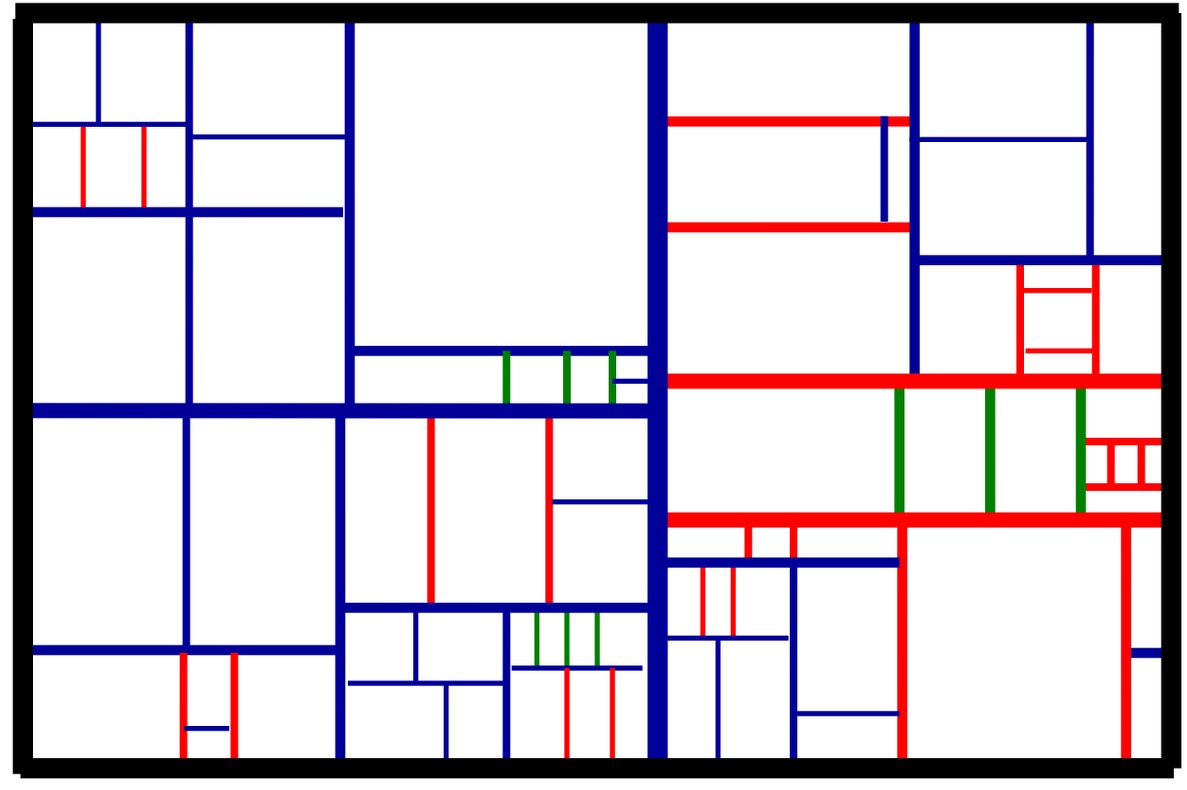


From [Heeringa & Jannen 16]

 copyright ©2025 m.patrignani

Cluster treemaps?

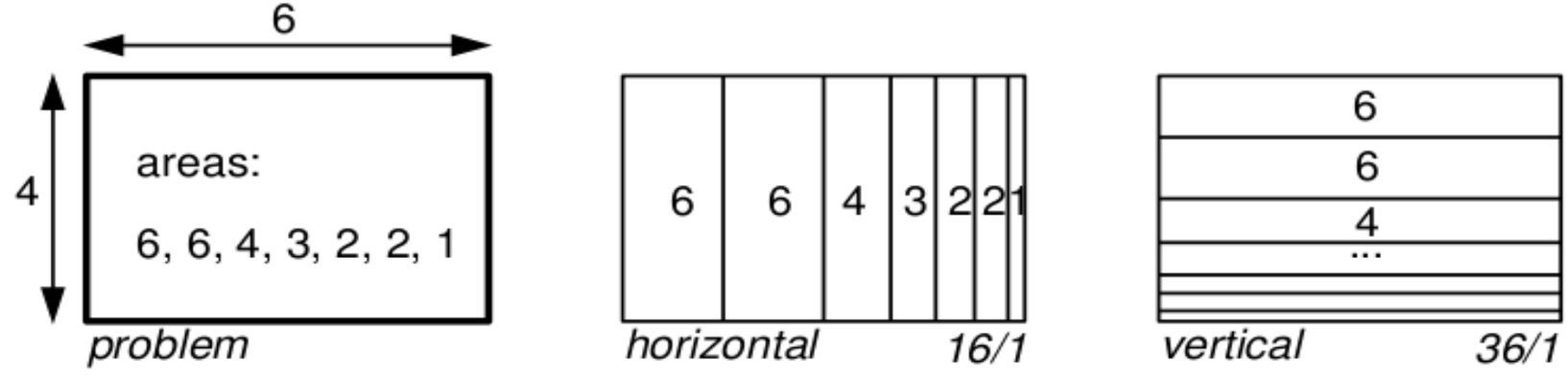
- Are a variant of binary treemaps that uses trees of arbitrary degree



 copyright ©2025 m.patrignani

Squarified treemaps

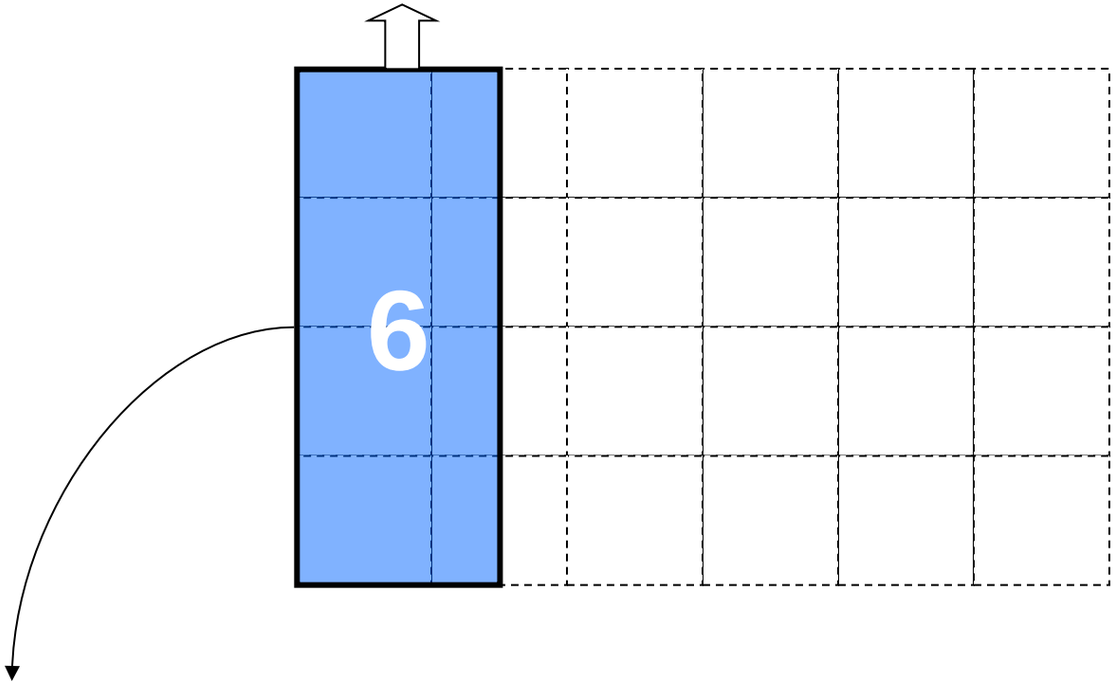
- Iterative heuristics introduced by [Bruls *et al.* 00]
  - subdivide each rectangular area among the children independently
  - sort the children by size
- Example
  - consider a rectangualar area 6 × 4 (weight 24) with children: 6, 6, 4, 3, 2, 2, 1
    - aspect ratio is 6/4 (bigger over smaller side)



 copyright ©2025 m.patrignani

Squarified treemaps example

- First build a stack going up



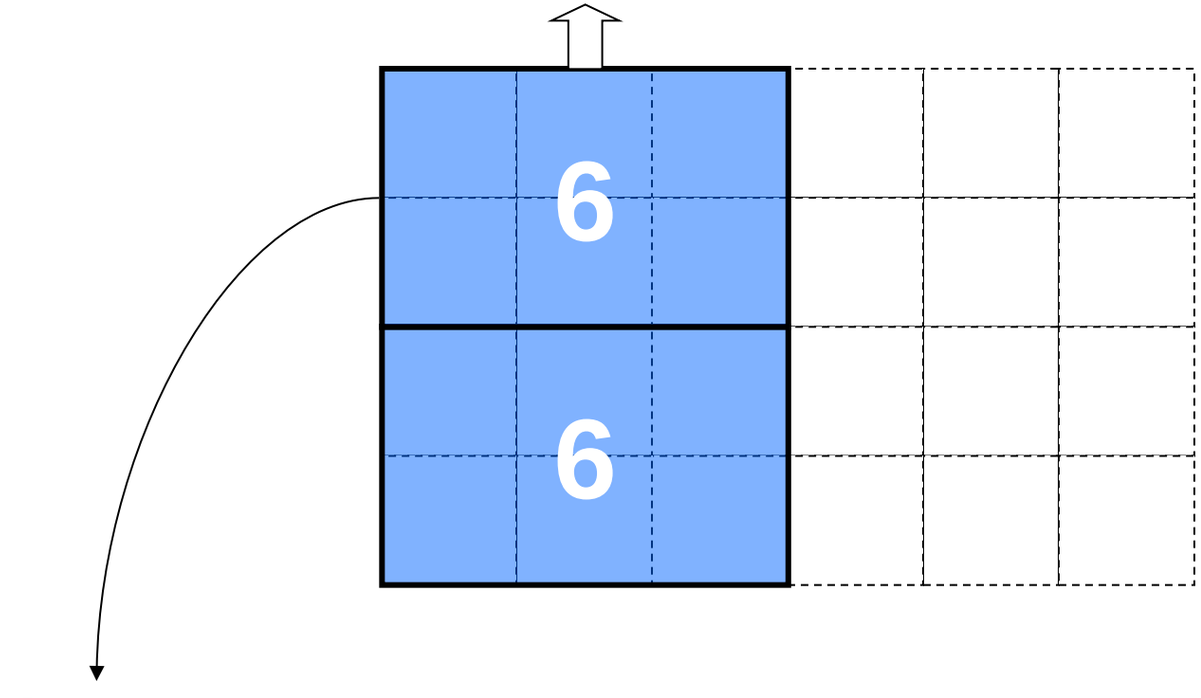
$$\frac{4}{1.5} = 2.6\overline{7}$$

best aspect ratio for this stack = 2.67

 copyright ©2025 m.patrignani

Squarified treemaps example

- First build a stack going up

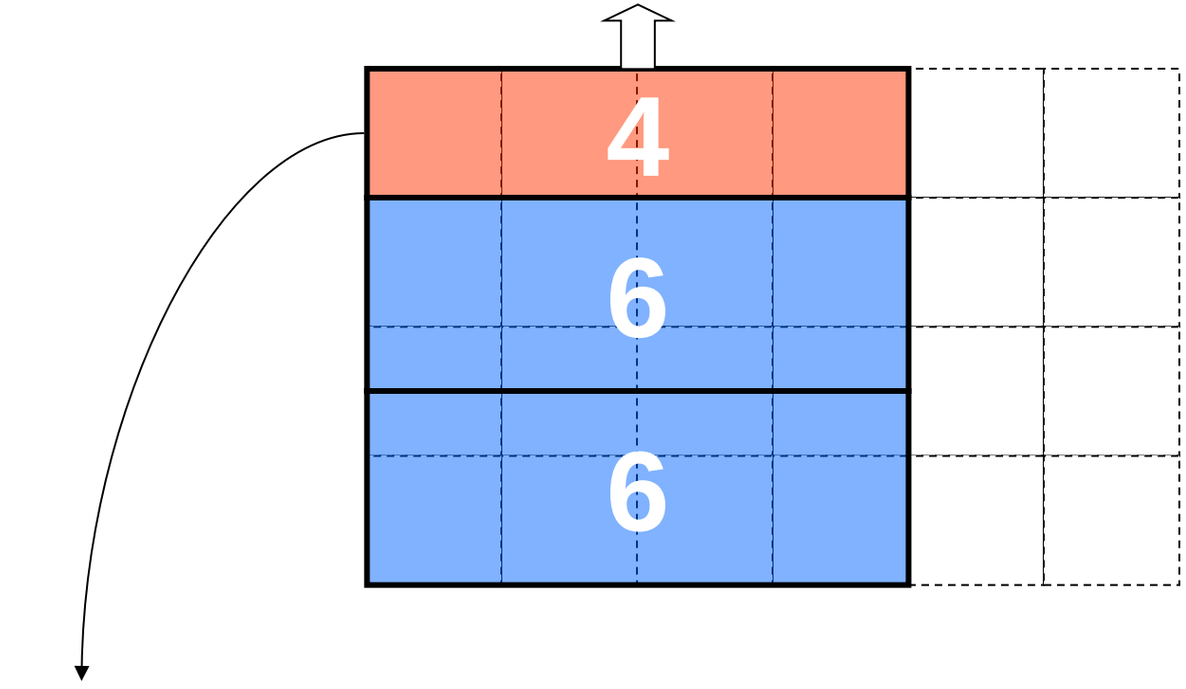


$\frac{3}{2} = 1.5$ → best aspect ratio for this stack = 1.5

 copyright ©2025 m.patrignani

Squarified treemaps example

- First build a stack going up



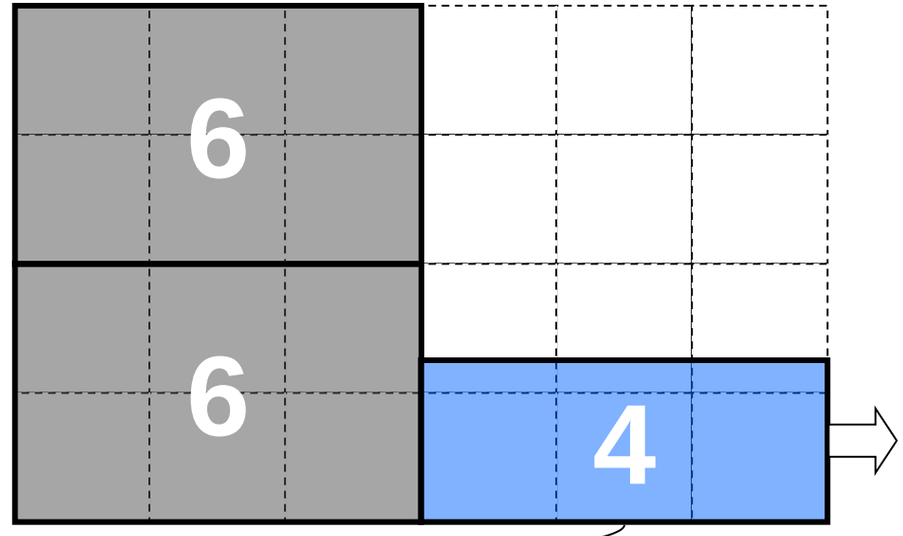
$$\frac{4}{1} = 4$$

best aspect ratio for this stack = 1.5

 copyright ©2025 m.patrignani

Squarified treemaps example

- Build a stack going right

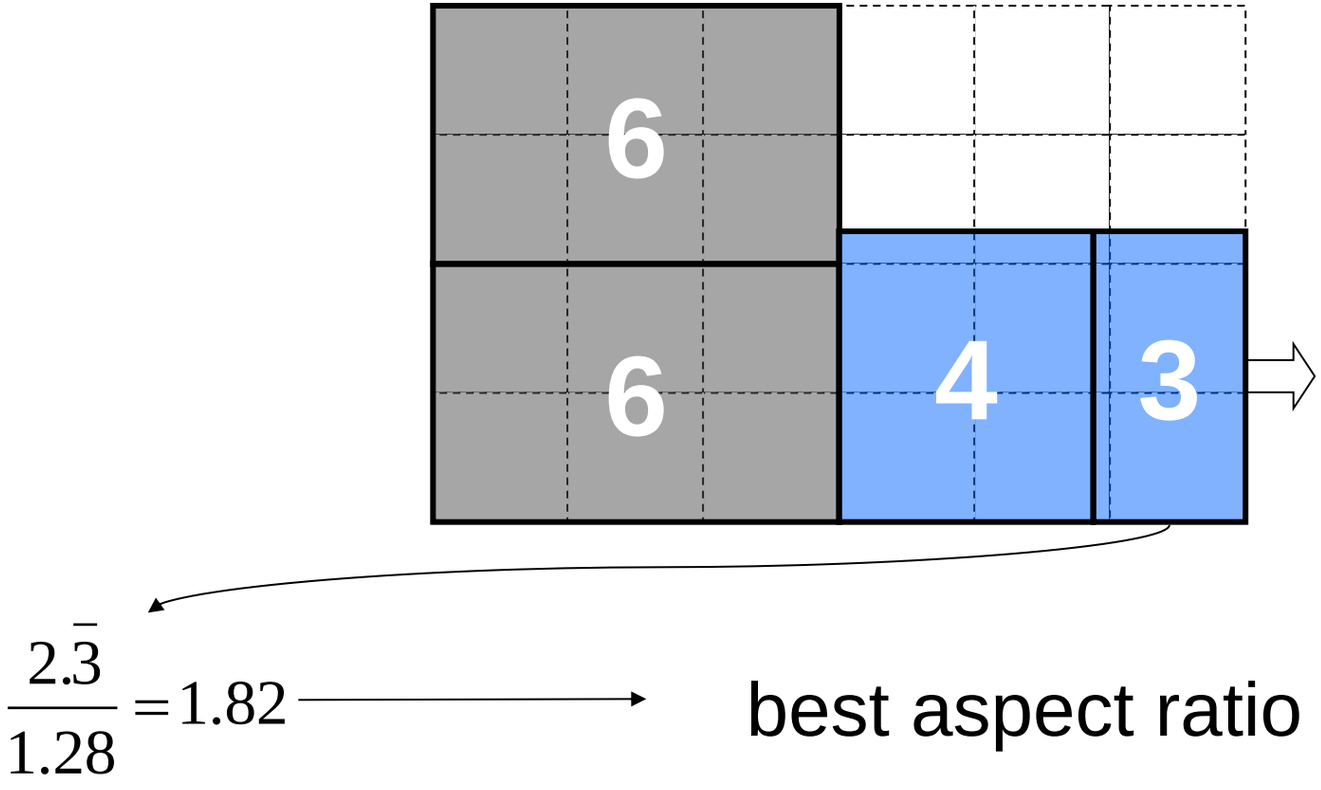


$$\frac{3}{1.\overline{3}} = 2.25$$ → best aspect ratio for this stack = 2.25

 copyright ©2025 m.patrignani

Squarified treemaps example

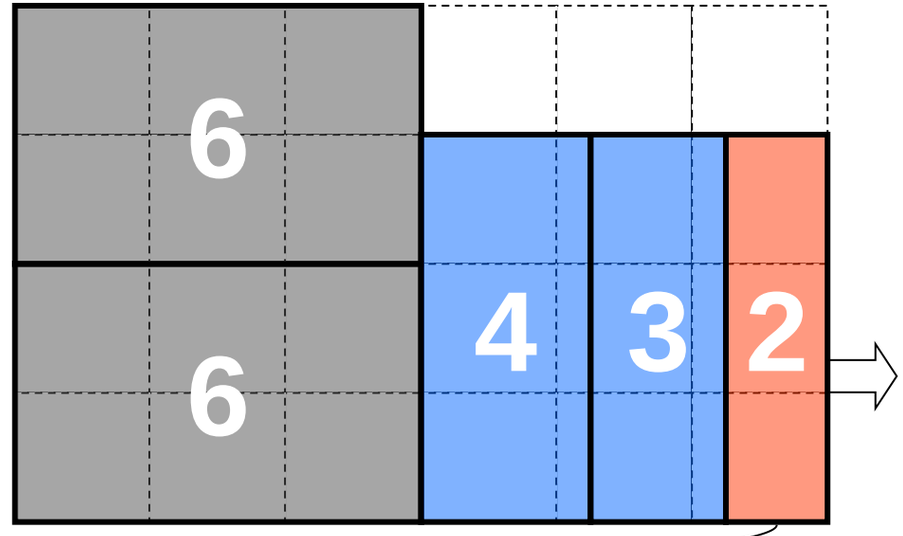
- Build a stack going right



 copyright ©2025 m.patrignani

Squarified treemaps example

- Build a stack going right



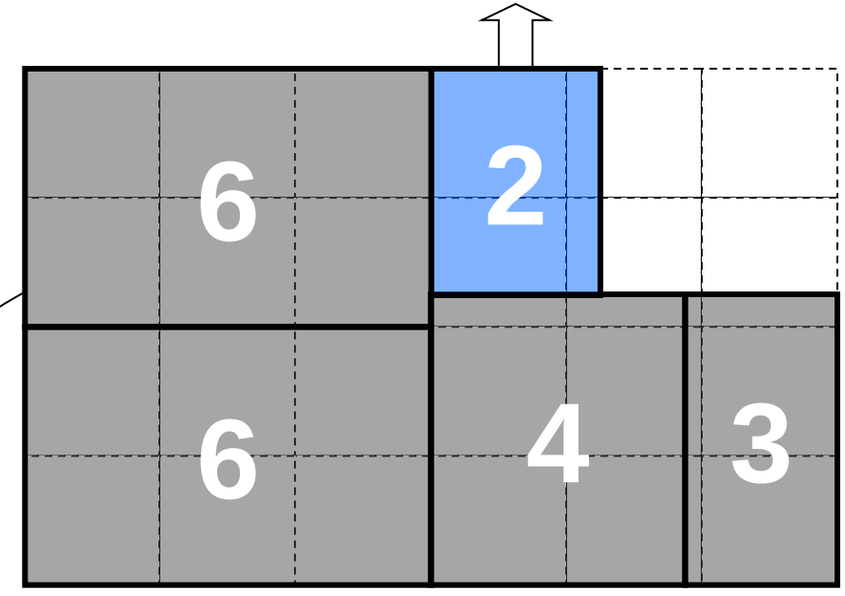
$$\frac{3}{0.6\overline{6}} = 4.5$$

best aspect ratio for this stack = 1.82

 copyright ©2025 m.patrignani

Squarified treemaps example

- Build a stack going up



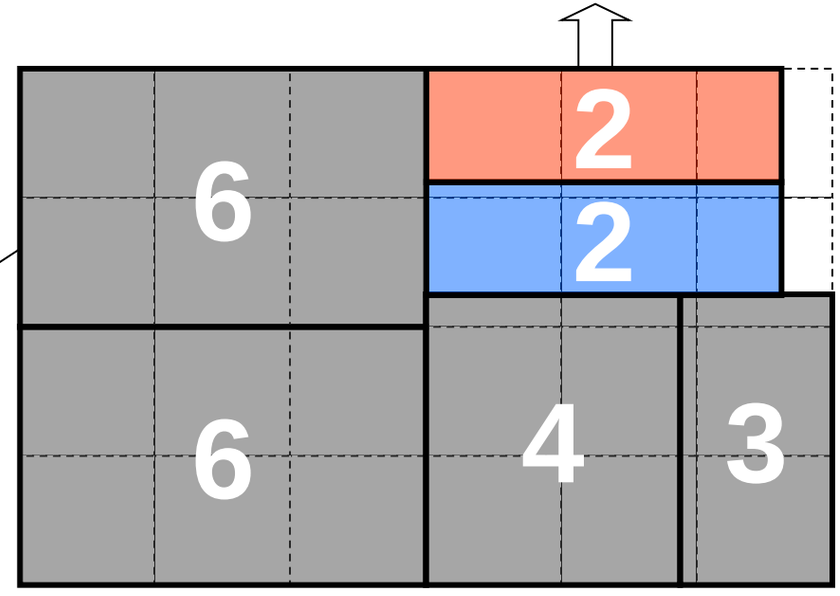
$2.\bar{3}$

$\frac{1.6\bar{6}}{1.2} = 1.38$ → best aspect ratio for this stack = 1.38

 copyright ©2025 m.patrignani

Squarified treemaps example

- Build a stack going up



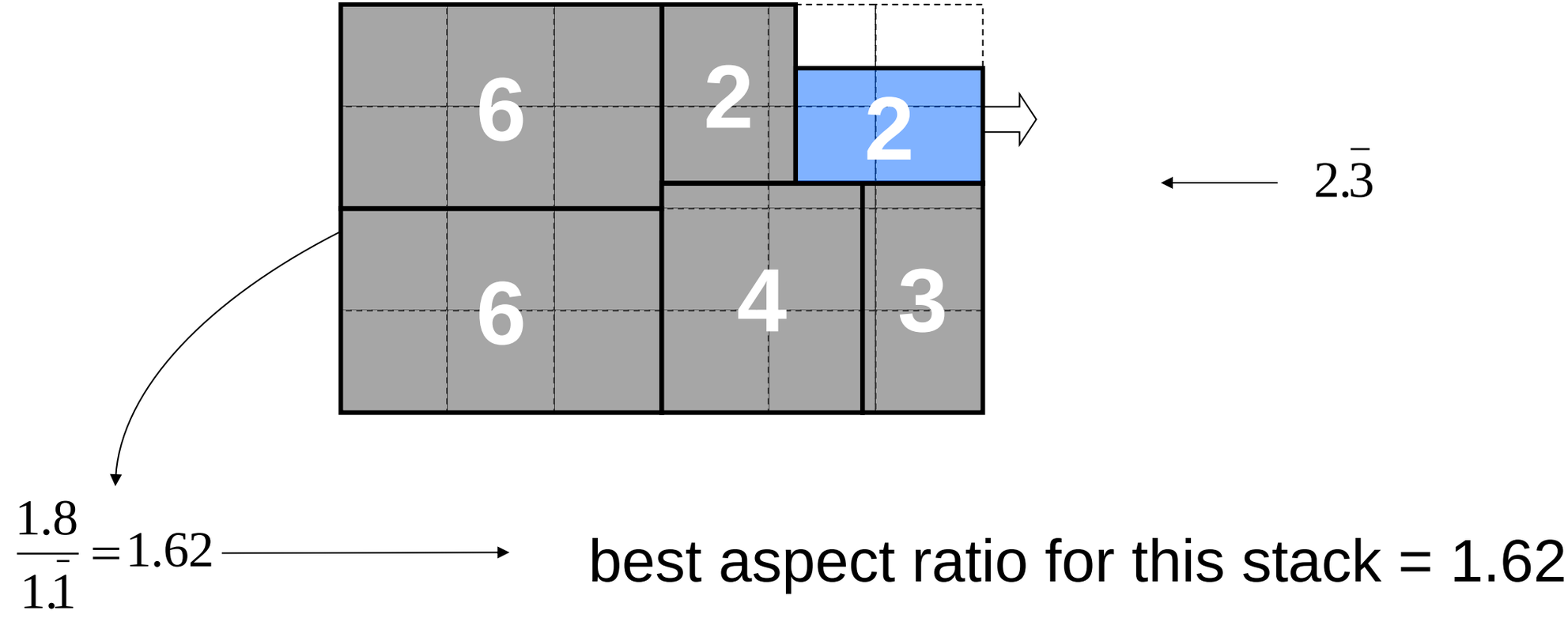
$$\frac{2.76}{0.728} = 3.61$$

best aspect ratio for this stack = 1.38

copyright ©2025 m.patrignani

Squarified treemaps example

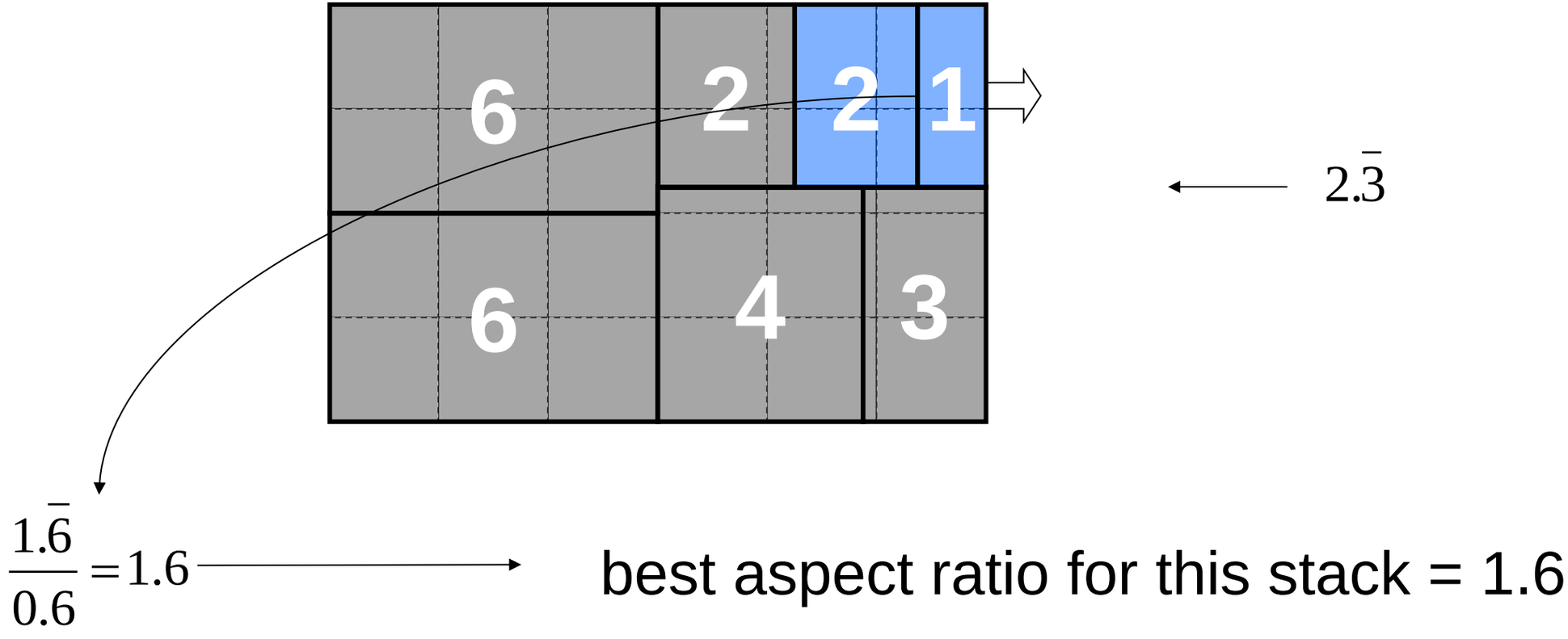
- Build a stack going right



 copyright ©2025 m.patrignani

Squarified treemaps example

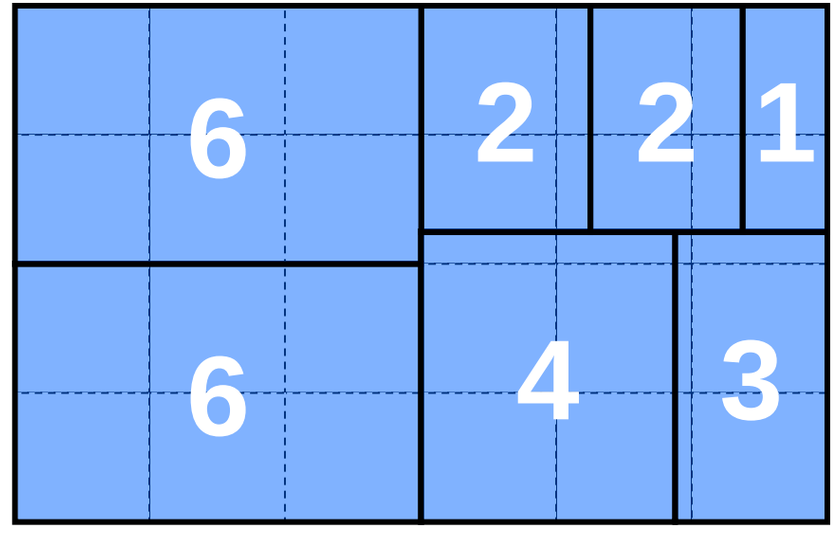
- Build a stack going right



 copyright ©2025 m.patrignani

Squarified treemaps example

- Final representation



  - example from [Bruls *et al.*, 00]

 copyright ©2025 m.patrignani

Examples

- File system with 1400 files

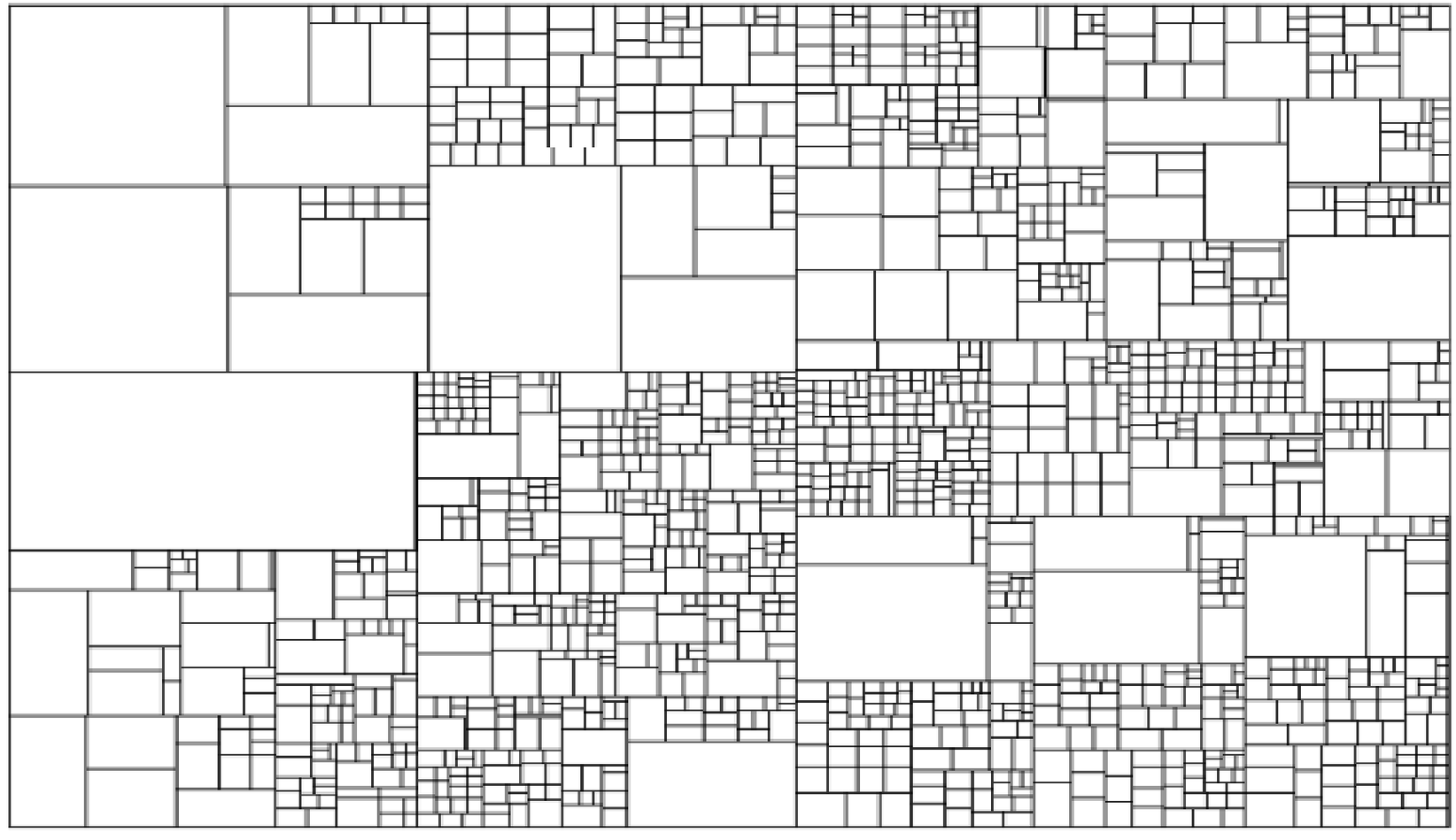


Traditional slice-and-dice

 copyright ©2025 m.patrignani

Examples

- File system with 1400 files

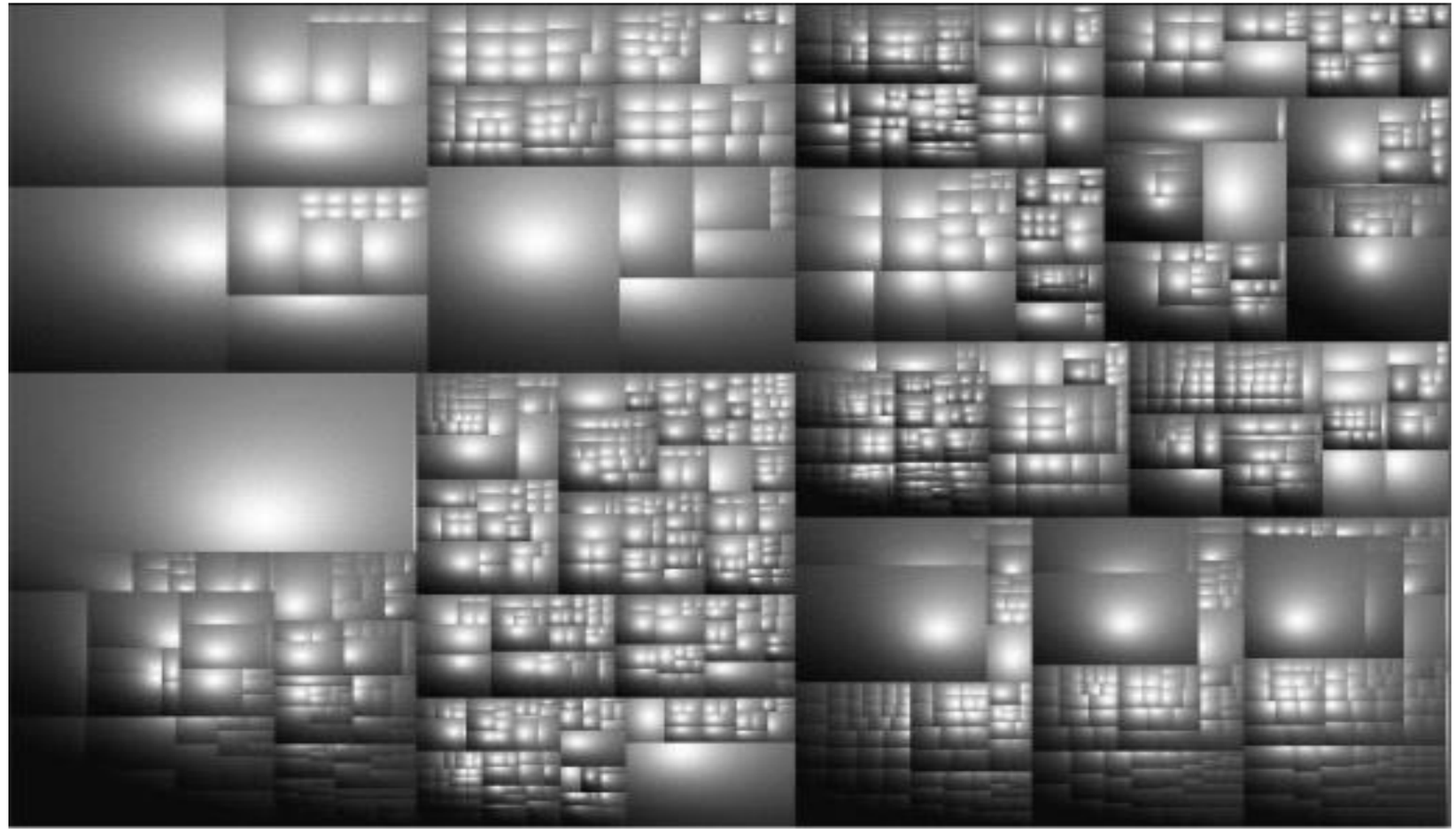


Squarified treemaps

 copyright ©2025 m.patrignani

Examples

- File system with 1400 files

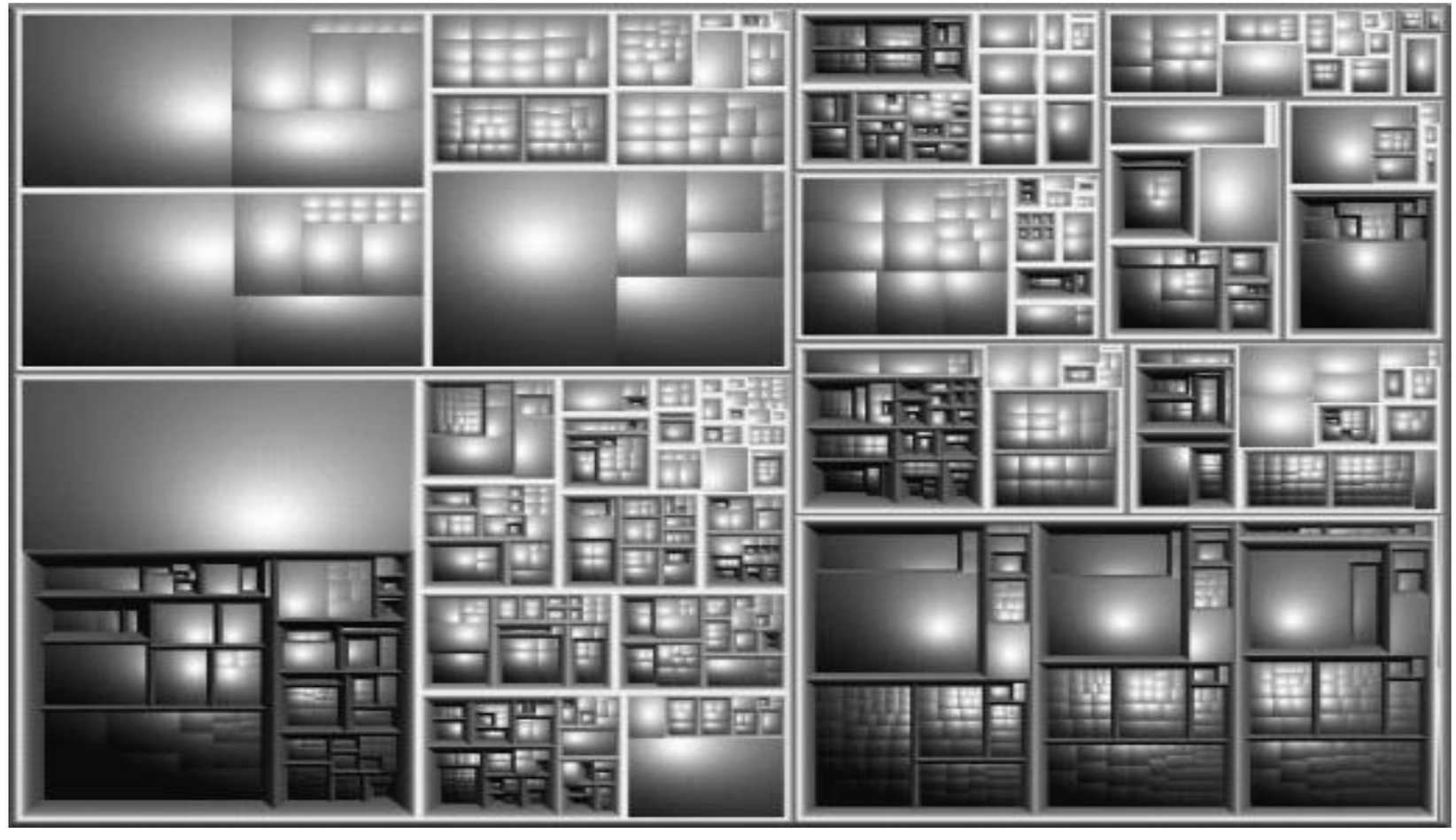


Squarified cushion treemaps

 copyright ©2025 m.patrignani

Examples

- File system with 1400 files

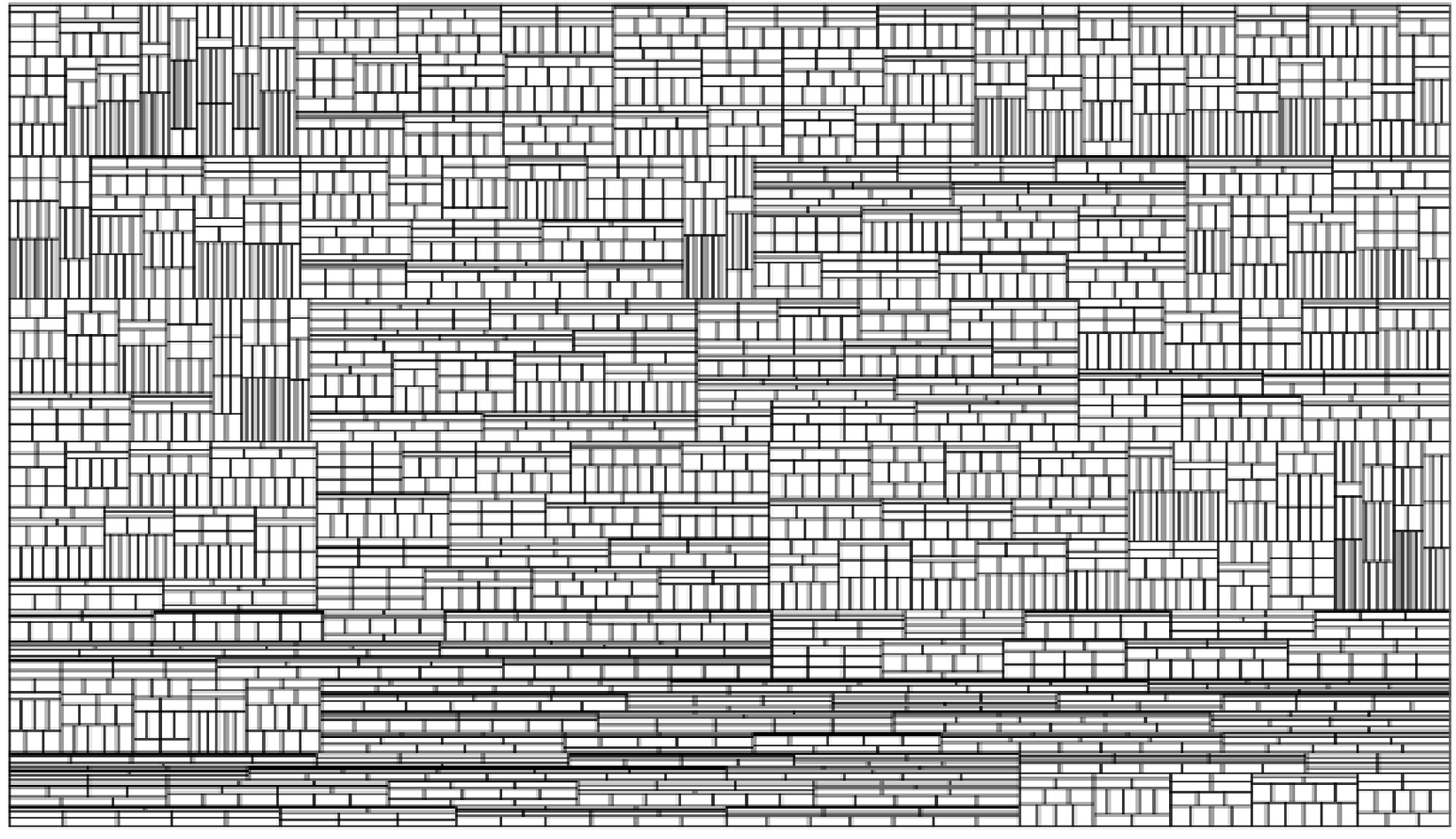


Squarified nested cushion treemaps

 copyright ©2025 m.patrignani

Examples

- Artificial organization chart (six levels and 3060 employees)

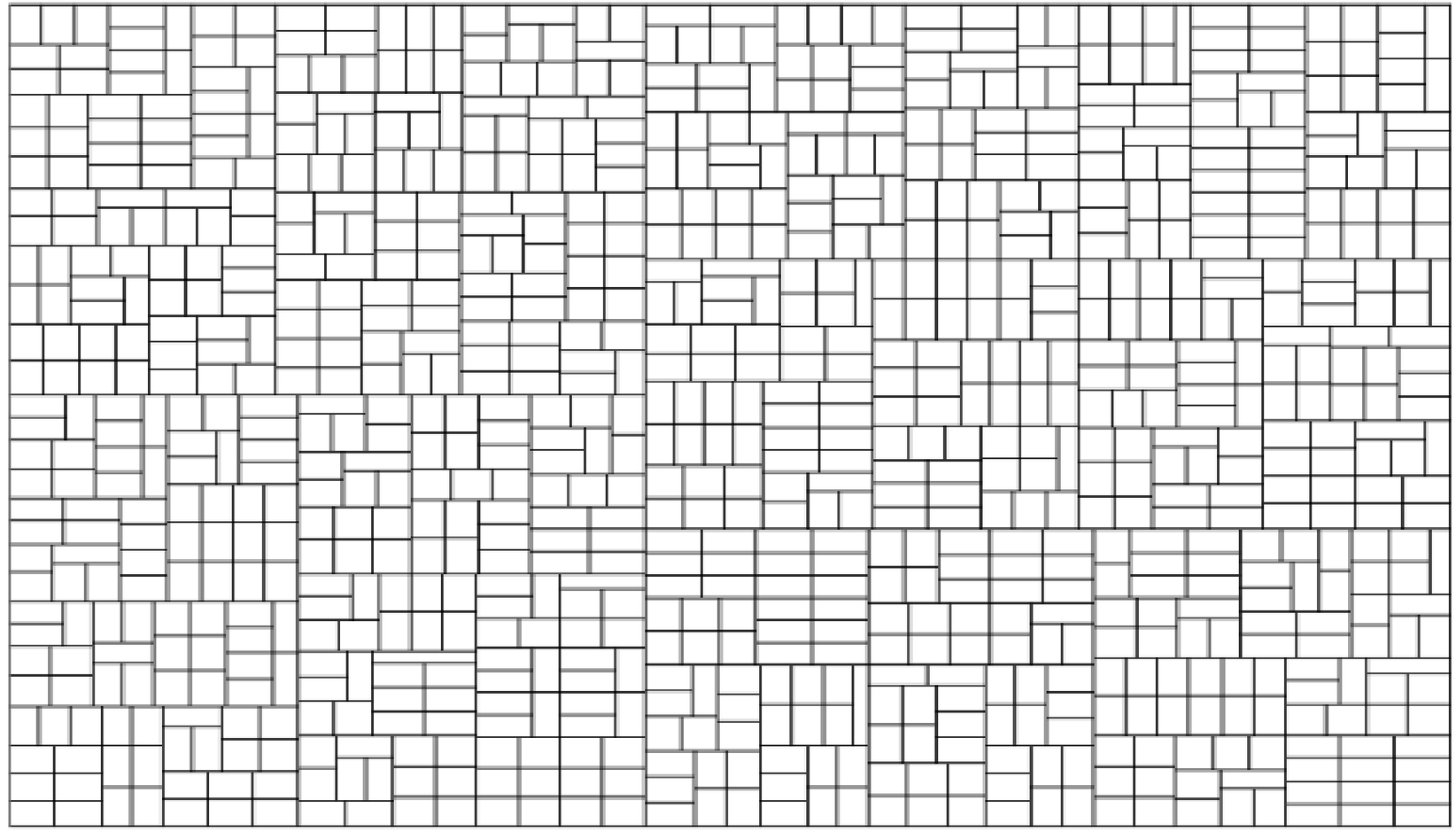


Traditional slice-and-dice

 copyright ©2025 m.patrignani

Examples

- Artificial organization chart (six levels and 3060 employees)

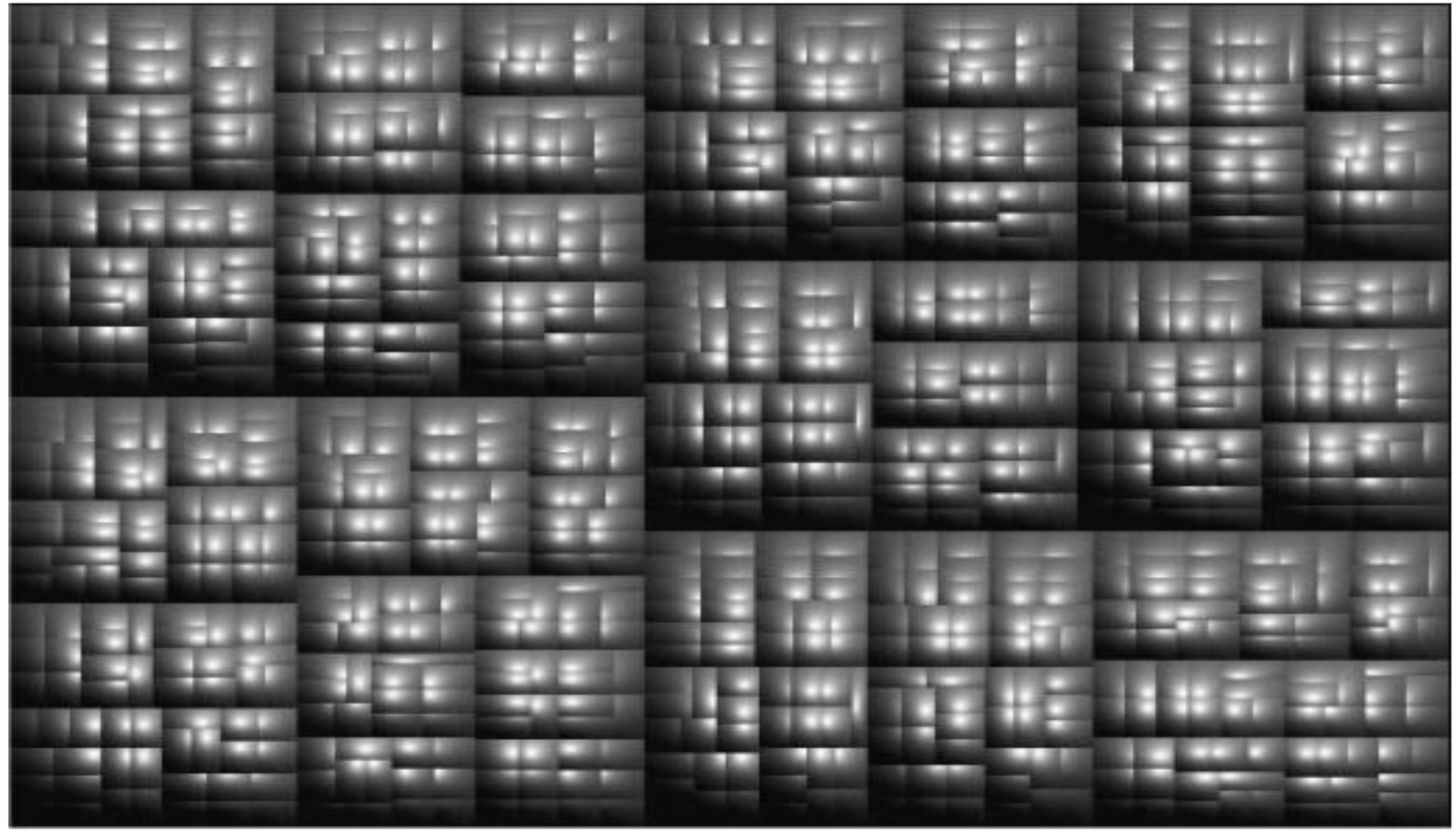


Squarified treemaps

 copyright ©2025 m.patrignani

Examples

- Artificial organization chart (six levels and 3060 employees)

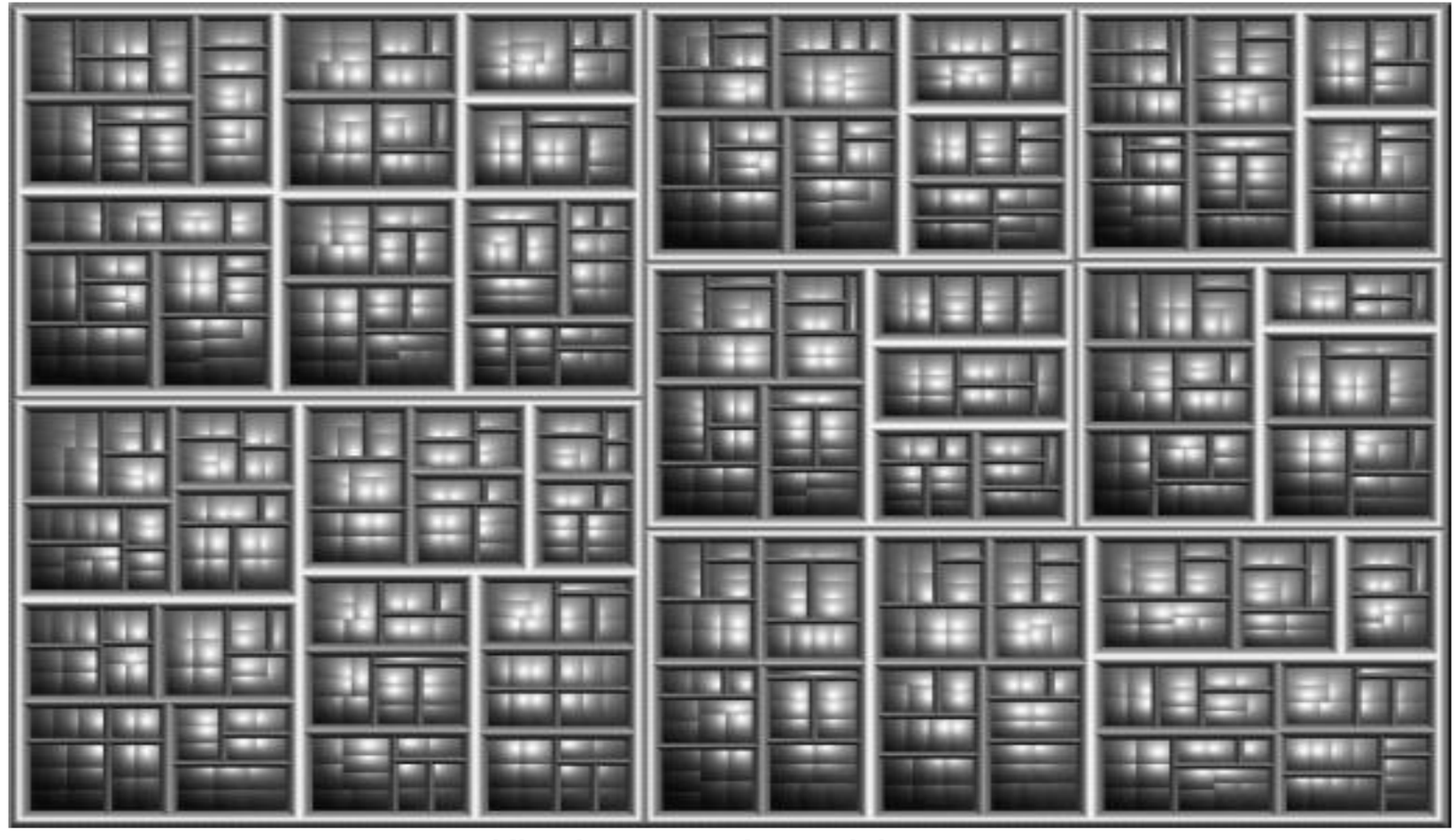


Squarified cushion treemaps

 copyright ©2025 m.patrignani

Examples

- Artificial organization chart (six levels and 3060 employees)



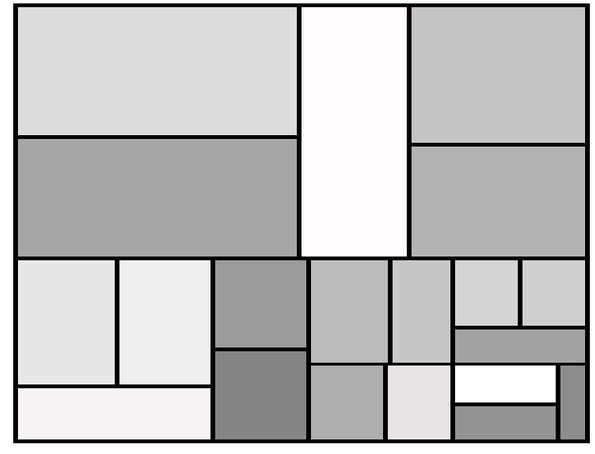
Squarified nested cushion treemaps

 copyright ©2025 m.patrignani

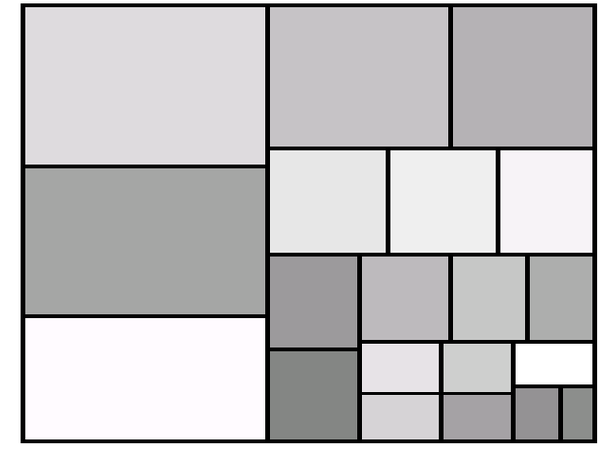
Cluster and squarified treemaps issues

- Order is not preserved
- Changes in the data set can cause dramatic discontinuous changes in the layouts
  - the output of the slice-and-dice algorithm, instead, would change continuously

Cluster treemaps



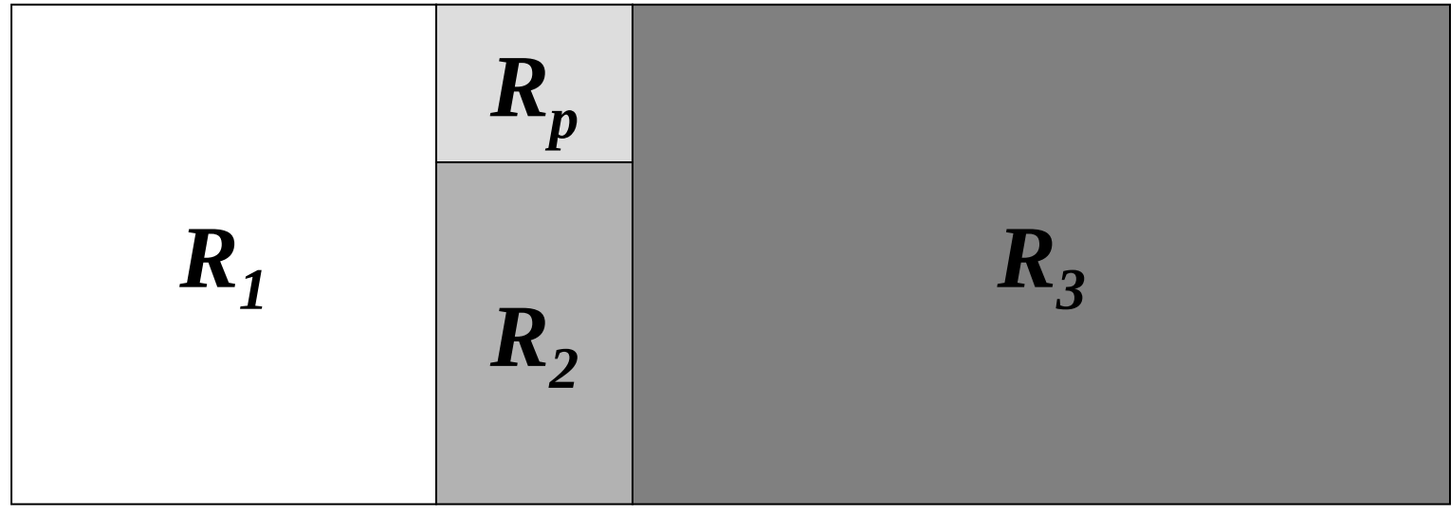
Squarified treemaps



copyright ©2025 m.patrignani

Ordered treemaps

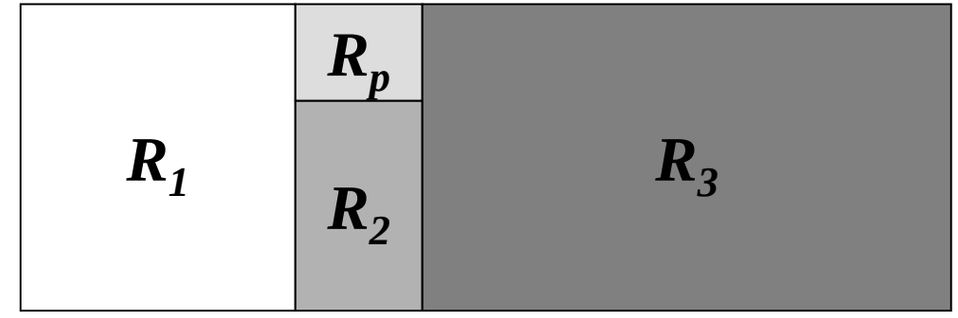
- Basic idea of [Shneiderman & Wattenberg 01]
  - use one element as the *pivot*
    - for example the biggest element or the middle element
  - subdivide all other elements into tree sets
    - elements preceding the pivot will be drawn into R_1
    - elements following the pivot are divided into two sets as best as possible to optimize the aspect ratio of R_p
  - recursively draw the three sets into R_1, R_2, R_3

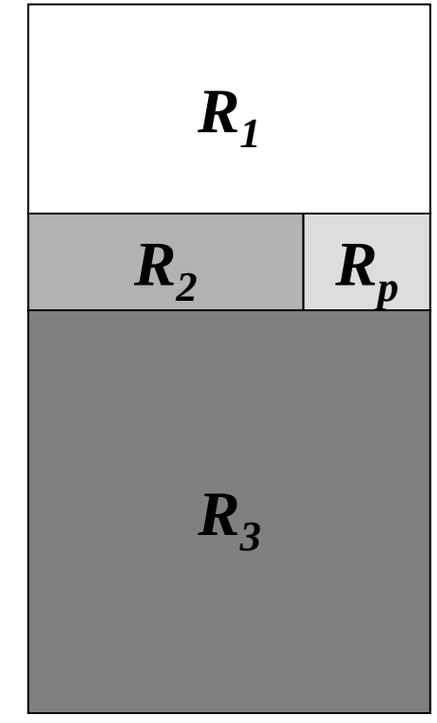


 copyright ©2025 m.patrignani

Ordered treemaps

- If the rectangle is horizontal subdivide it vertically
- If the rectangle is vertical subdivide it horizontally
- Two versions of the algorithm
  - if the pivot is the biggest element, the algorithm is called *pivot-by-size*
  - if the pivot is the middle element, the algorithm is called *pivot-by-middle*



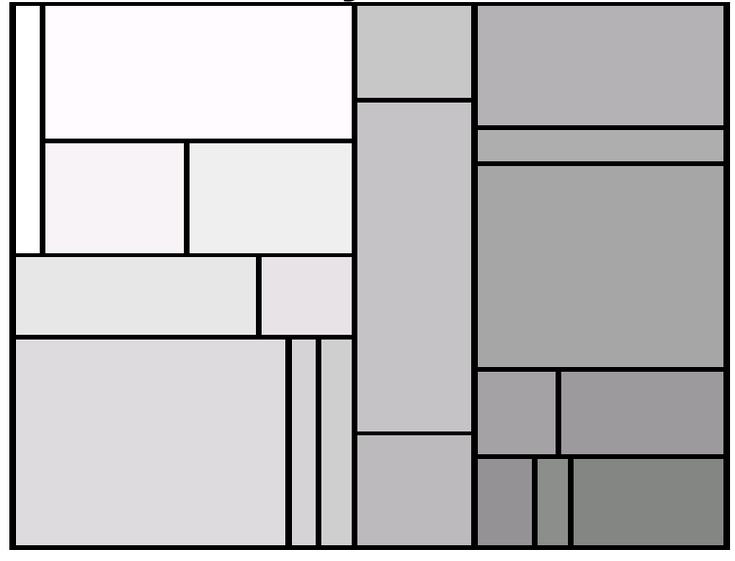


 copyright ©2025 m.patrignani

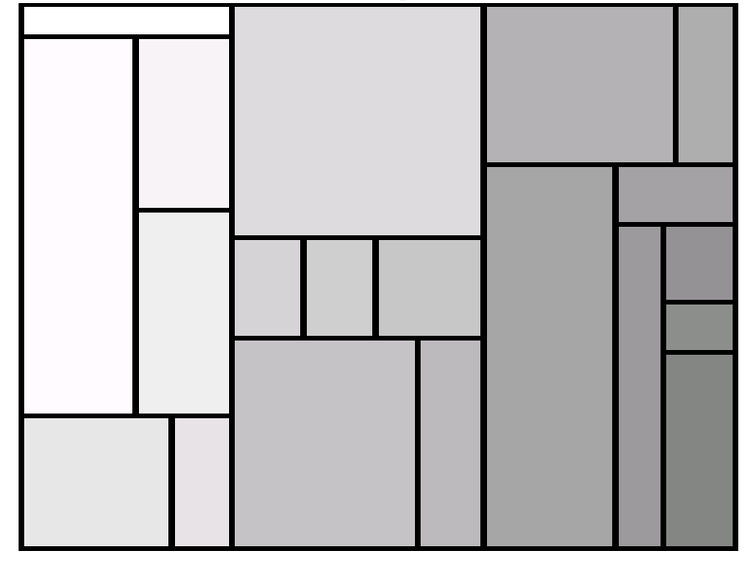
Pivot-by-middle and pivot-by-size

- Aspect ratio is good (but not optimal)
- Order is roughly preserved
- Pivot-by-middle is less prone to dynamic changes
- Tree structure is lost!

Pivot-by-middle



Pivot-by-size



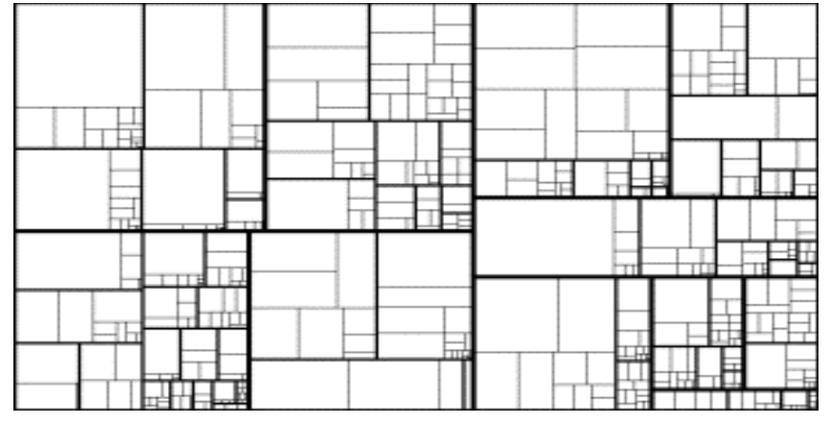
copyright ©2025 m.patrignani

Comparison

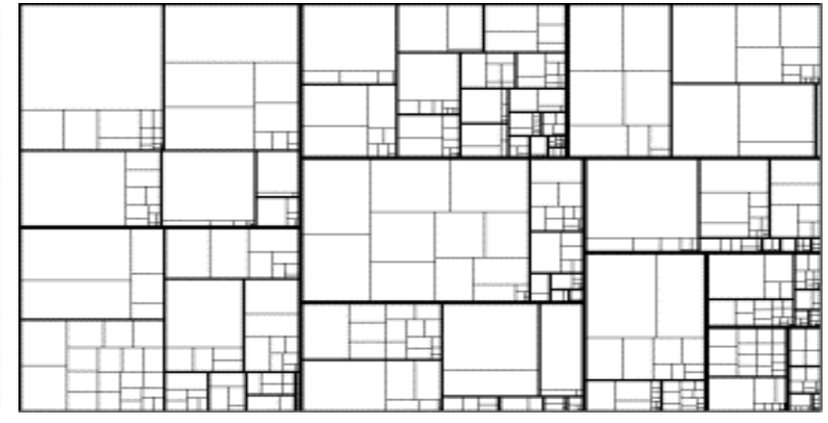
- On market data (from [Shneiderman & Wattenberg, 01])



Slice-and-dice



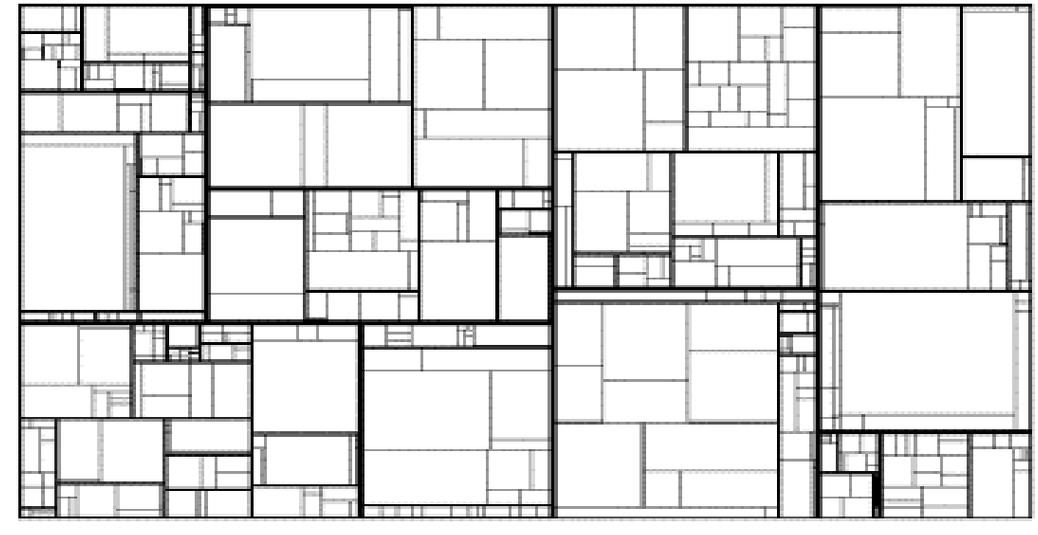
Cluster layout



Squarified layout



Pivot-by-middle



Pivot-by-size

copyright ©2025 m.patrignani

Comparison

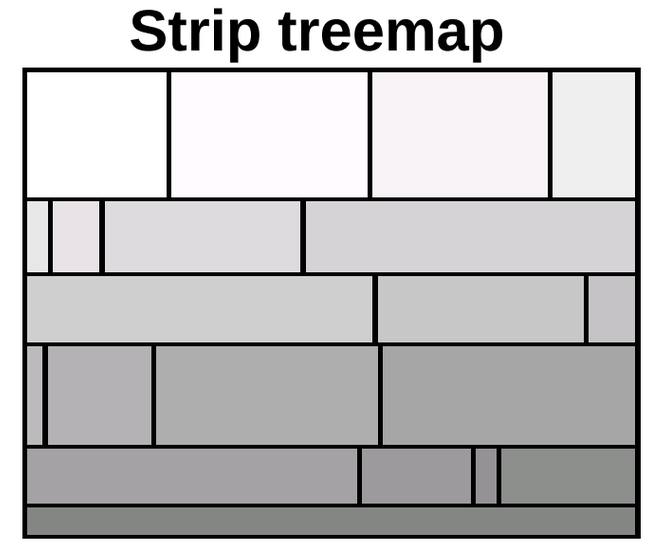
- Algorithm pivot-by-middle and pivot-by-size sacrifice aspect ratio to roughly preserve order

Algorithm	Aspect Ratio
Slice-and-dice	369.83
Pivot-by-middle	19.30
Pivot-by-size	22.04
Cluster layout	3.74
Squarified layout	3.21

 copyright ©2025 m.patrignani

Strip treemaps

- Introduced in [Bederson *et al.* 02]
- Modification of squarified treemaps of [Bruls *et al.* 00]
  - add to first strip untill average aspect ratio does not increase
  - recur on the remaining area
- Good aspect ratio
  - better than pivot-by-size and pivot-by-middle
  - worse than cluster and squarified treemaps
- Strict order



 copyright ©2025 m.patrignani

Quantum treemaps

- Introduced in [Bederson *et al.* 02]
- Generate rectangles whose sizes are multiples of a single input dimension
- Useful for laying out images or other fixed-size objects
  - standard photos (usually 4×6)
  - document pages (A4 or letter size)
- Any treemap algorithm can be quantized
  - in [Bederson *et al.* 02] strip treemaps is used

 copyright ©2025 m.patrignani

PhotoMesa

- Zoomable image browser (Windows)

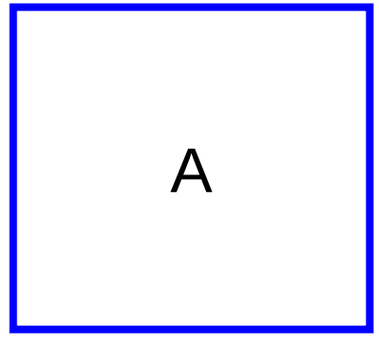


www.photomesa.com/

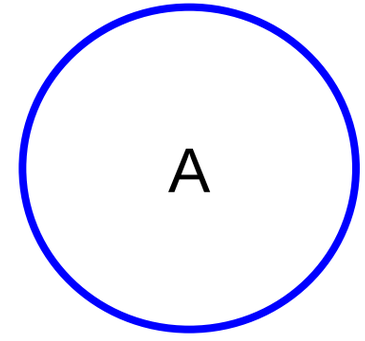
 copyright ©2025 m.patrignani

Implicit representations of trees

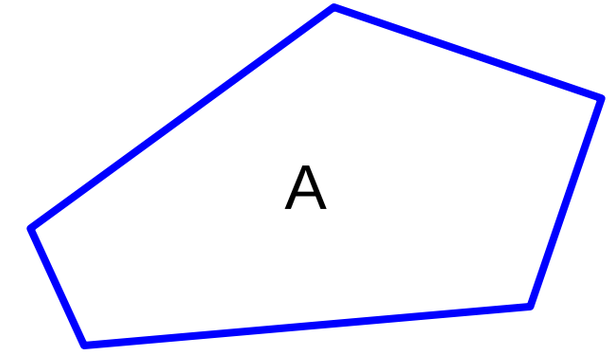
- Node representations



Squared nodes



Circle nodes

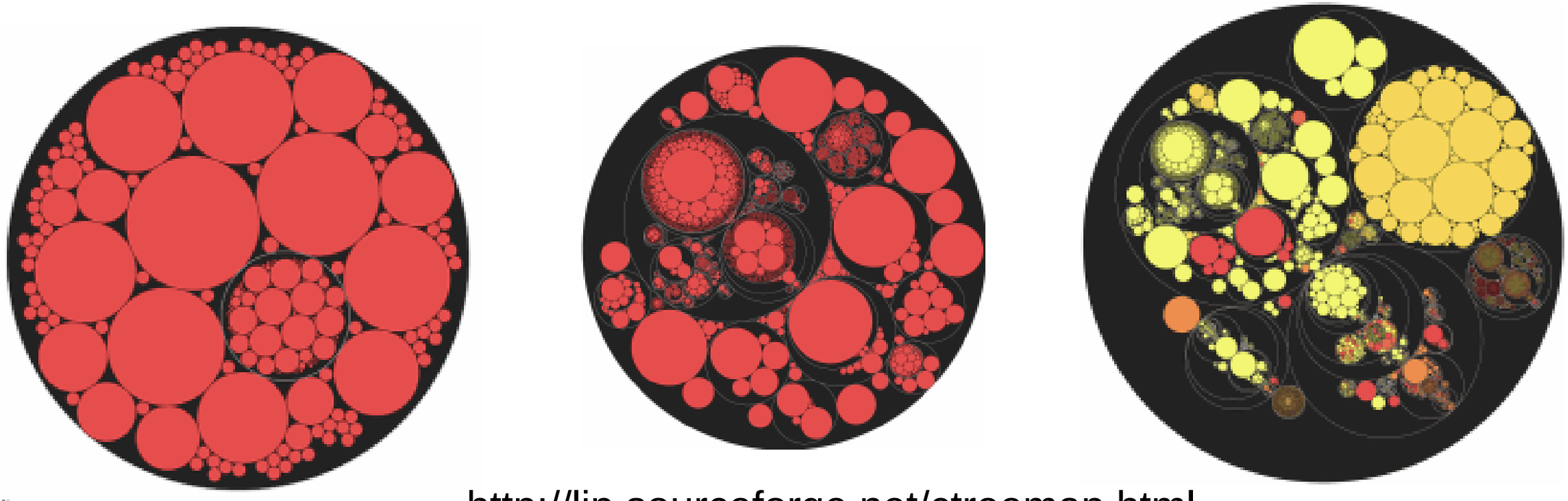


Nodes of arbitrary shape

 copyright ©2025 m.patrignani

Circular treemaps

- They don't fill the available space completely
- The aspect ratio is always the same
- The internal layout of a circle does not depend on its position in the tree

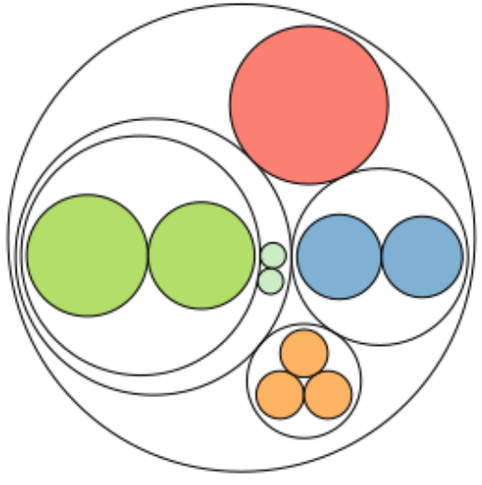


http://lip.sourceforge.net/ctreemap.html

 copyright ©2025 m.patrignani

Bubble treemaps

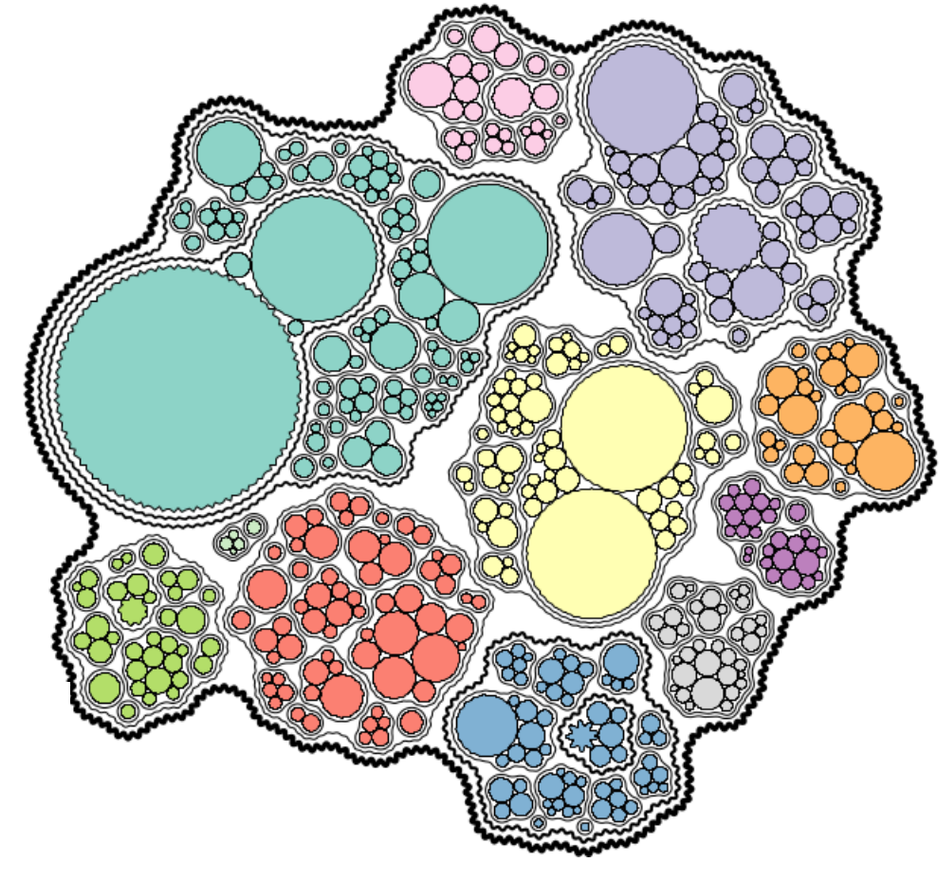
- Similar to circular treemaps
  - [Görtler *et al* 18]
  - uses a force-directed layout algorithm
  - “wave” on borders accounts for uncertanty



circular treemaps



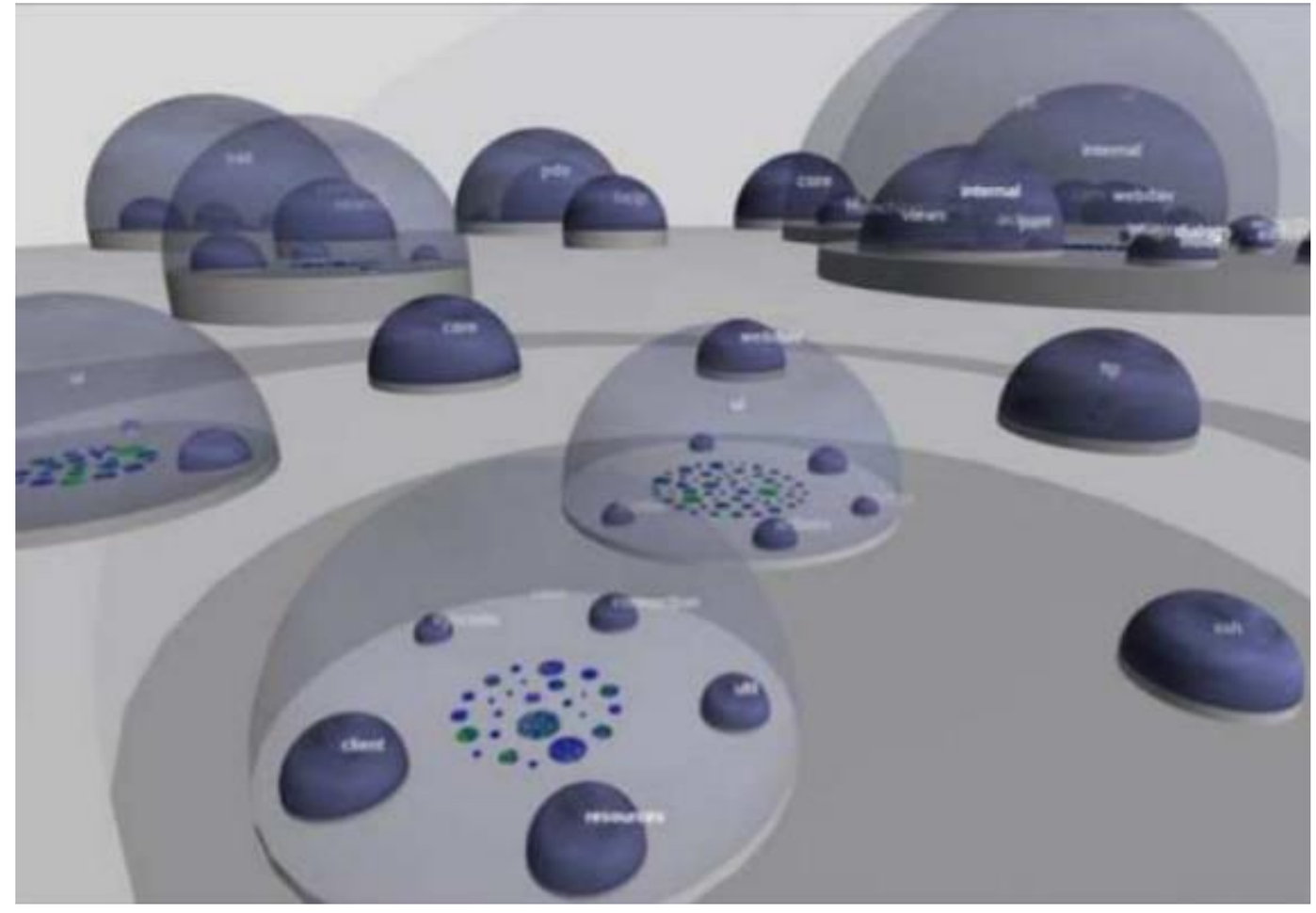
bubble treemaps



copyright ©2025 m.patrignani

3D implicit representations

- Nested hemispheres show hierarchical relationship
- Distribution of hemispheres based on Voronoi diagrams
  - [Balzer & Deussen 04]



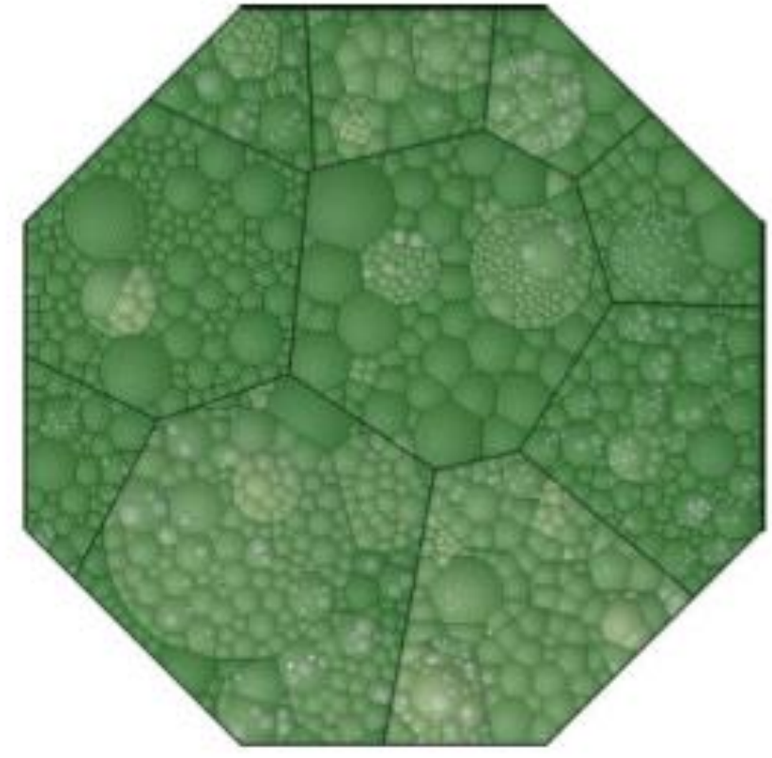
Picture from [Balzer & Deussen 04]

copyright ©2025 m.patrignani

Voronoi treemaps

- Introduced by [Balzer *et al.* 05]
- Replace the rectangular tessellation (subdivision) of the plane with a Voronoi tessellation

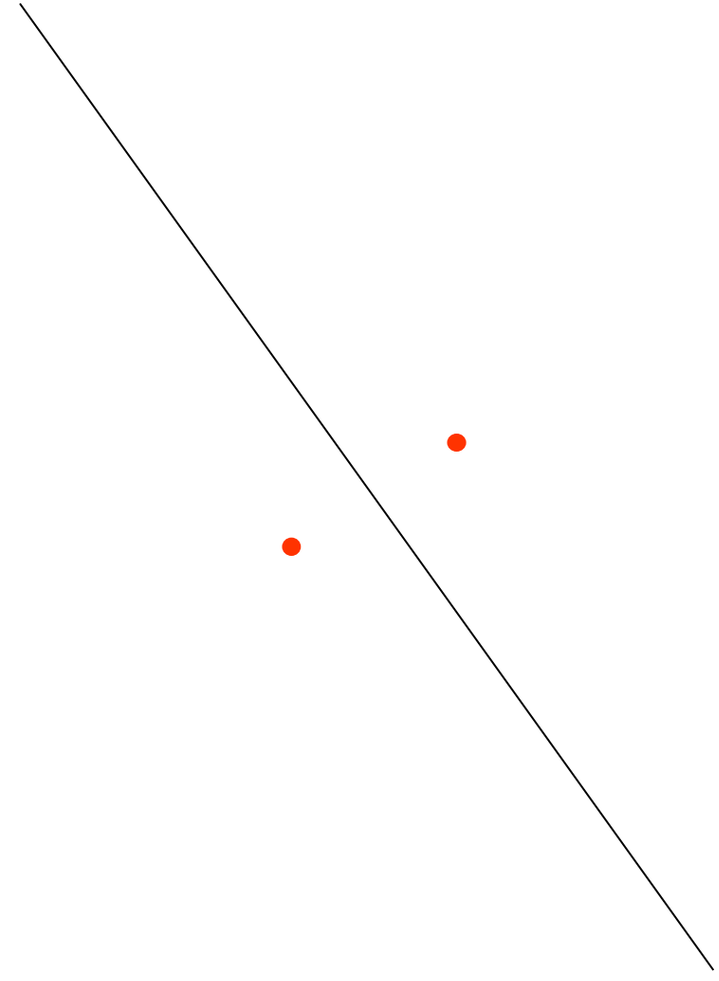




copyright ©2025 m.patrignani

Voronoi tessellation

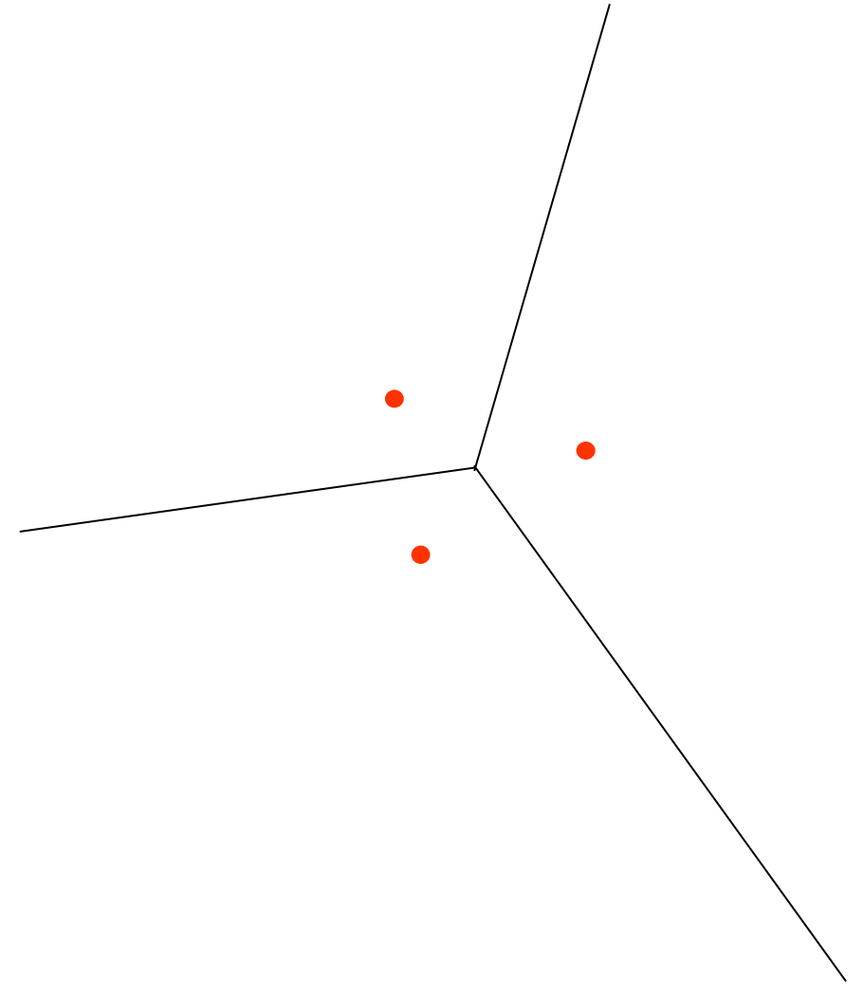
- A tessellation partitions the plane into regions
  - the Voronoi tessellation of two seeds partitions the plane into two half-planes
  - each half-plane contains the points that are nearer to one seed than to the other



 copyright ©2025 m.patrignani

Voronoi tessellation

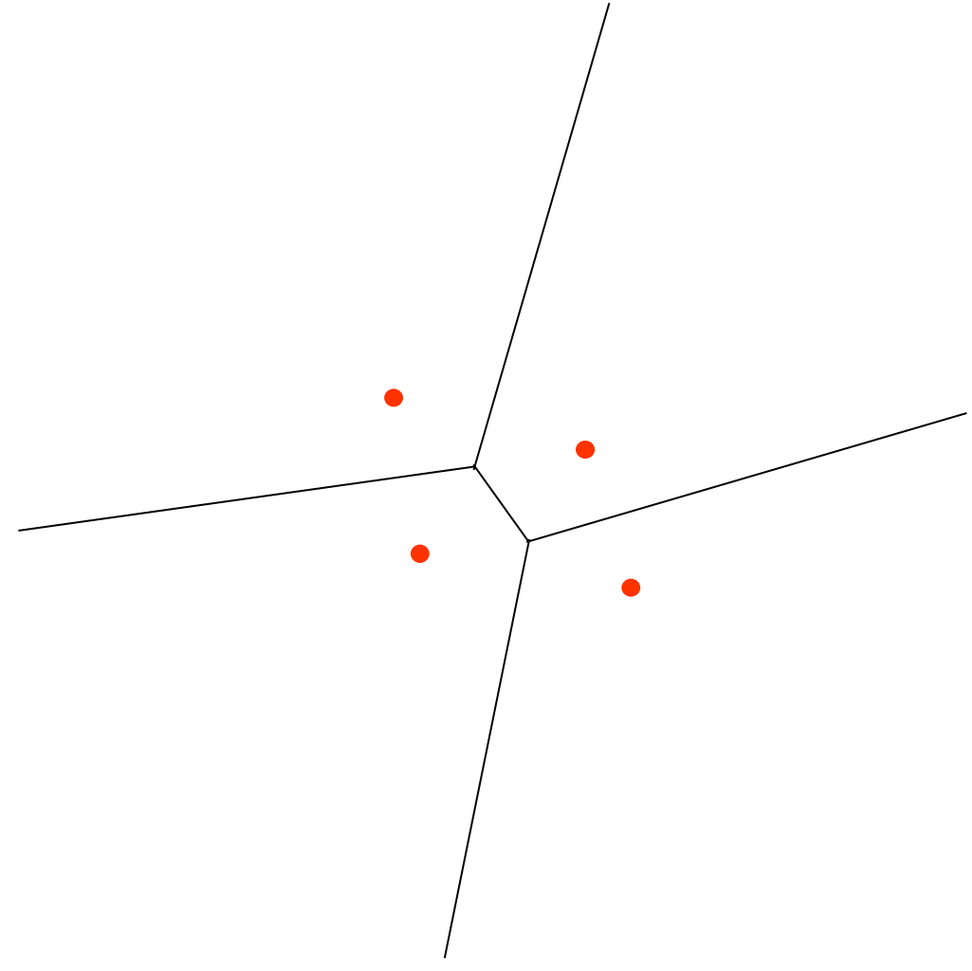
- A tessellation partitions the plane into regions
  - the Voronoi tessellation of three seeds partitions the plane into three convex regions



 copyright ©2025 m.patrignani

Voronoi tessellation

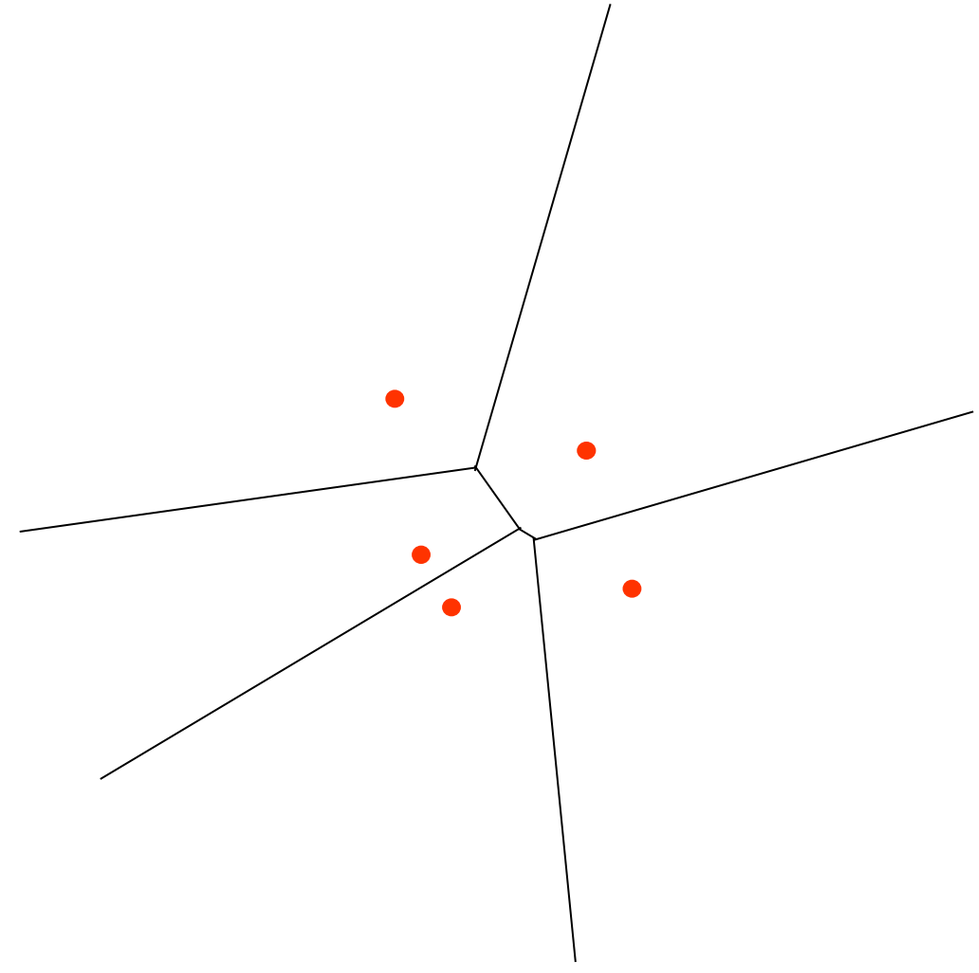
- A tessellation partitions the plane into regions
  - the Voronoi tessellation of four seeds



 copyright ©2025 m.patrignani

Voronoi tessellation

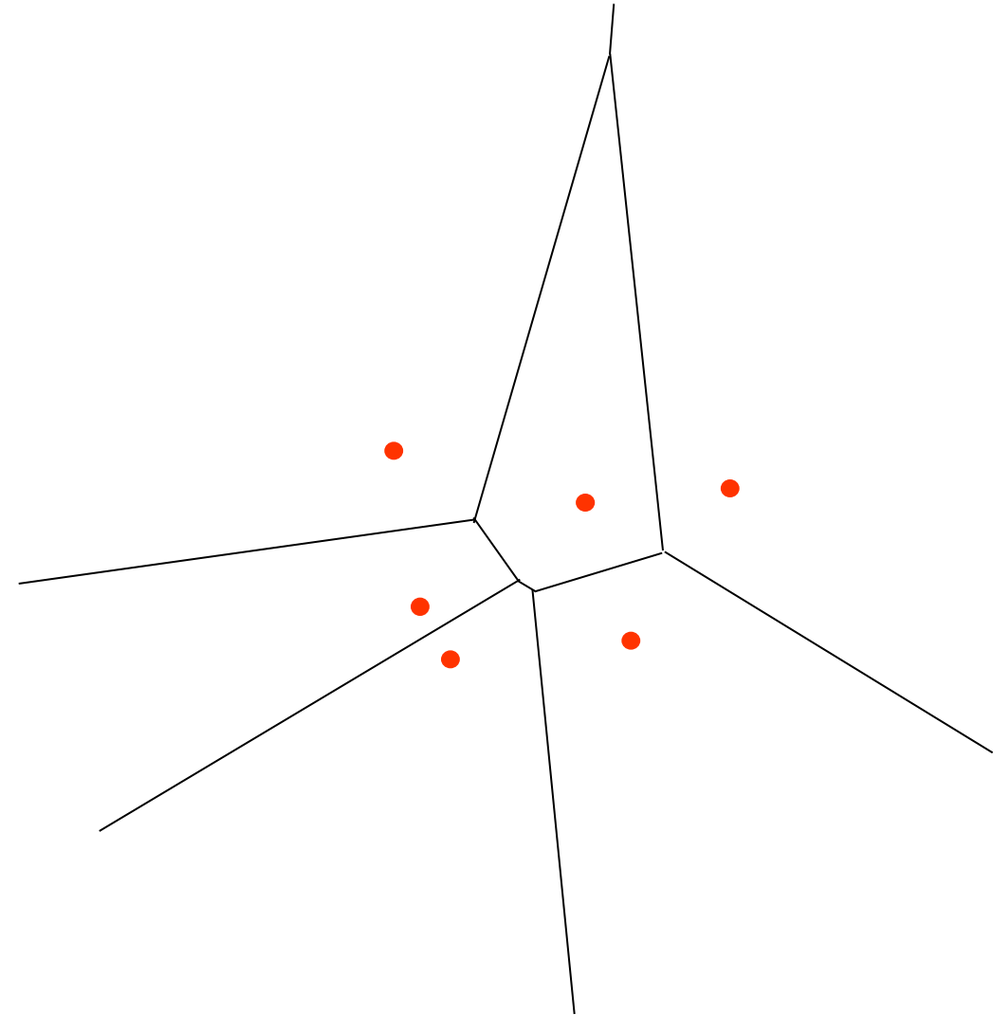
- A tessellation partitions the plane into regions
  - the Voronoi tessellation of five seeds



 copyright ©2025 m.patrignani

Voronoi tessellation

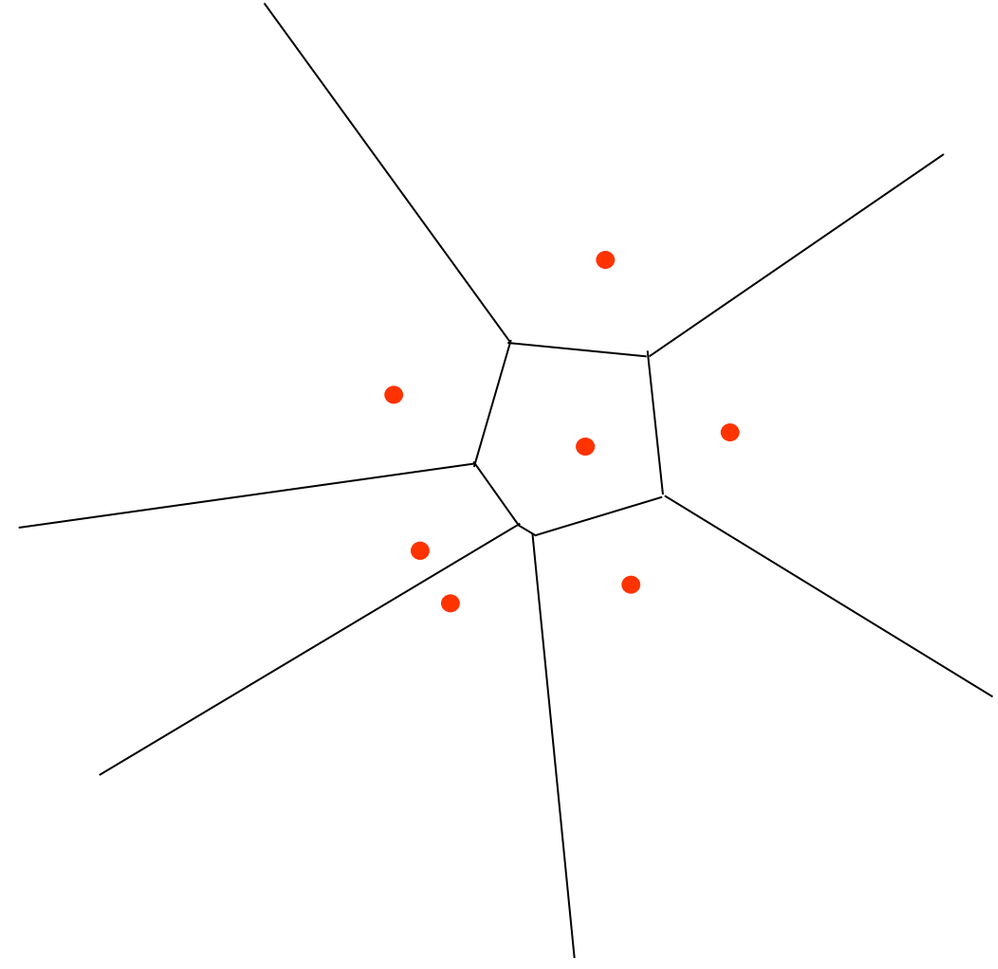
- A tessellation partitions the plane into regions
  - the Voronoi tessellation of six seeds



 copyright ©2025 m.patrignani

Voronoi tessellation

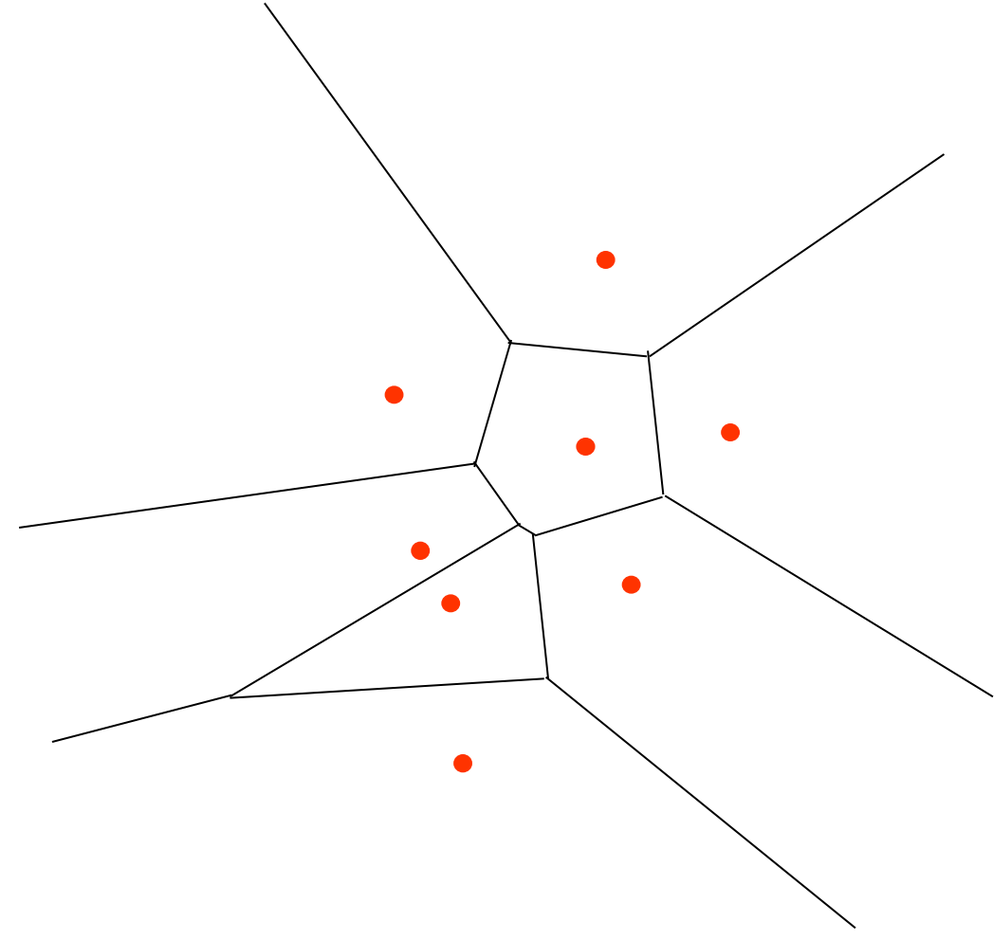
- A tessellation partitions the plane into regions
  - the Voronoi tessellation of seven seeds



 copyright ©2025 m.patrignani

Voronoi tessellation

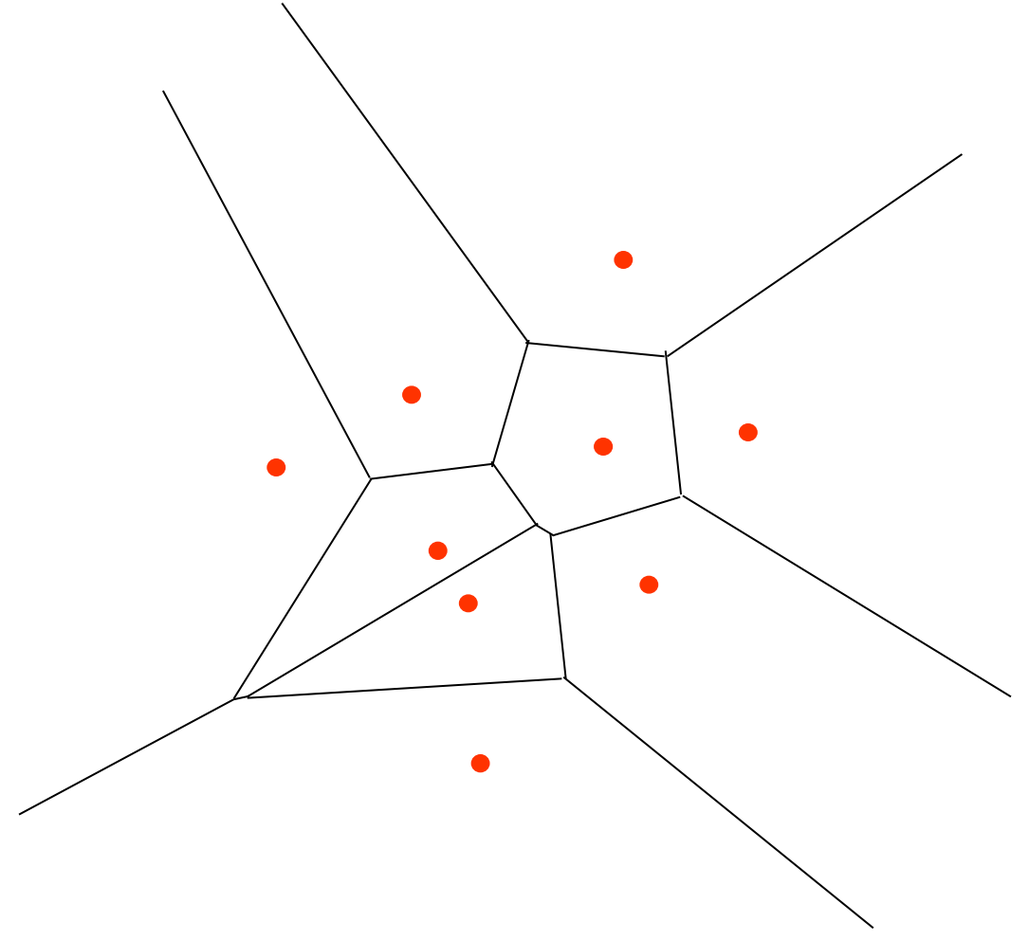
- A tessellation partitions the plane into regions
  - the Voronoi tessellation of many seeds



 copyright ©2025 m.patrignani

Voronoi tessellation

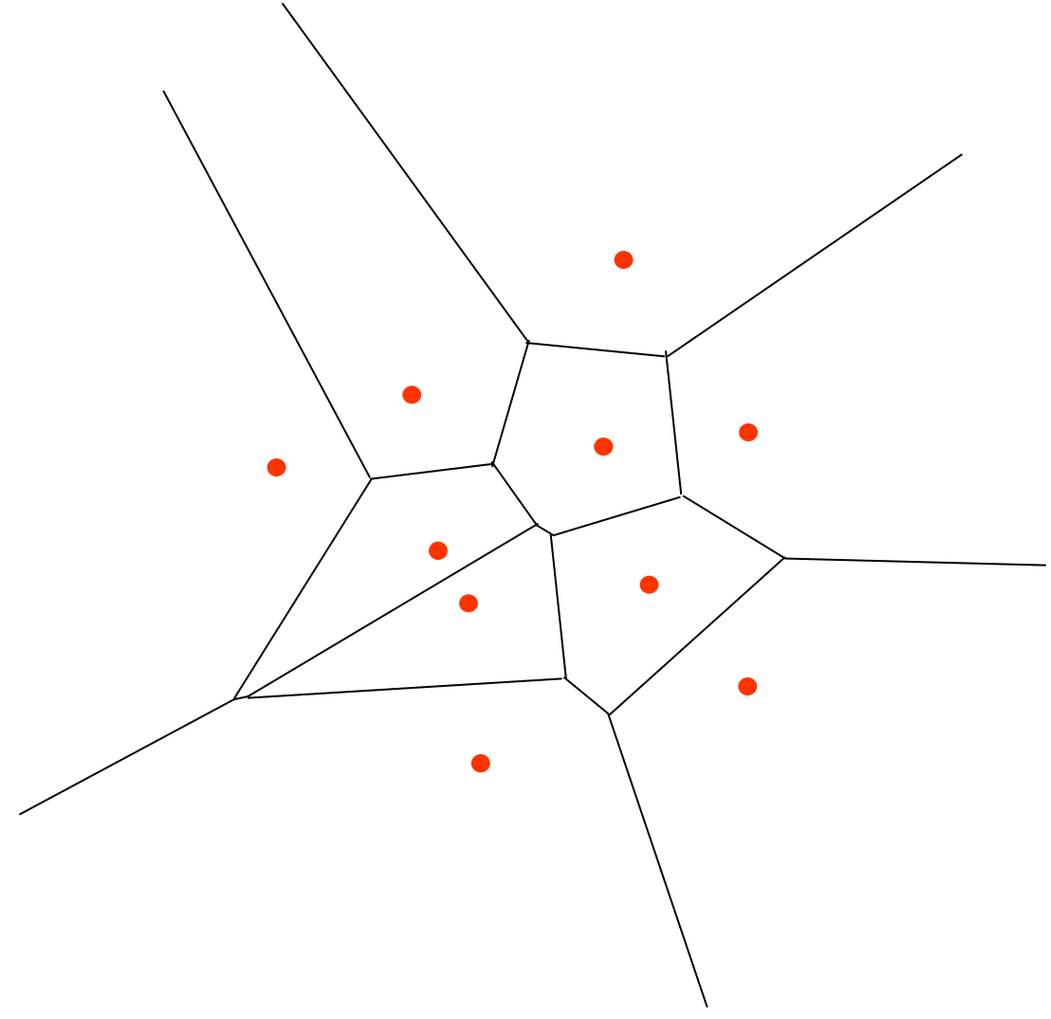
- A tessellation partitions the plane into regions
  - the Voronoi tessellation of many seeds



 copyright ©2025 m.patrignani

Voronoi tessellation

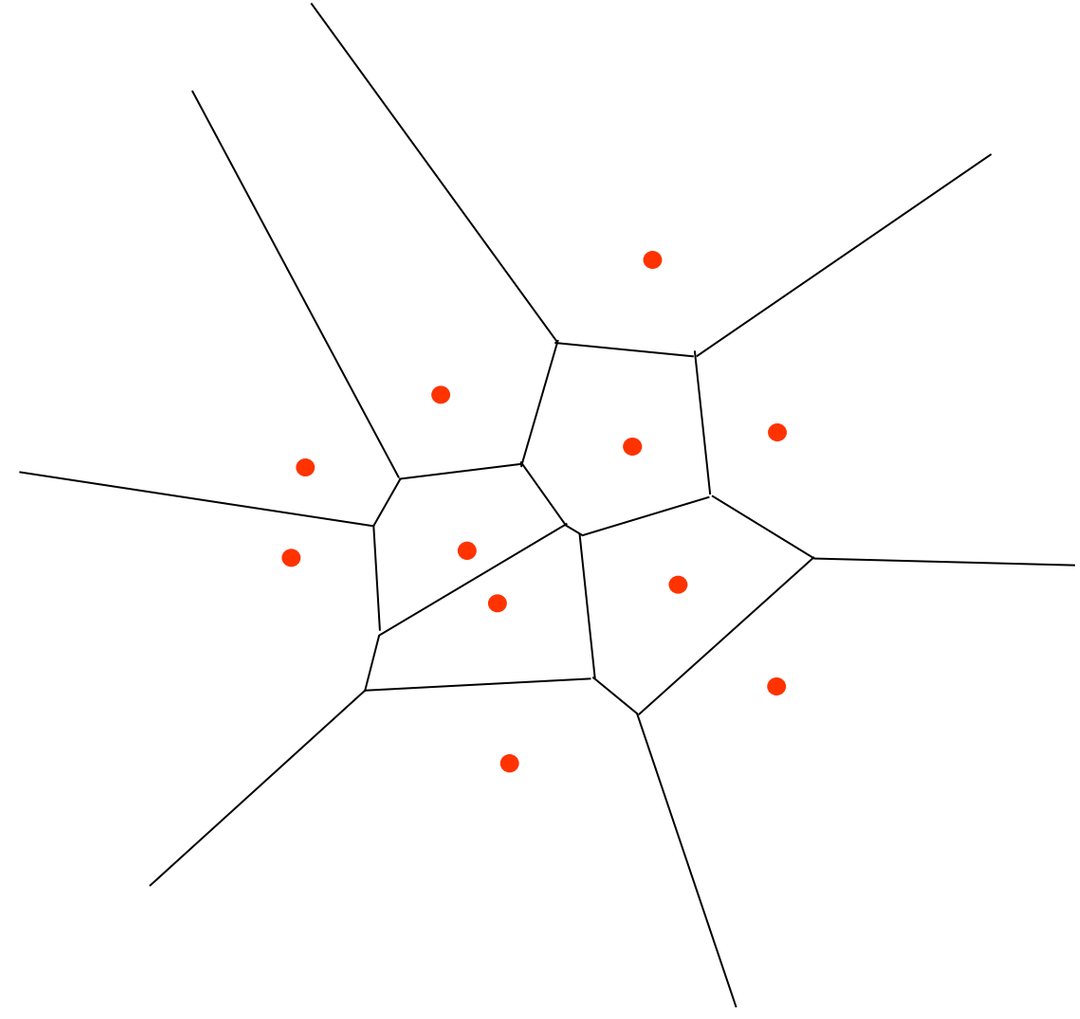
- A tessellation partitions the plane into regions
  - the Voronoi tessellation of many seeds



 copyright ©2025 m.patrignani

Voronoi tessellation

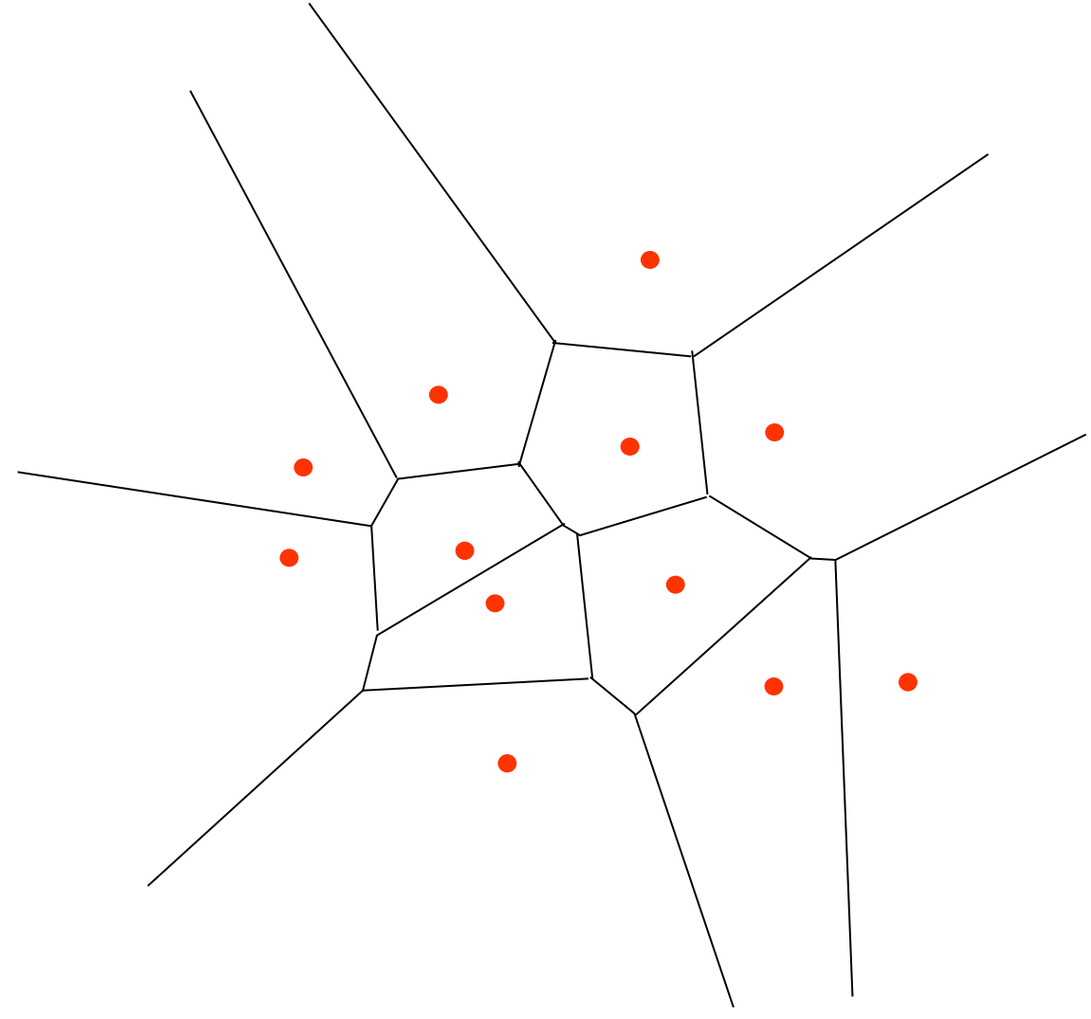
- A tessellation partitions the plane into regions
  - the Voronoi tessellation of many seeds



 copyright ©2025 m.patrignani

Voronoi tessellation

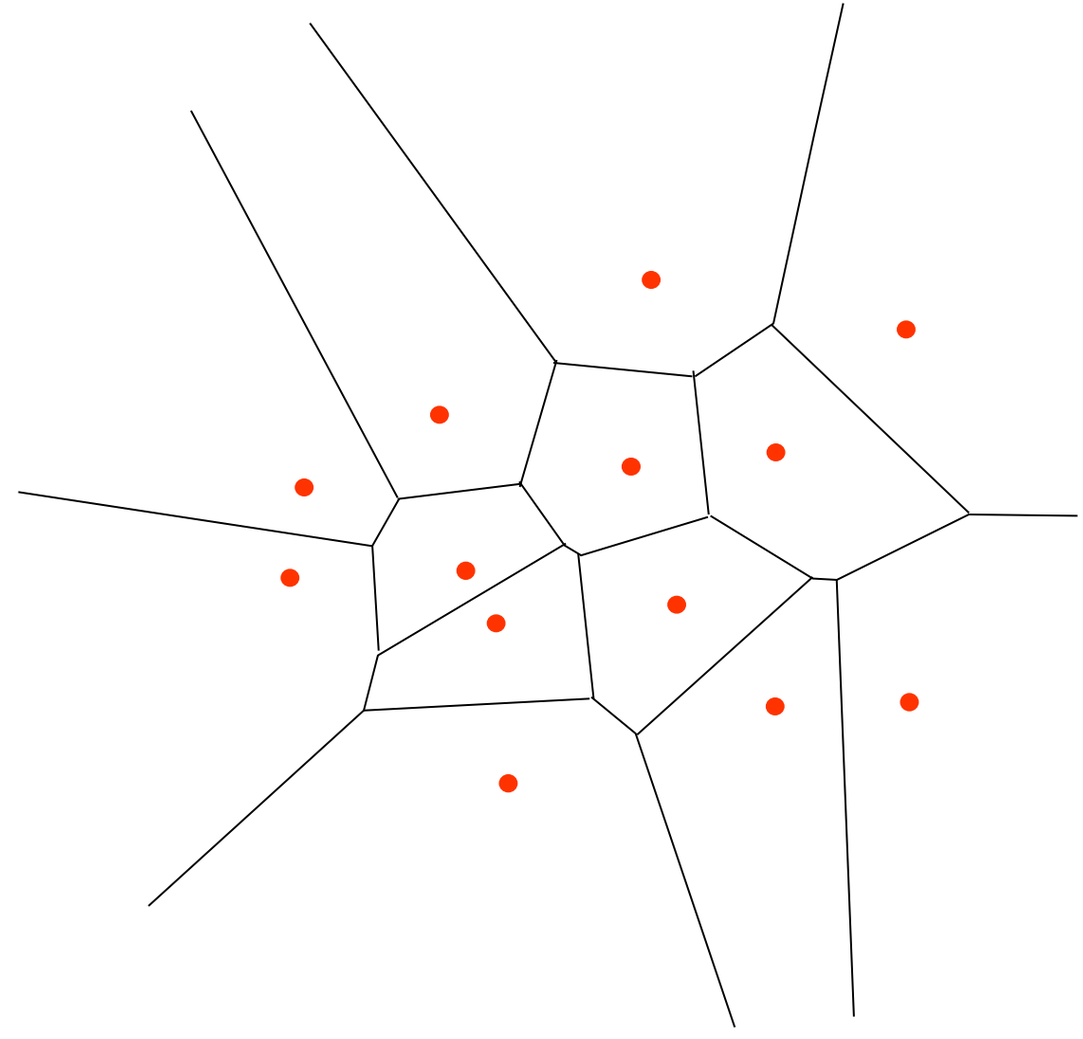
- A tessellation partitions the plane into regions
  - the Voronoi tessellation of many seeds



 copyright ©2025 m.patrignani

Voronoi tessellation

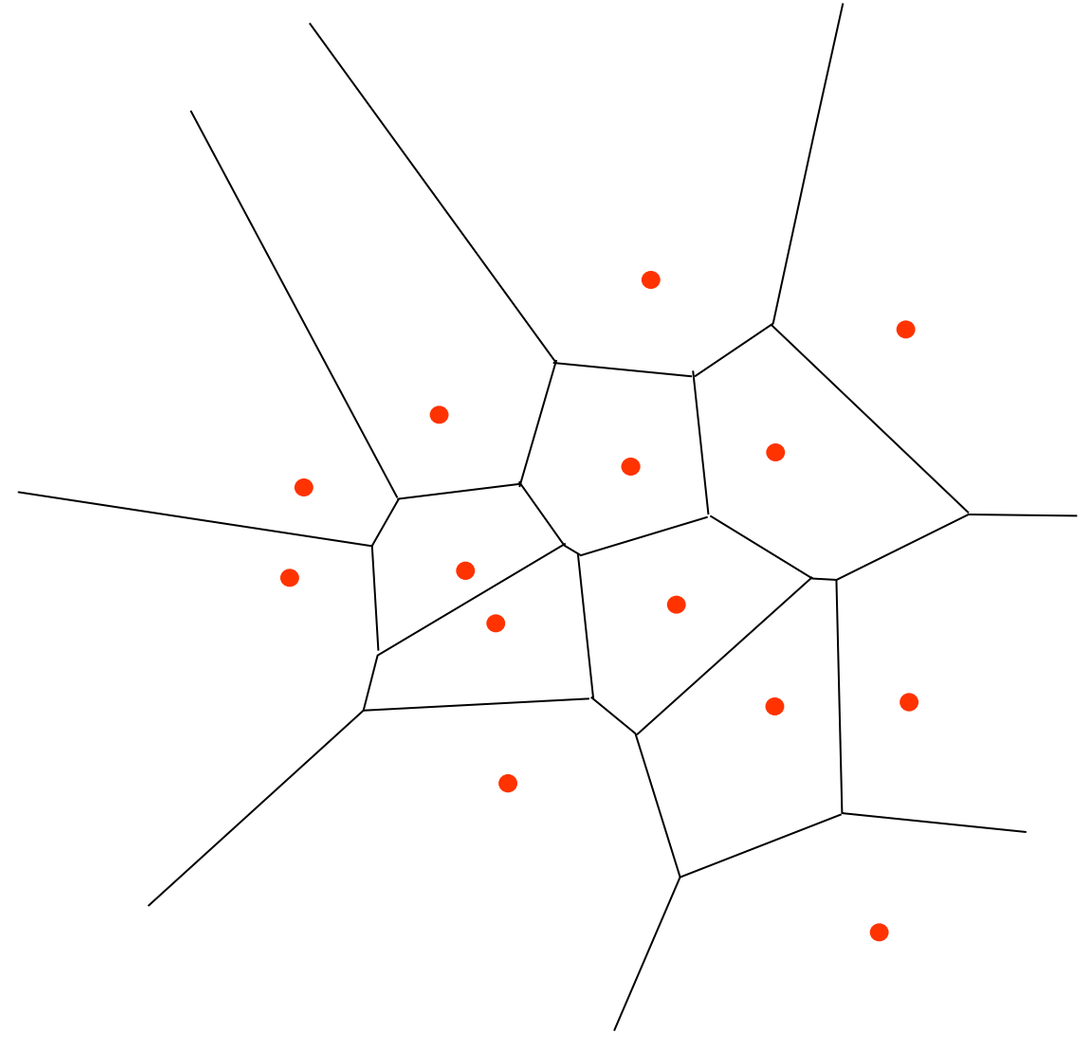
- A tessellation partitions the plane into regions
  - the Voronoi tessellation of many seeds



 copyright ©2025 m.patrignani

Voronoi tessellation

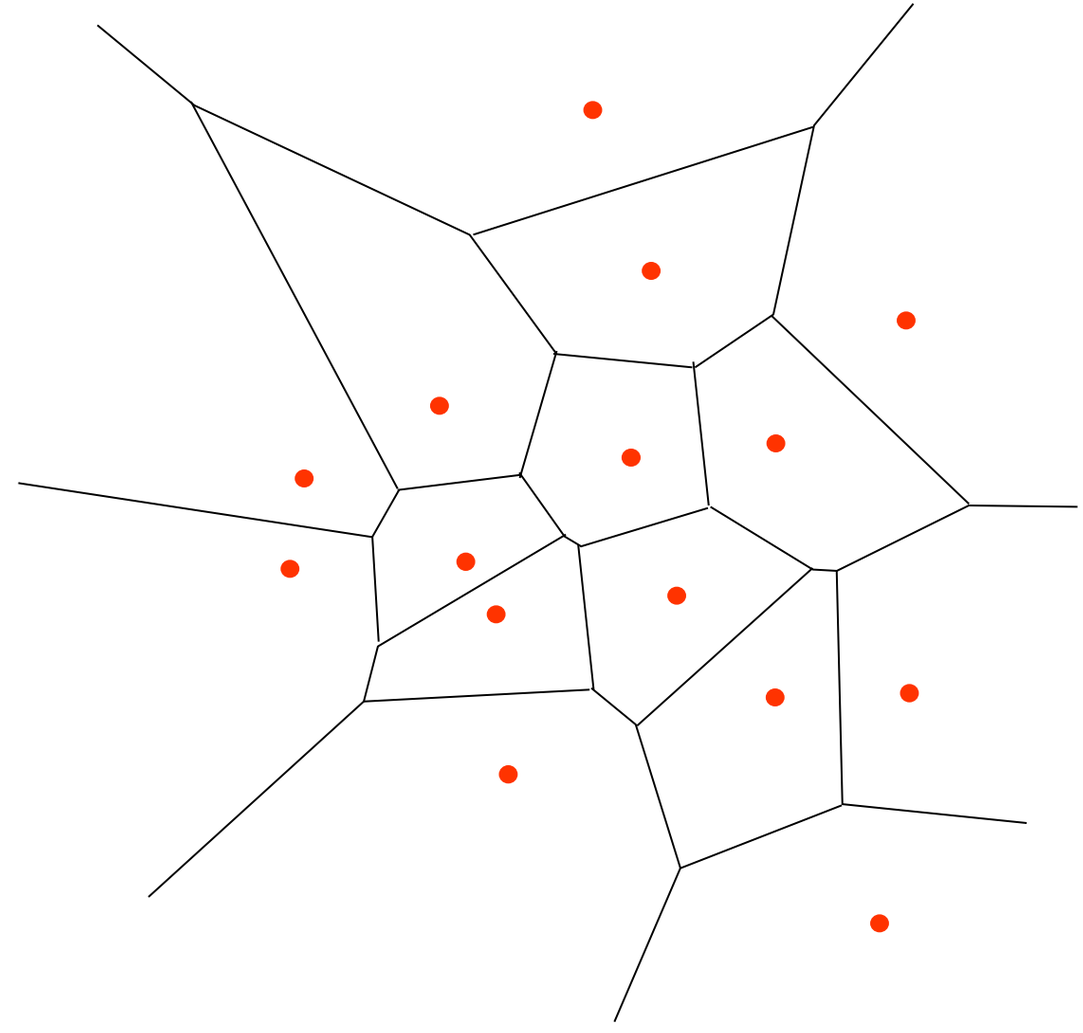
- A tessellation partitions the plane into regions
  - the Voronoi tessellation of many seeds



 copyright ©2025 m.patrignani

Voronoi tessellation

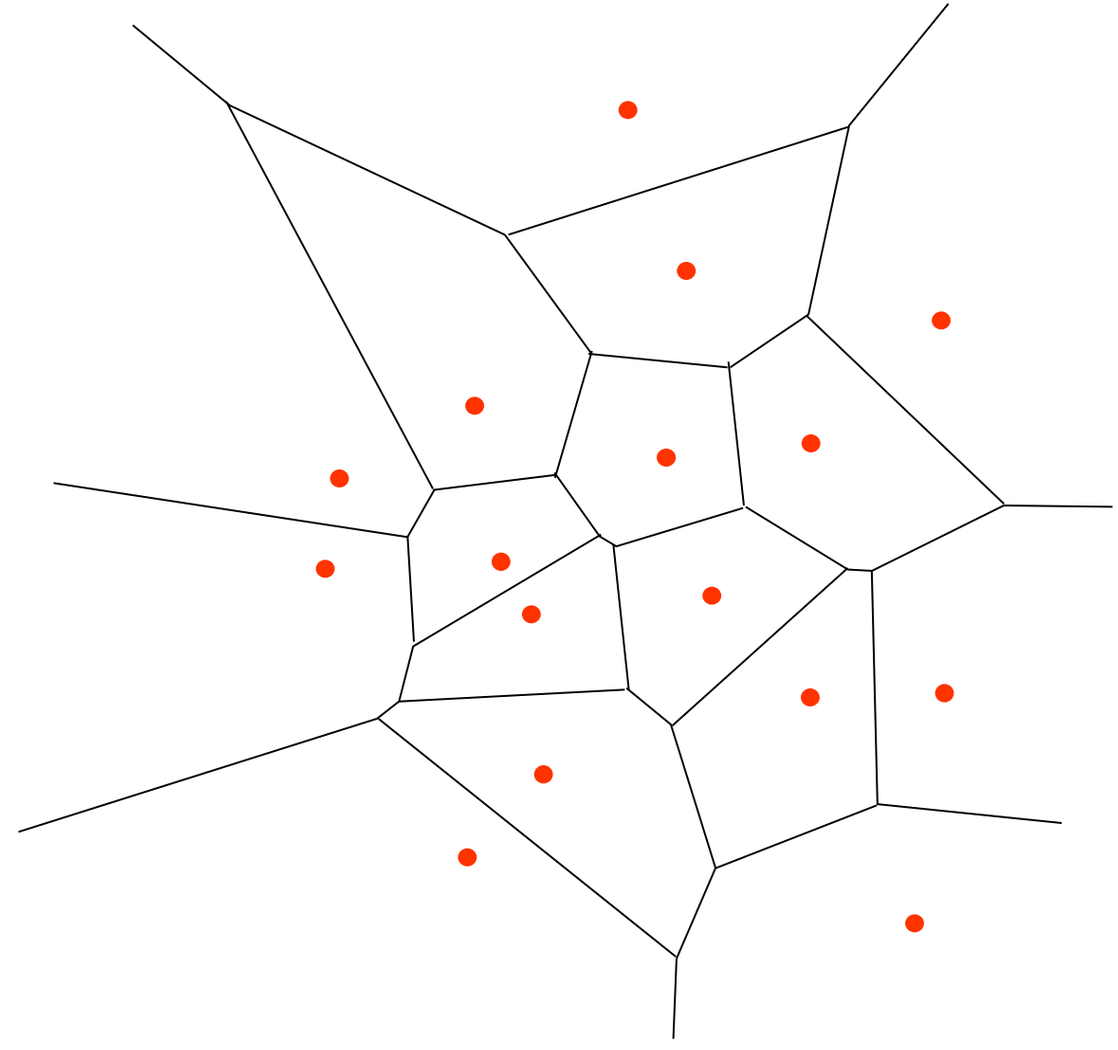
- A tessellation partitions the plane into regions
  - the Voronoi tessellation of many seeds



 copyright ©2025 m.patrignani

Voronoi tessellation

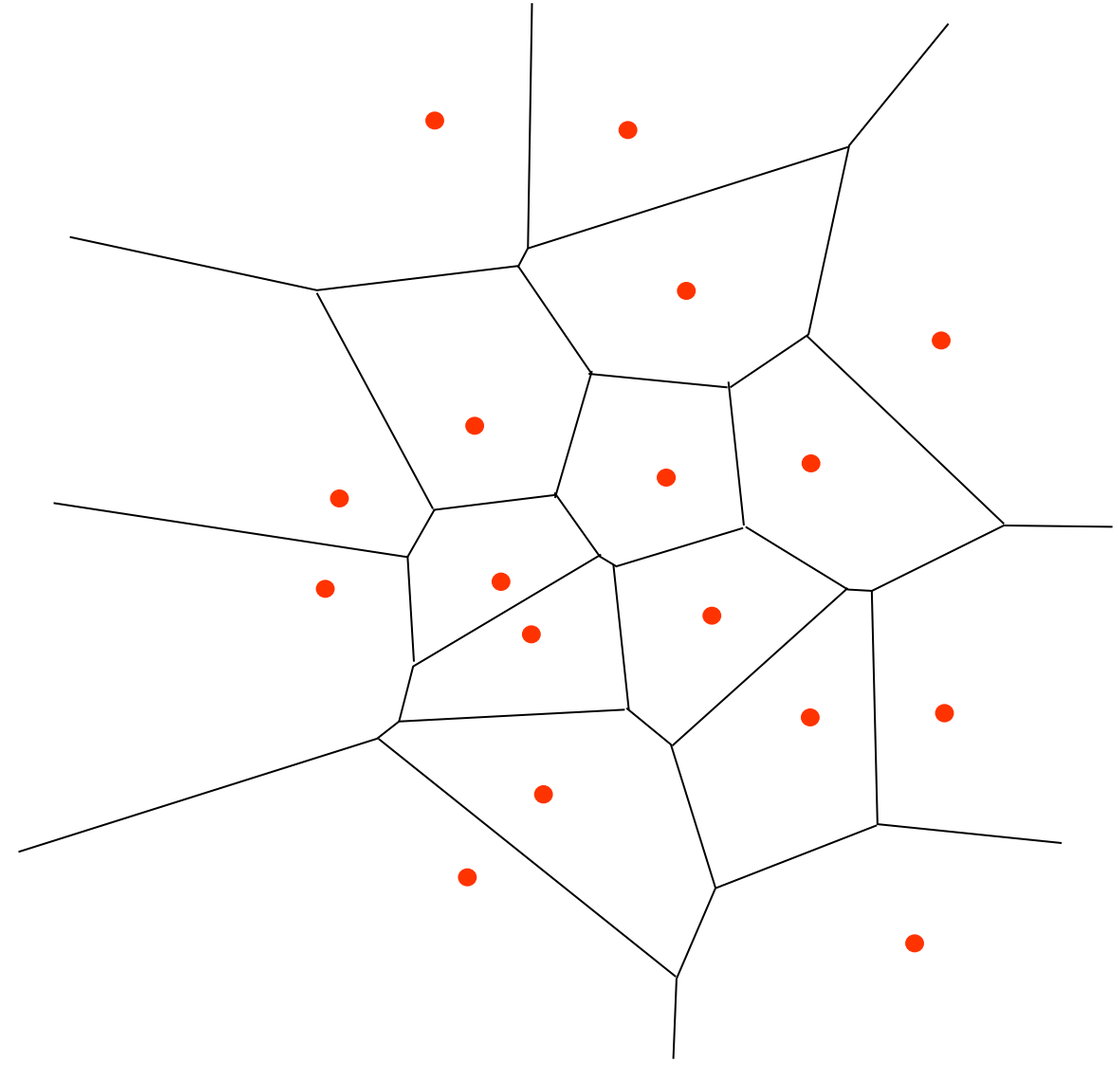
- A tessellation partitions the plane into regions
  - the Voronoi tessellation of many seeds



 copyright ©2025 m.patrignani

Voronoi tessellation

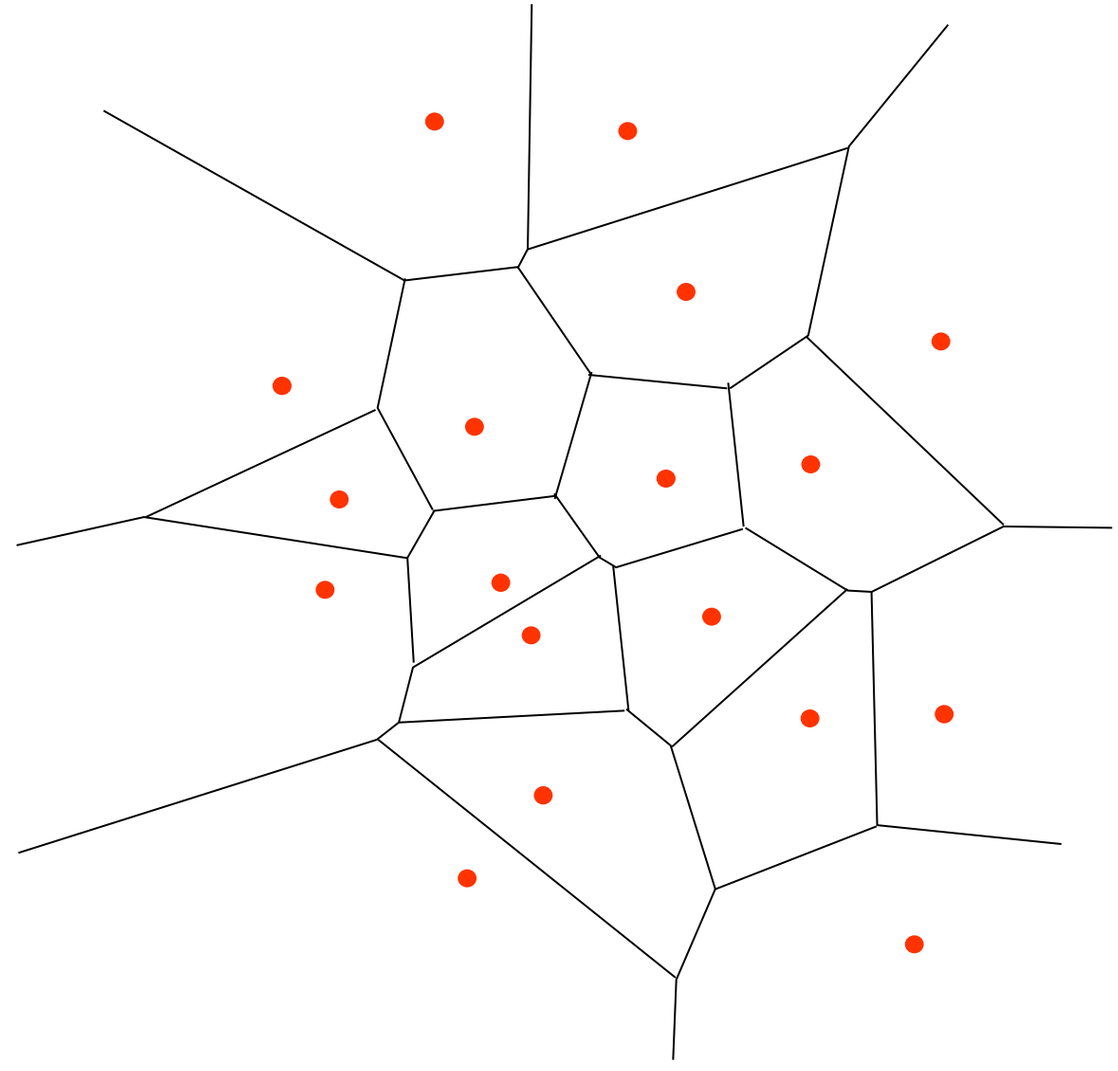
- A tessellation partitions the plane into regions
  - the Voronoi tessellation of many seeds



 copyright ©2025 m.patrignani

Voronoi tessellation

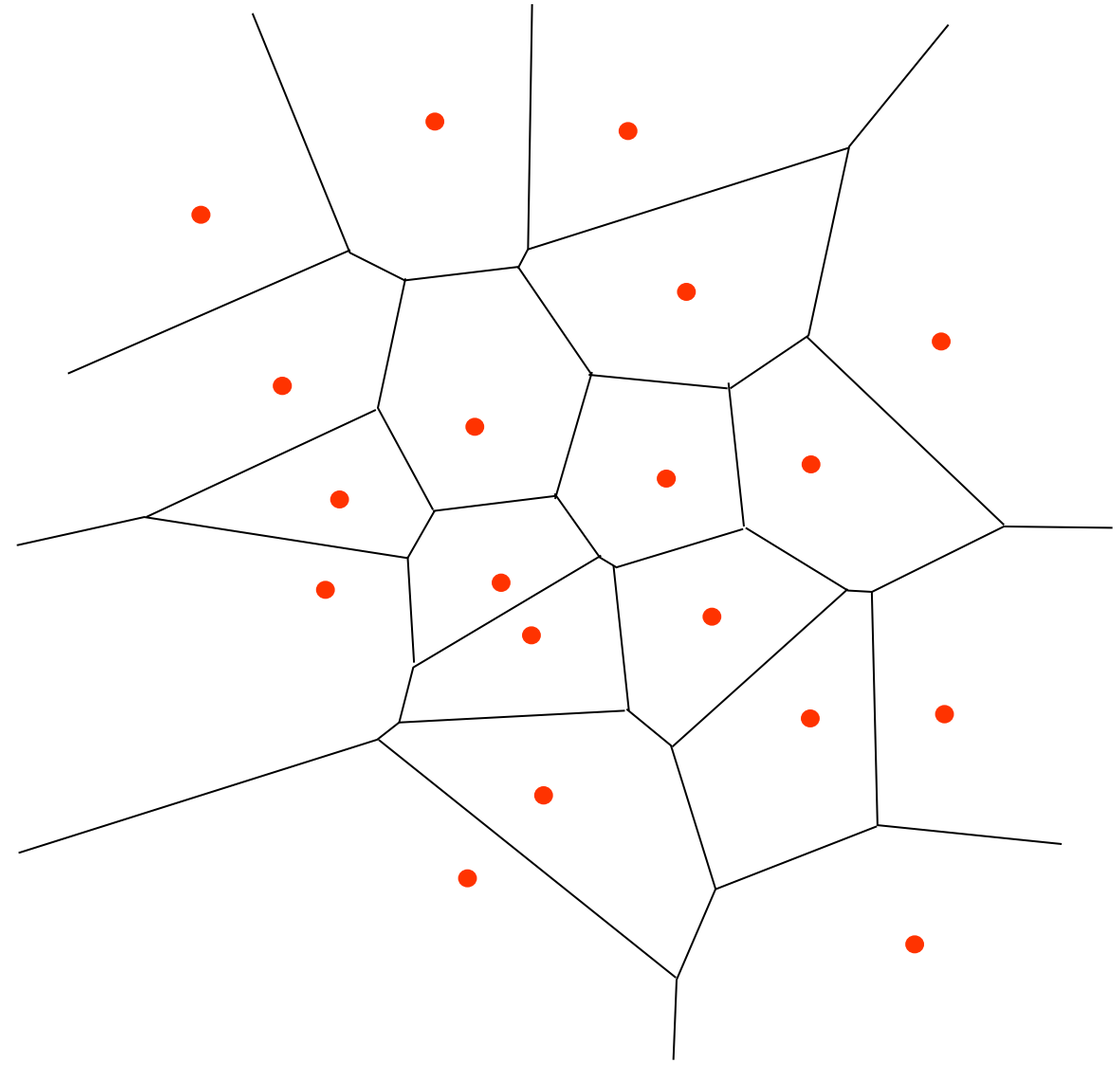
- A tessellation partitions the plane into regions
  - the Voronoi tessellation of many seeds



 copyright ©2025 m.patrignani

Voronoi tessellation

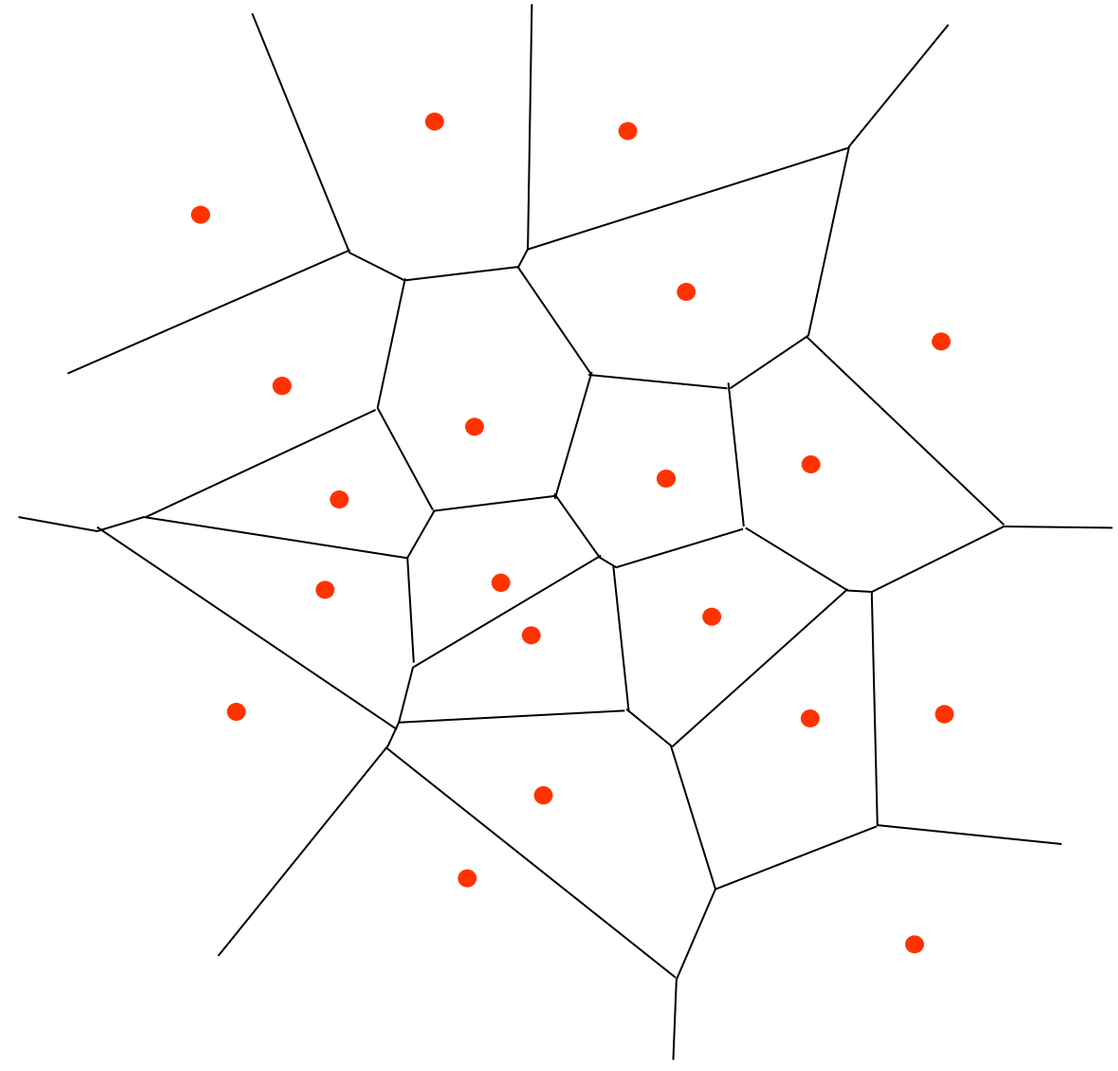
- A tessellation partitions the plane into regions
  - the Voronoi tessellation of many seeds



copyright ©2025 m.patrignani

Voronoi tessellation

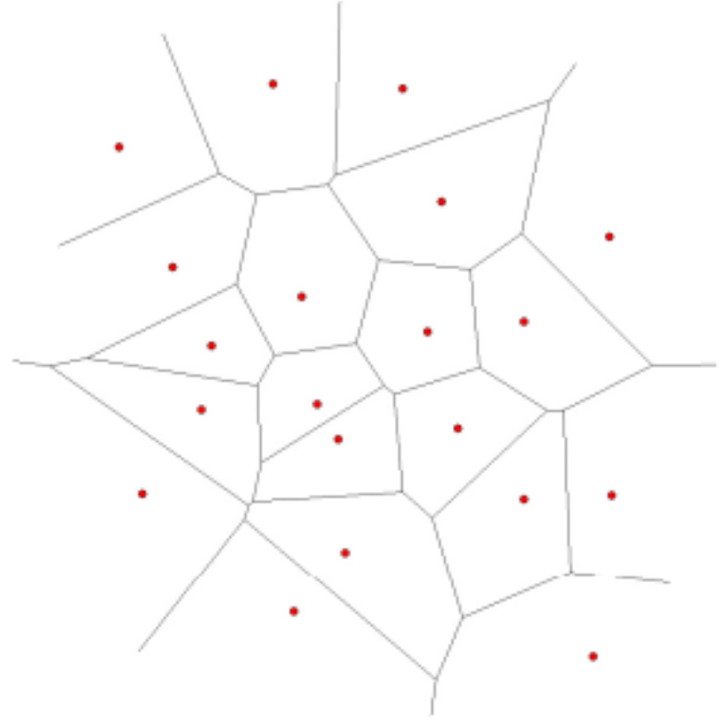
- A tessellation partitions the plane into regions
  - the Voronoi tessellation of many seeds



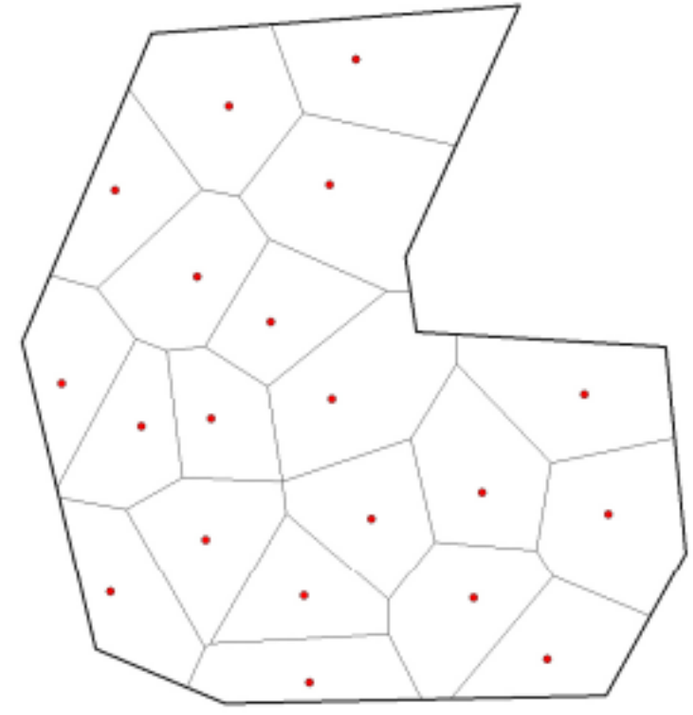
 copyright ©2025 m.patrignani

Voronoi tessellation

- Each region contains a seed
- Each region is composed by the points nearer to the region seed than to other seeds



Unbounded Voronoi tessellation



Bounded Voronoi tessellation

copyright ©2025 m.patrignani

Weighted distance function

- Since the nodes of the input tree are weighted, we need a weighted distance function
  - let *s* be a seed with weight w_s
  - let *p* be a point of the plane
- The *additive weighted distance function* is

$$d_{AW}(s, p) = \|s - p\| - w_s$$

- Where

$$\|s - p\| = \sqrt{(x_s - x_p)^2 + (y_s - y_p)^2}$$

copyright ©2025 m.patrignani

Power weighted distance function

- Alternative distance functions may be proposed
- The *power weighted distance function* is the following

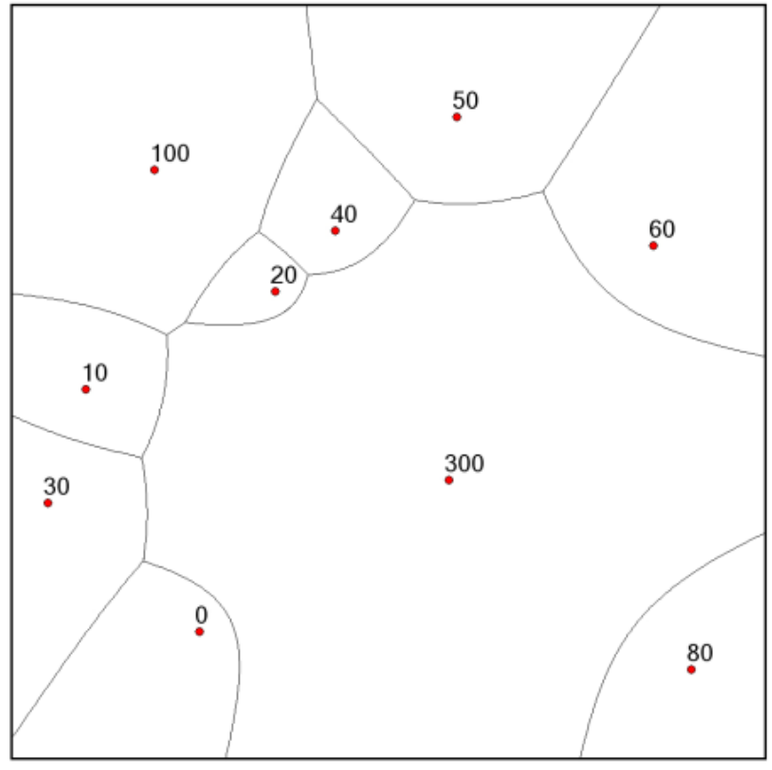
$$d_{PW} = \|s - p\|^2 - w_s$$

$$d_{PW} = (x_s - x_p)^2 + (y_s - y_p)^2 - w_s$$

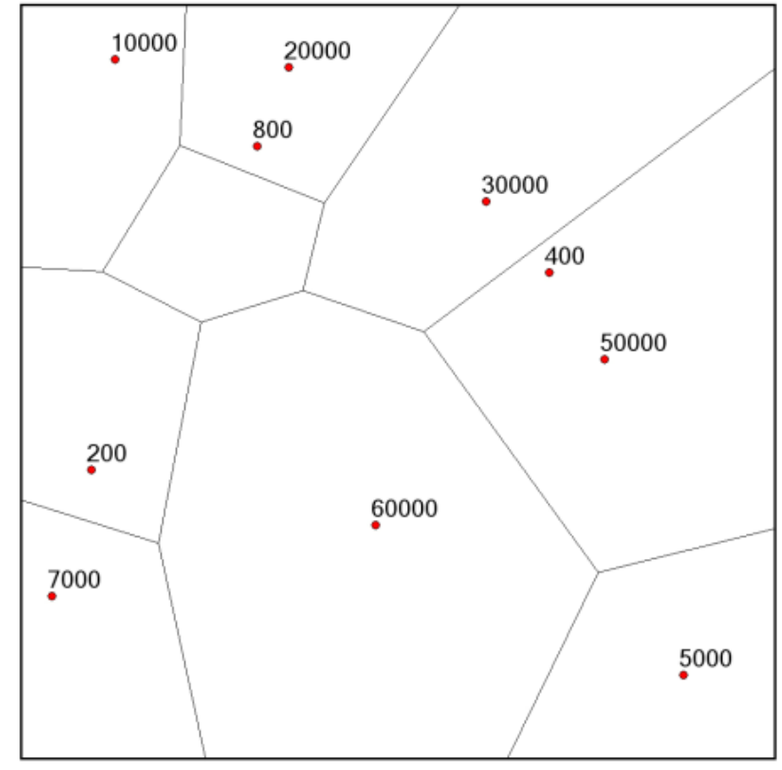
 copyright ©2025 m.patrignani

Additive and power distance

- Different distance functions yield different appearences of the Voronoi tessellation



Additive weighted distance function



Power weighted distance function

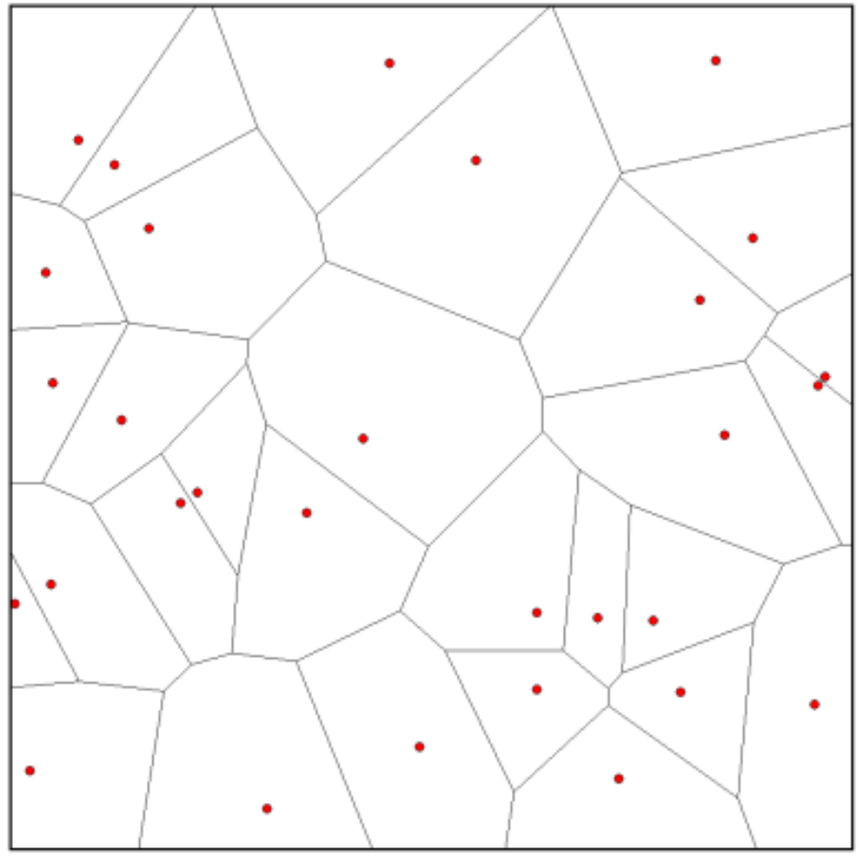
copyright ©2025 m.patrignani

Placing the seeds

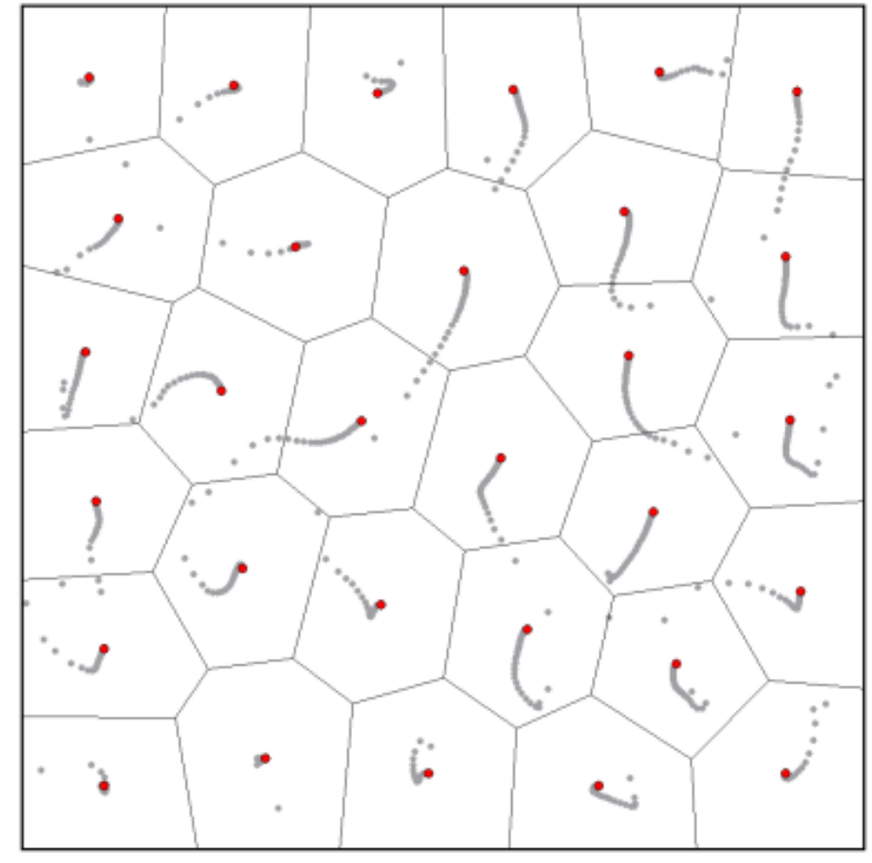
- Start by a random placement of the seeds inside a given polygonal region
- For each seed *s*, compute the centroid c_s of the Voronoi region associated with *s*
  - the centroid is the “center of mass” of the convex polygon associated to *s*
  - the centroid is always internal to such a polygon
- Iteratively move each seed *s* towards its centroid c_s

 copyright ©2025 m.patrignani

Example of seed movement



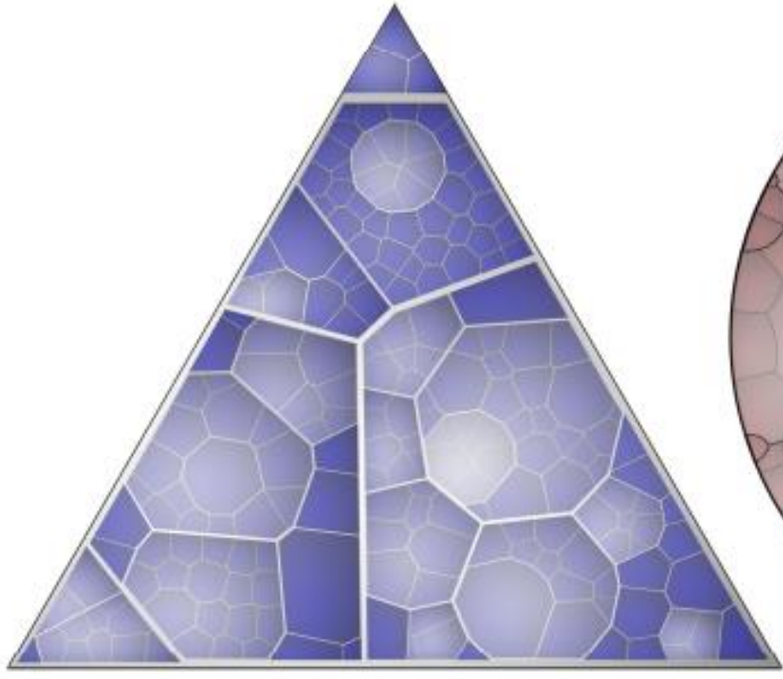
A random placement



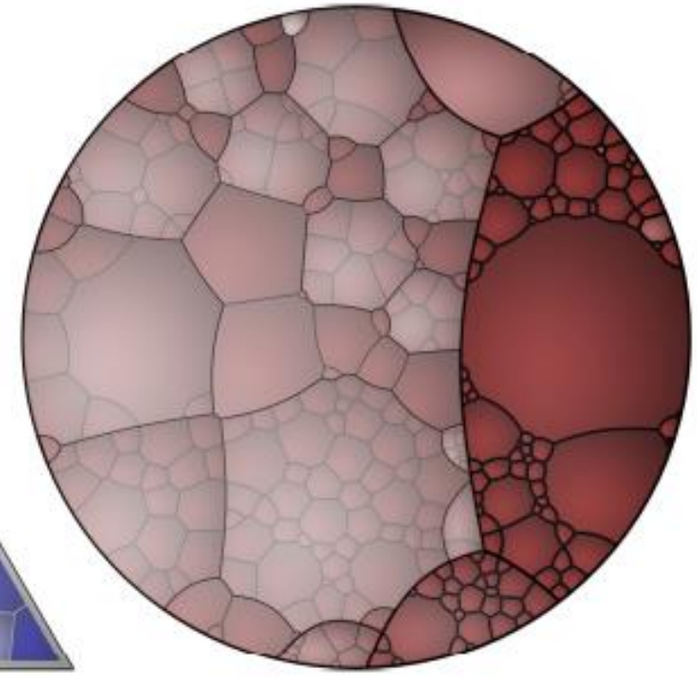
Tracing centroid movements

copyright ©2025 m.patrignani

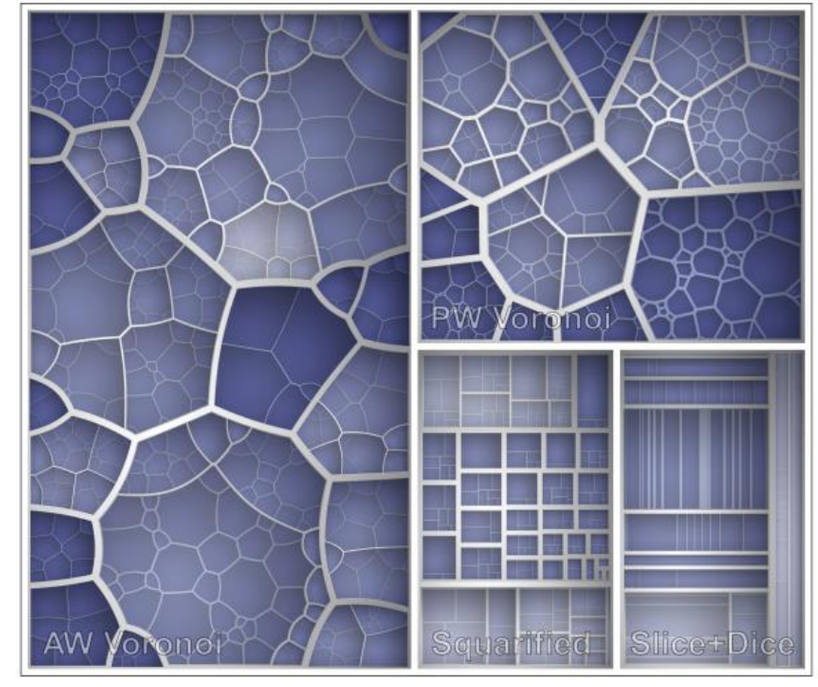
Example representations



Power weighted distance function



Additive weighted distance function

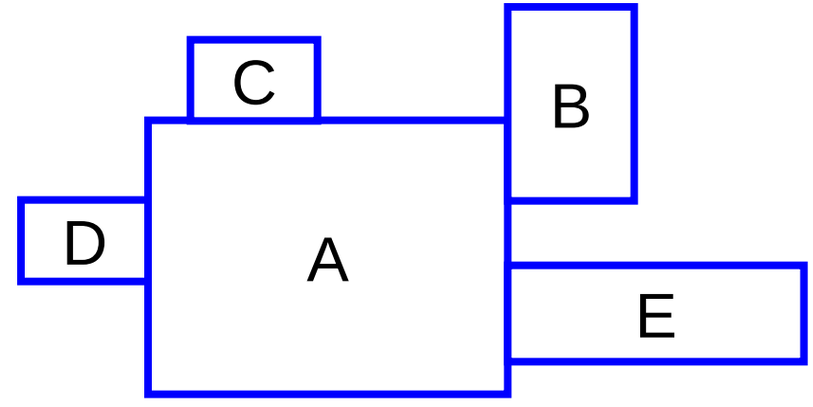


Hybrid representation

copyright ©2025 m.patrignani

Implicit representation by contact

- The relationship between a parent and a child in the tree is represented by the contact of two regions of the plane

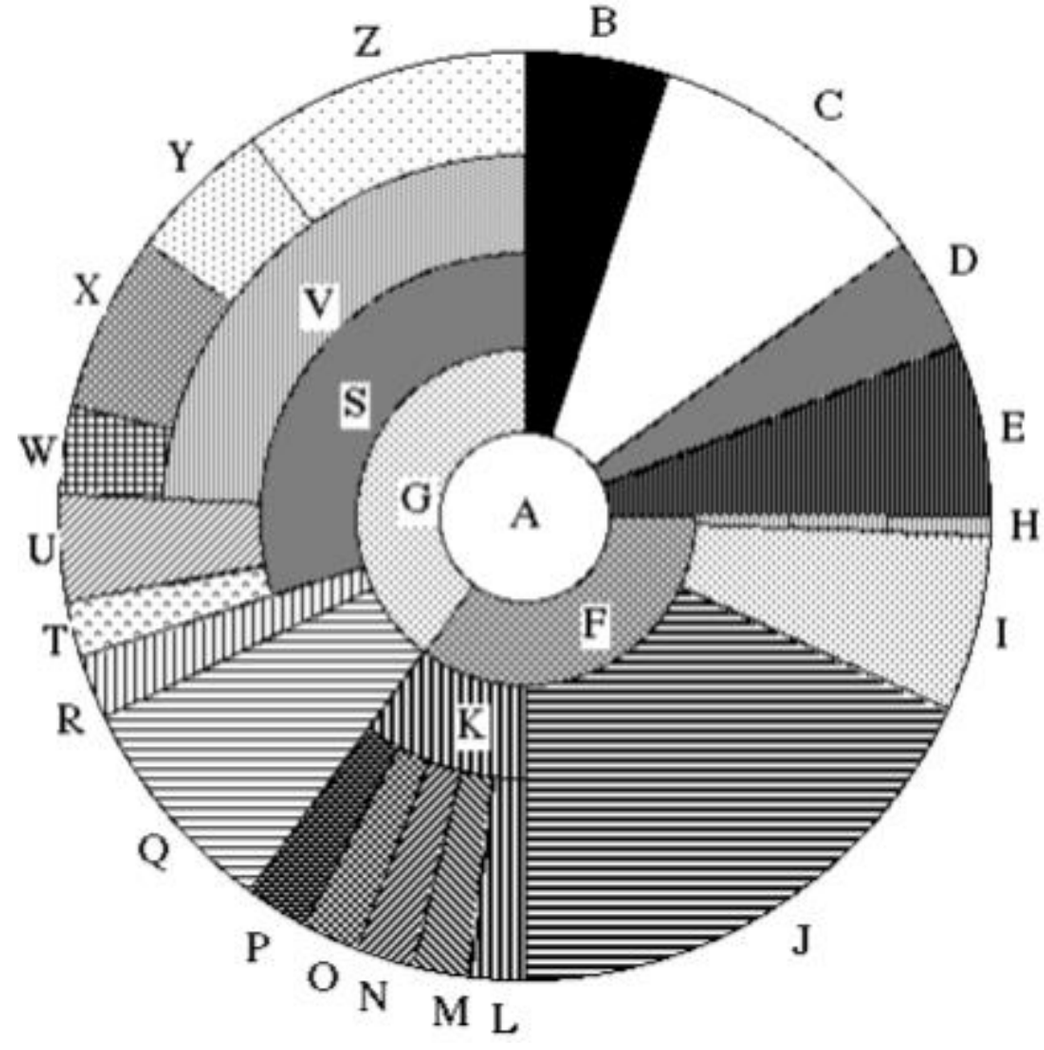


implicit layout by contact

 copyright ©2025 m.patrignani

Polar treemaps

- Represents a tree where each node has a weight
- Several variants described by [Johnson 93]

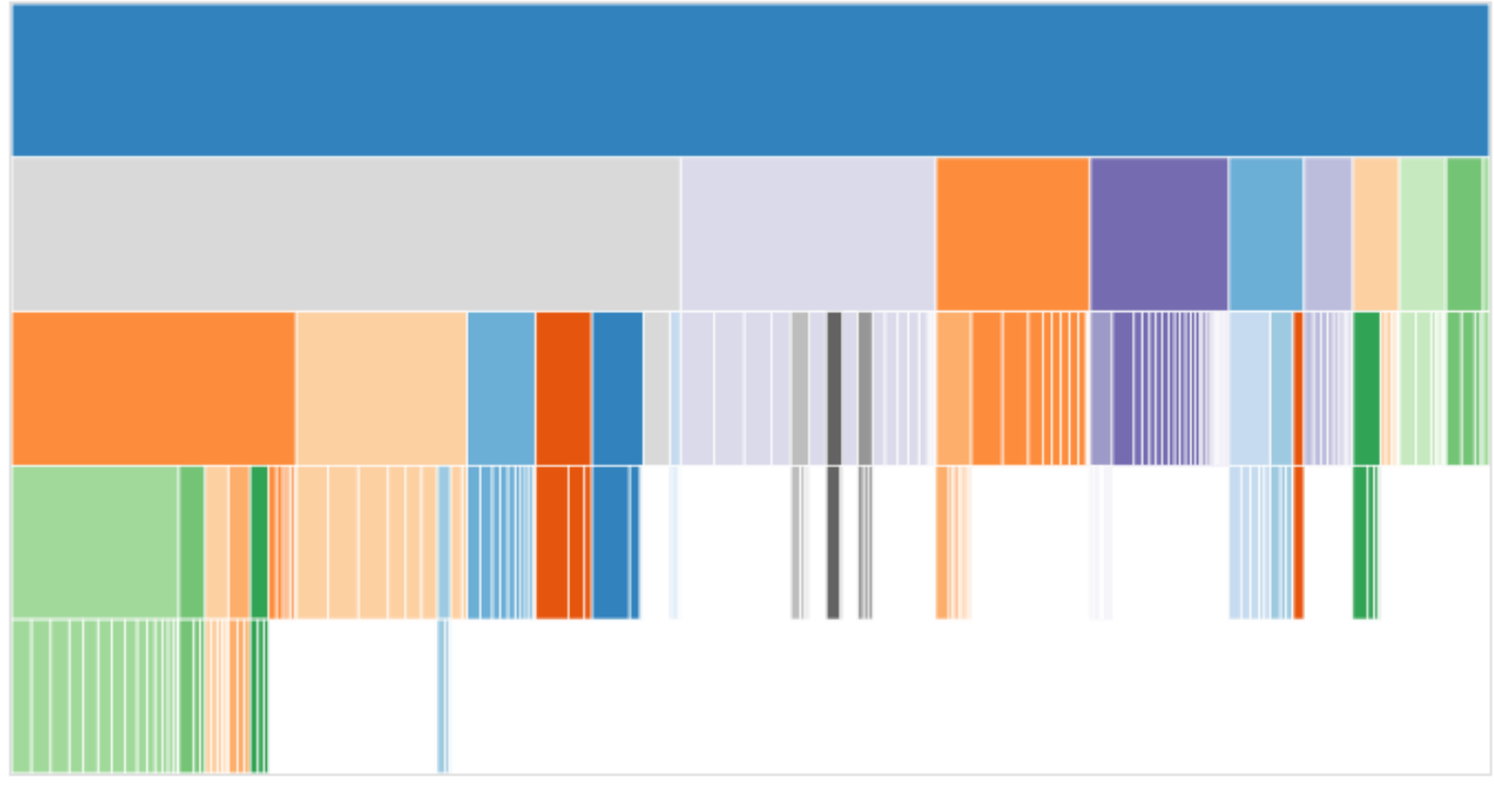


Picture from [Johnson 93]

 copyright ©2025 m.patrignani

Zoomable Icicle

- The tree is described by the vertical contact of rectangles

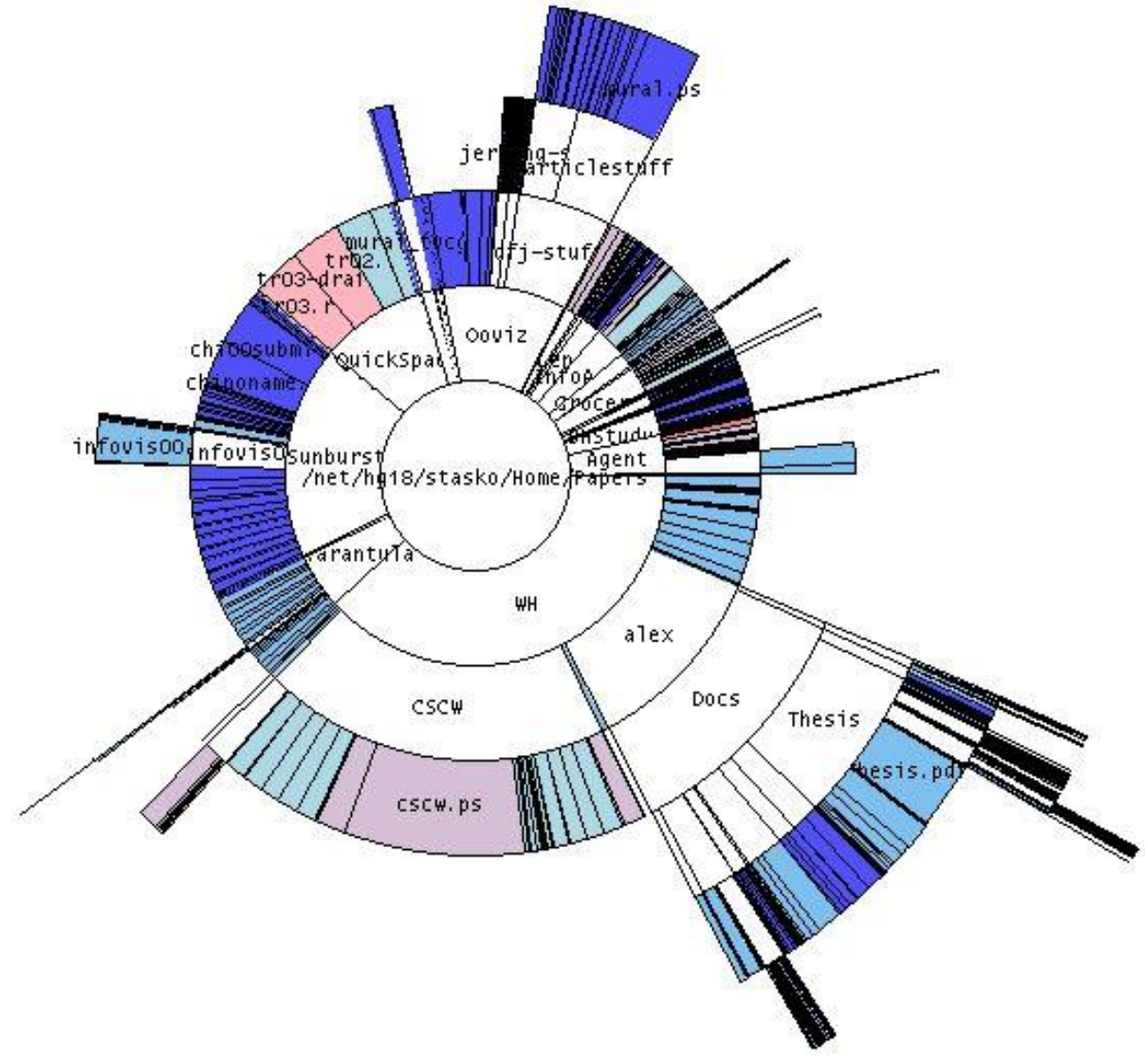


http://bl.ocks.org/mbostock/1005873

 copyright ©2025 m.patrignani

SunBurst

- Radial layout
- Root at the center
- Weight of nodes is shown by the angles of their sections
  - [Stasko & Zhang 00]



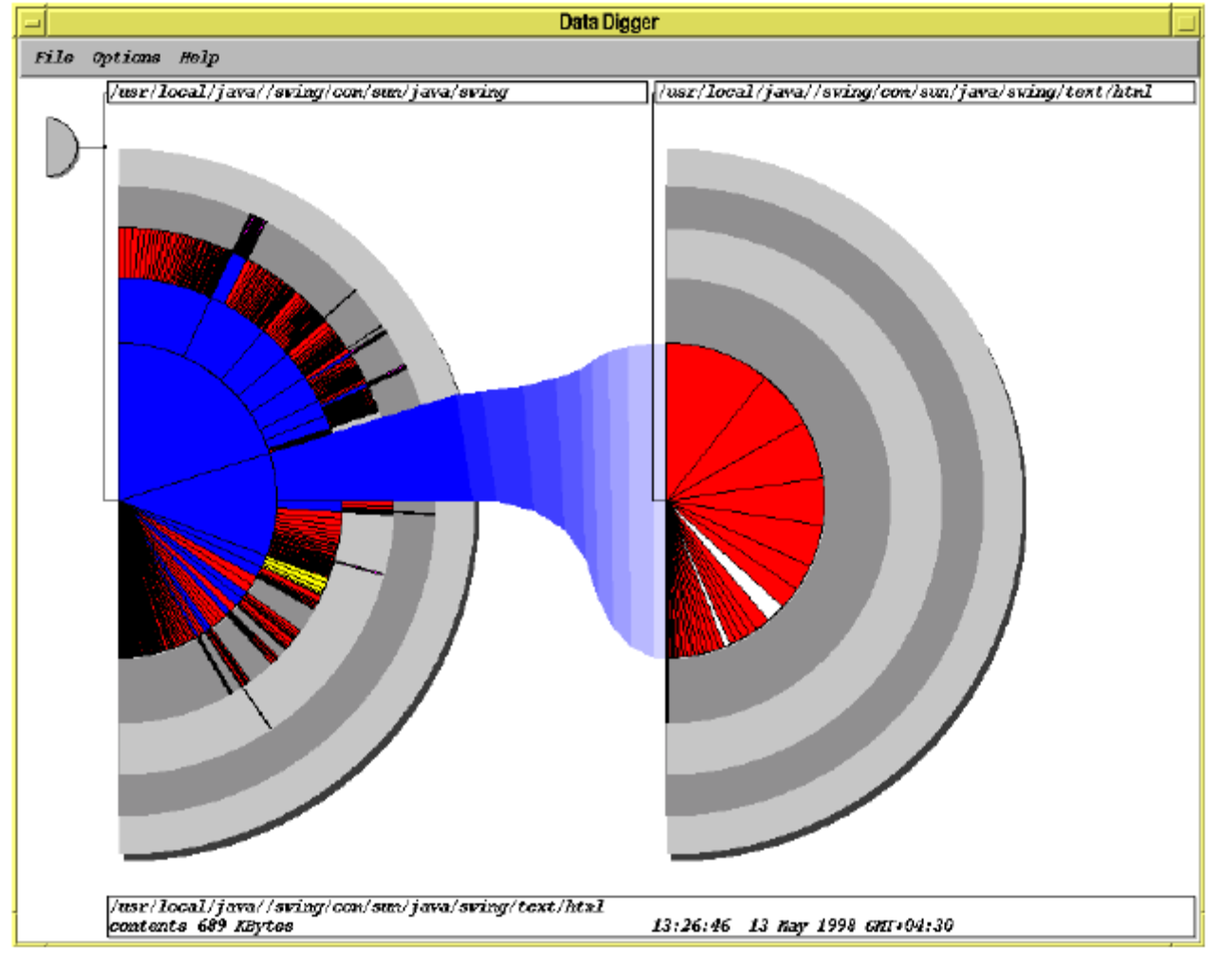
http://www.cc.gatech.edu/gvu/ii/sunburst/

 copyright ©2025 m.patrignani

SunBurst exploration

- Connected semicircle [Andrews & Heidegger 98]

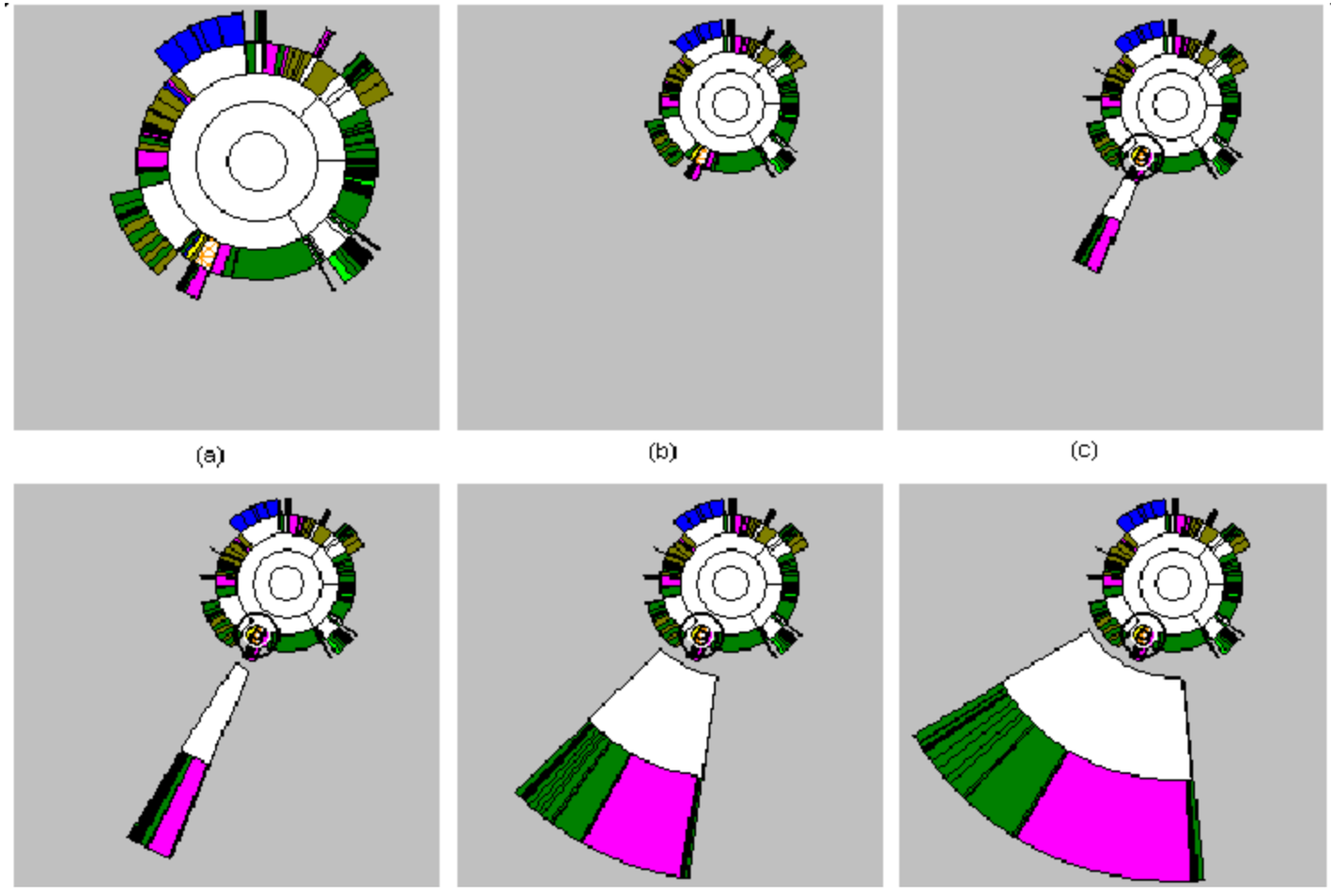
Picture from [Stasko & Zhang 00]



 copyright ©2025 m.patrignani

SunBurst exploration

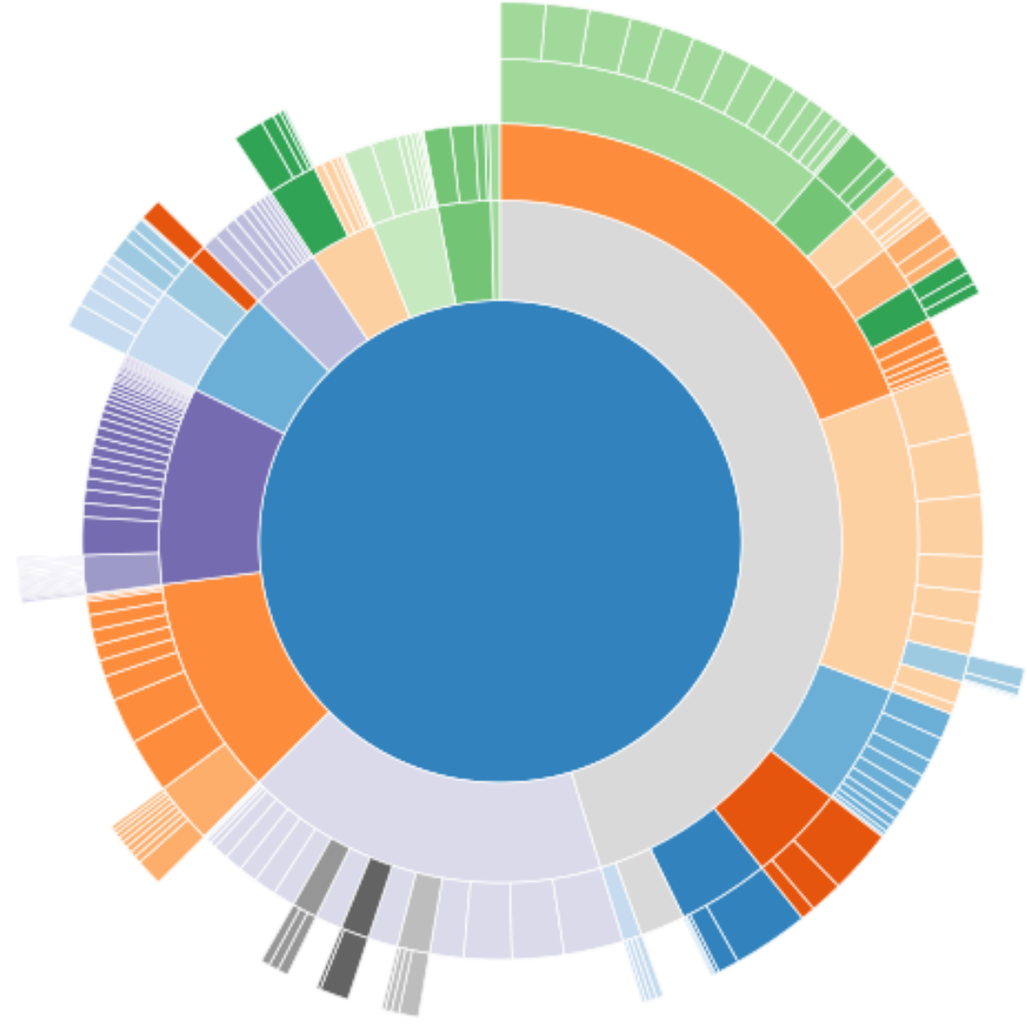
- “Detail inside”, “detail outside”, “angular detail”
  - see video (especially from 00:03:30 on)



ftp://ftp.cc.gatech.edu/pub/people/stasko/movies/sunburst-infovis00.mpg

SunBurst exploration: zooming

- Zooming hides the part of the hierarchy that is not of interest at the moment
  - play with the tool following the link

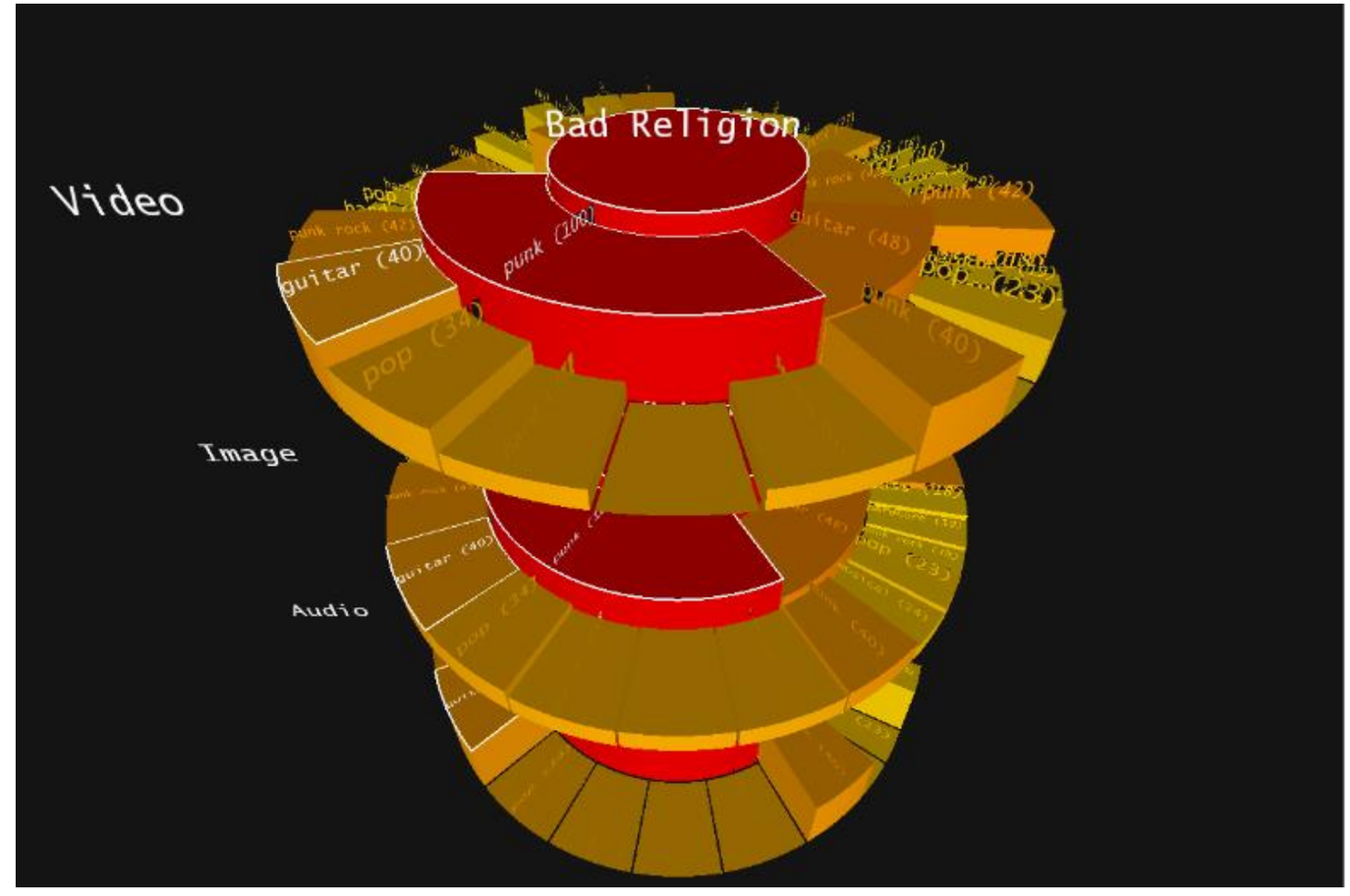


http://bl.ocks.org/mbostock/4348373

copyright ©2025 m.patrignani

3D SubBurst

- Allows for encoding additional data in the height of each arc segment
- 3D SunBurst can be stacked to compare different data
  - [Schedl et al. 07]

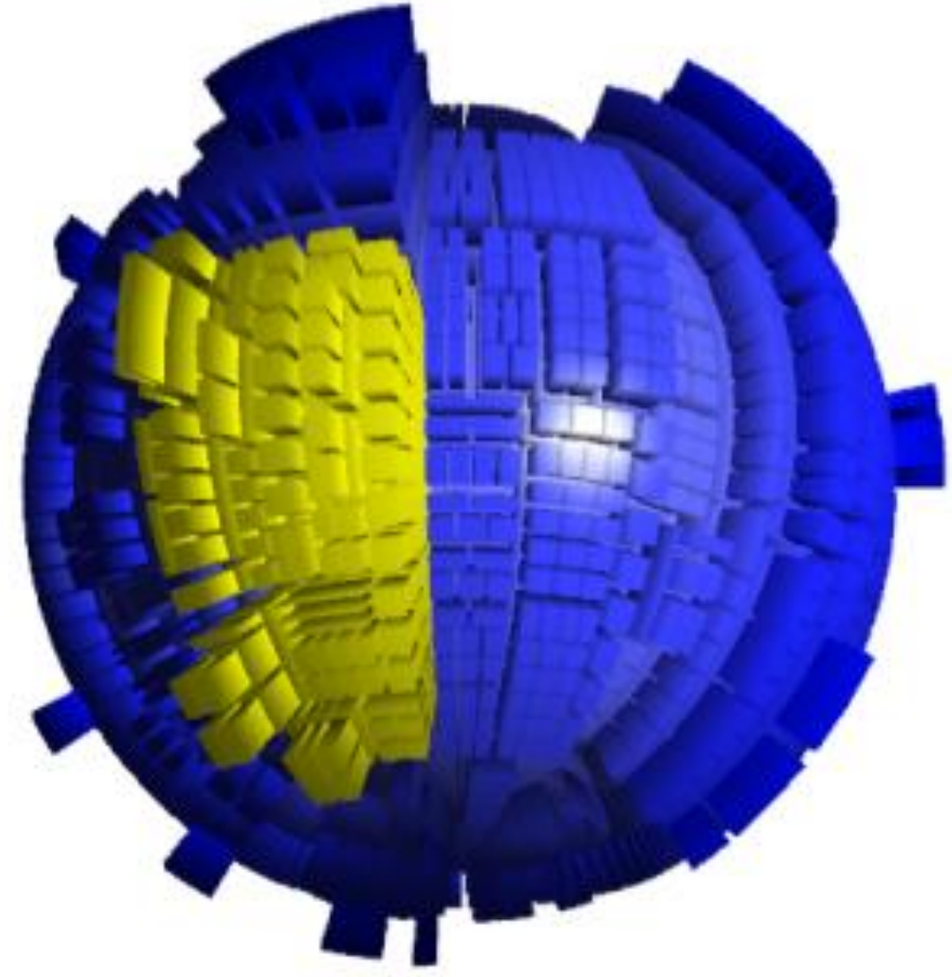


Picture from [Schedl et al. 07]

copyright ©2025 m.patrignani

3D spherical SunBurst

- [Schulz *et al.* 11]
  - surveys the design space for implicit tree representation
  - exploits new models, such as the sphere



Picture from [Schulz *et al.* 11]

copyright ©2025 m.patrignani

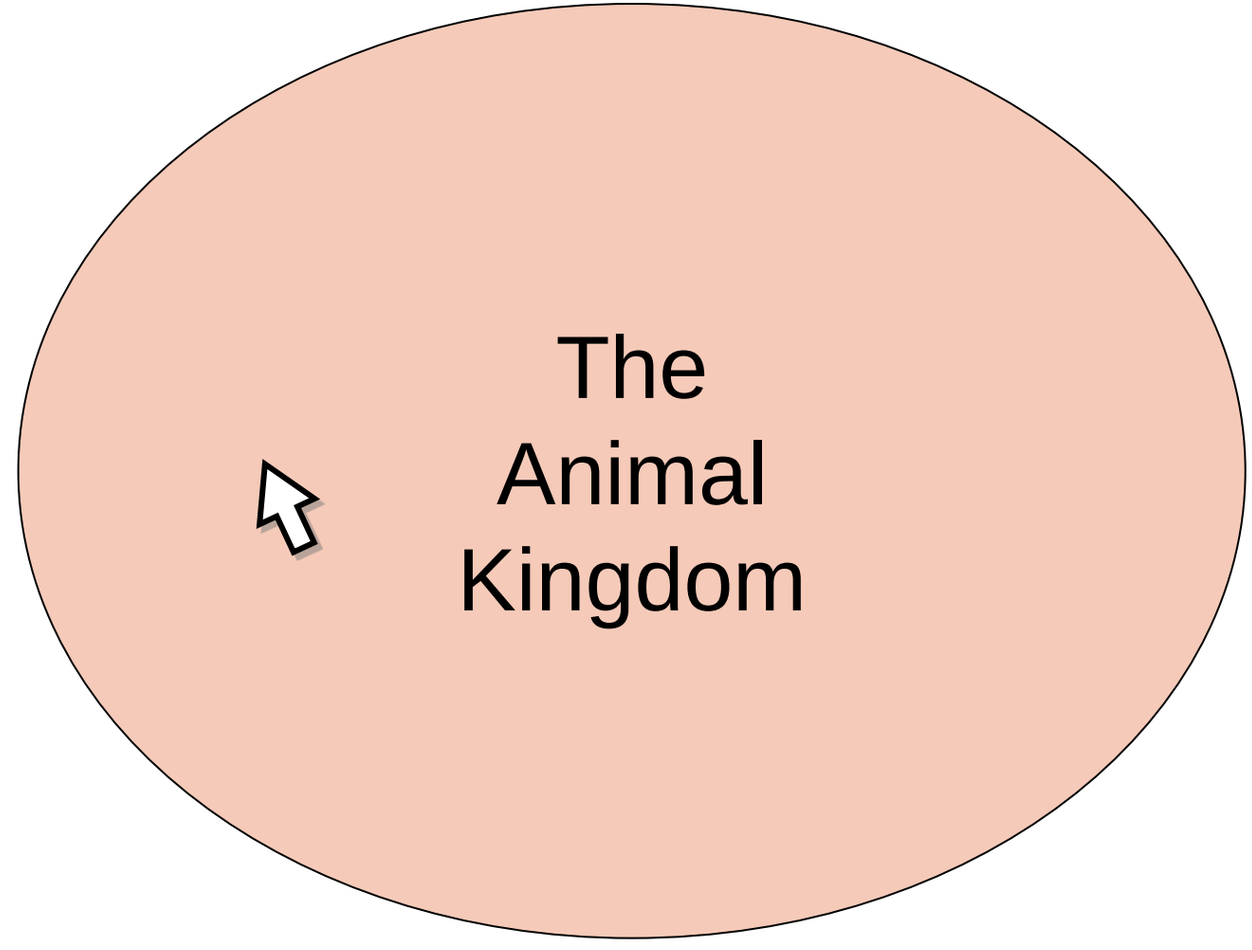
Hybrid representations

- Mix two representation paradigms
  - node-link
  - space filling
- Examples
  - Bubble trees
    - use node-link hints to explicit the tree structure in a space filling representation
  - HP-drawings
    - represent host-parasite co-phylogenetic trees
    - use node-link paradigm for the parasites and space filling paradigm for the hosts

 copyright ©2025 m.patrignani

Hybrid representations: bubble trees

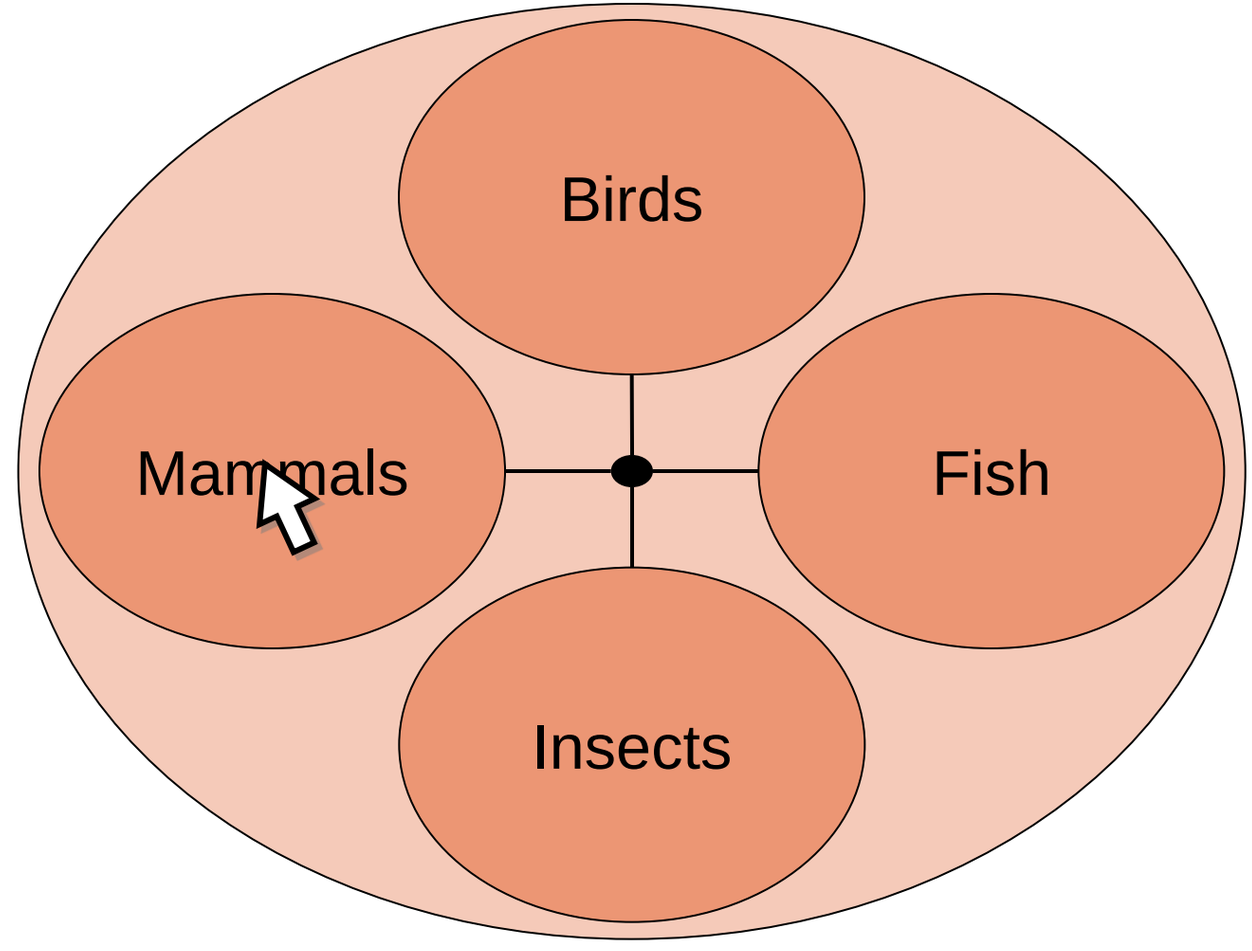
- Mixed node-link and space filling representation
- Allows to interactively browse a tree
  - [Boardman 00]



 copyright ©2025 m.patrignani

Hybrid representations: bubble trees

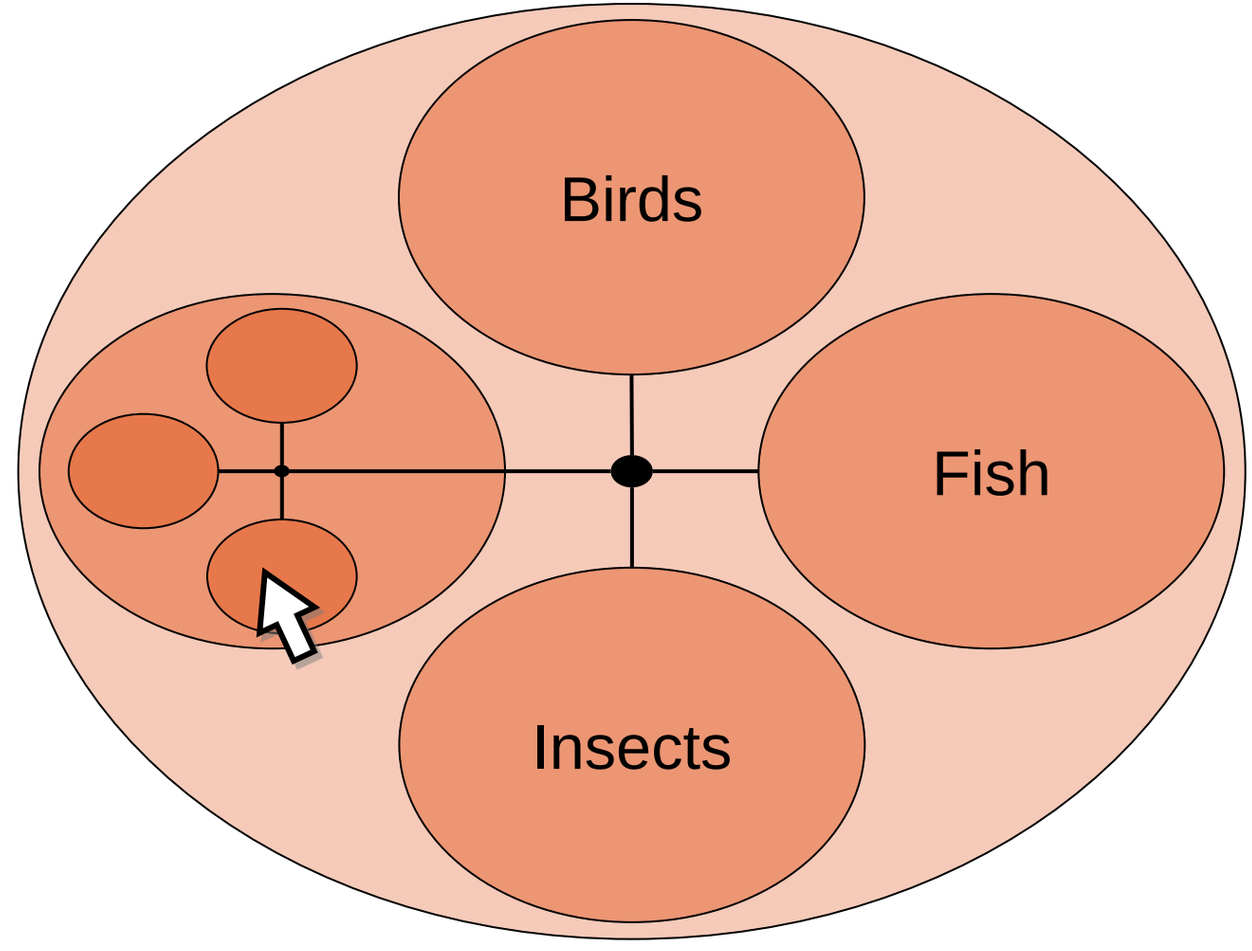
- Mixed node-link and space filling representation
- Allows to interactively browse a tree
  - [Boardman 00]



 copyright ©2025 m.patrignani

Hybrid representations: bubble trees

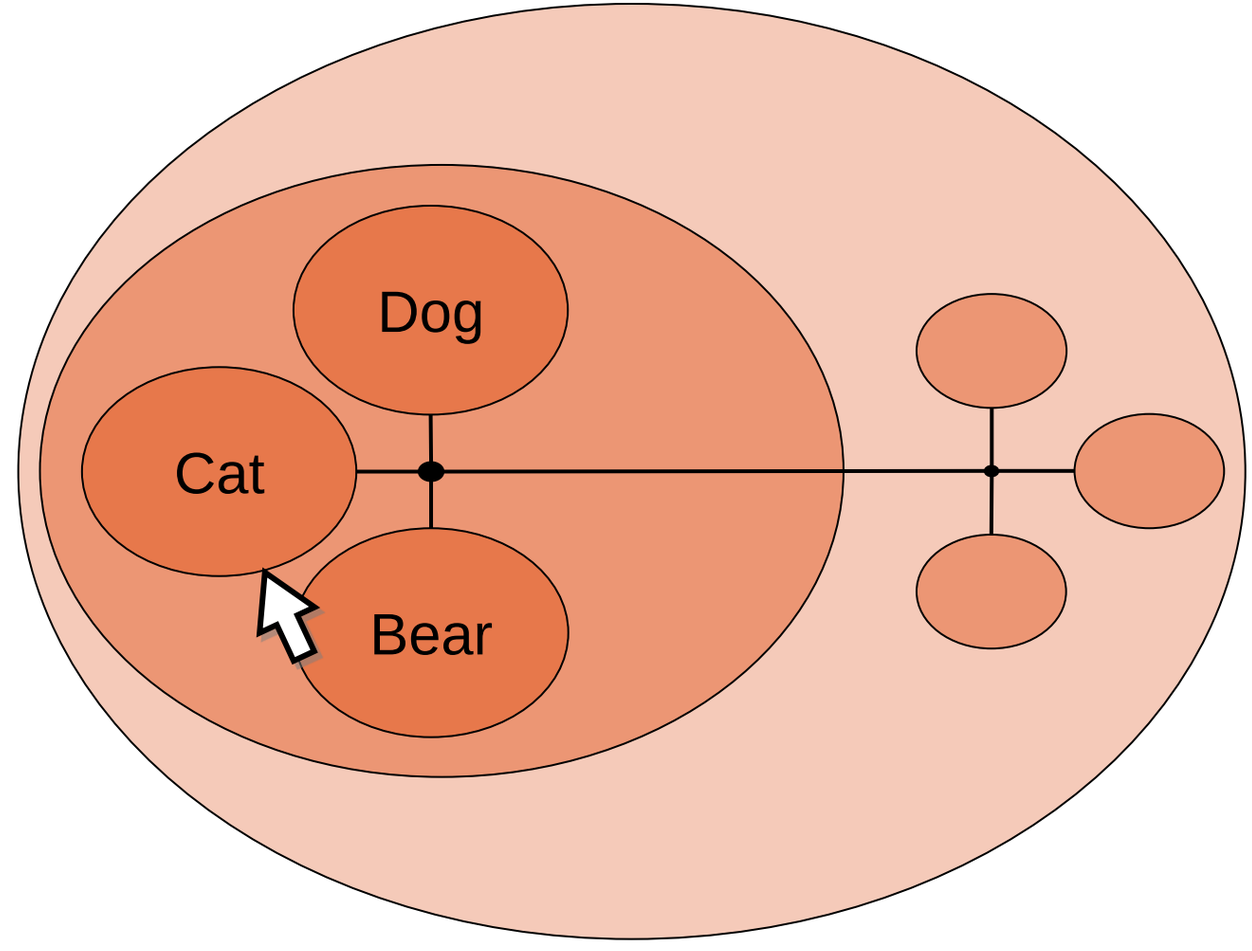
- Mixed node-link and space filling representation
- Allows to interactively browse a tree
  - [Boardman 00]



 copyright ©2025 m.patrignani

Hybrid representations: bubble trees

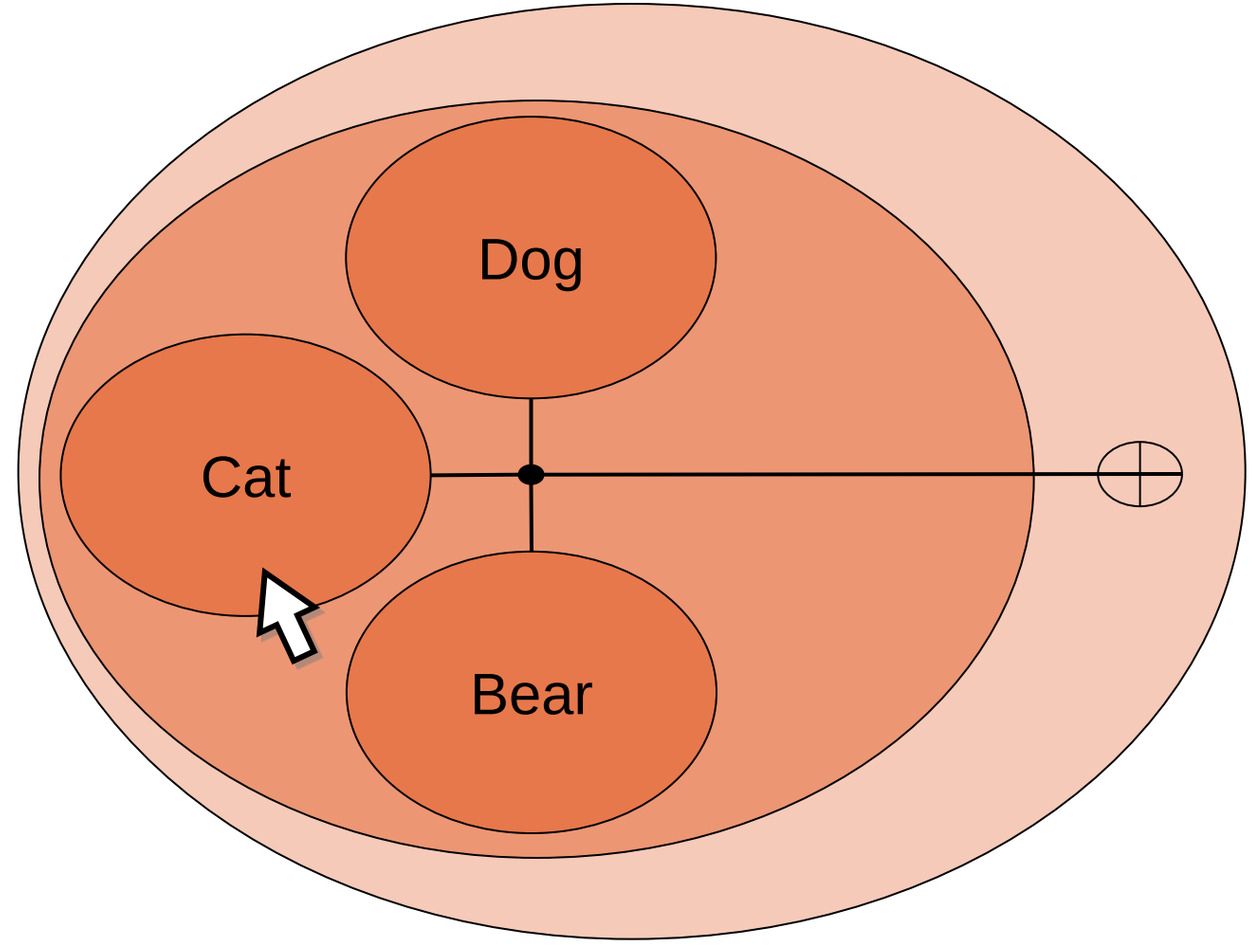
- Mixed node-link and space filling representation
- Allows to interactively browse a tree
  - [Boardman 00]



 copyright ©2025 m.patrignani

Hybrid representations: bubble trees

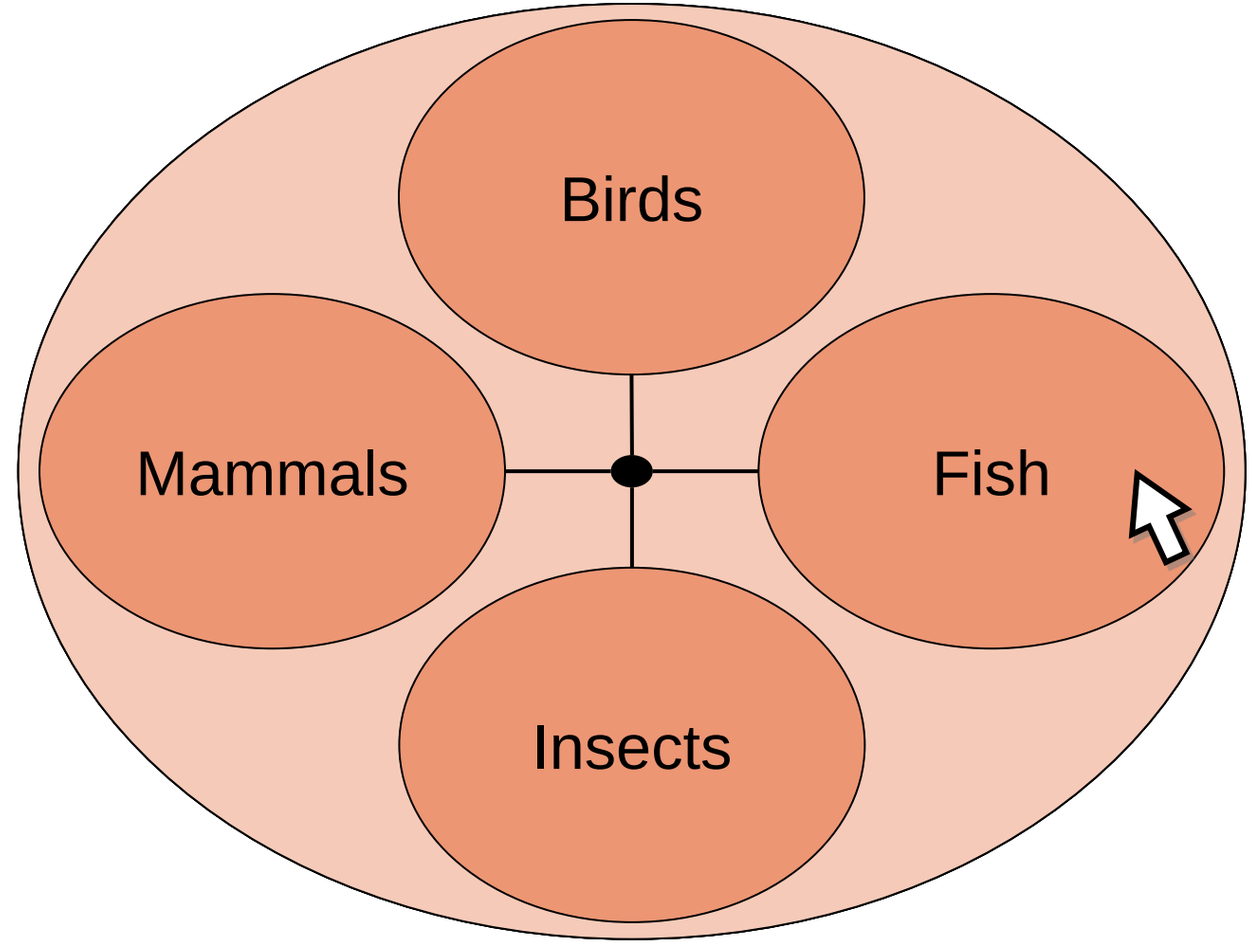
- Mixed node-link and space filling representation
- Allows to interactively browse a tree
  - [Boardman 00]



 copyright ©2025 m.patrignani

Hybrid representations: bubble trees

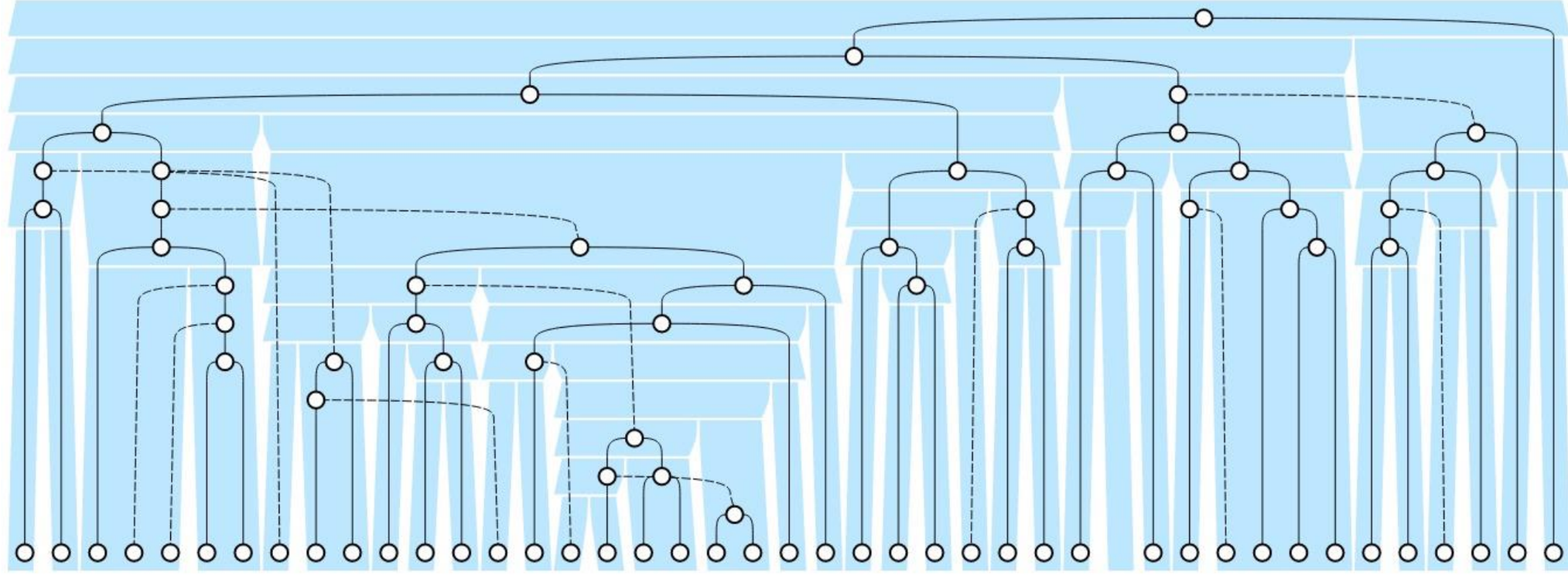
- Mixed node-link and space filling representation
- Allows to interactively browse a tree
  - [Boardman 00]



 copyright ©2025 m.patrignani

Co-phylogenetic trees

- Representation of co-phylogenetic trees in [Calamoneri *et al.* 17]
  - the host tree is represented as an icicle
  - the parasite tree has a node-link representation



 copyright ©2025 m.patrignani

Bibliography and further readings

- [Andrews & Heidegger 98] K. Andrews and H. Heidegger, “Information slices: Visualising and exploring large hierarchies using cascading, semicircular discs”. In IEEE Information Visualization 1998
- [Balzer & Deussen 04] Michael Balzer and Oliver Deussen, “Hierarchy based 3D visualization of large software structures”, poster at Visualization 2004
- [Balzer *et al.*, 05] Michael Balzer, Oliver Deussen, Claus Lewerentz, “Voronoi treemaps for the visualization of software metrics”, ACM Symp. on Software Visualization, 2005
- [Bederson *et al.*, 02] Benjamin B. Bederson, Ben Shneiderman, and Martin Wattenberg, “Ordered and quantum treemaps: Making effective use of 2D space to display hierarchies”, J ACM Trans. Graph. 2002
- [Boardman 00] Richard Boardman, “Bubble trees: The visualization of hierarchical information structures”, CHI EA, 2000
- [Bruls *et al.*, 00] M. Bruls, C. Huizing, J. J. van Wijk, “Squarified Treemaps”. Eurographics and IEEE TCVG Symposium on Visualization, 2000
- [Calamoneri *et al.* 17] Tiziana Calamoneri, Valentino Di Donato, Diego Mariottini, and Maurizio Patrignani, “Visualizing Co-Phylogenetic Reconciliations”, GD 2017
- [Görtler *et al* 18] Jochen Görtler, Christoph Schulz, Daniel Weiskopf, Oliver Deussen. “Bubble Treemaps for Uncertainty Visualization “ IEEE TVCG, 2018
- [Heeringa & Jannen 16] Brent Heeringa, Bill Jannen, “Treemaps” Lecture 15 of CS 135: Diving into the Deluge of Data, Williams College, 2016

 copyright ©2025 m.patrignani

Bibliography and further readings

- [Johnson 93] Brian Johnson, “Treemaps: Visualizing hierarchical and categorical data” PhD thesis, University of Maryland, 1993
- [Johnson & Shneiderman 91] B. Johnson and B. Shneiderman, “Treemaps: A Space-Filling Approach to the Visualization of Hierarchical Information Structures” IEEE Infovis, 1991
- [Reingold & Tilford, 81] Edward M. Reingold and John S. Tilford, “Tidier drawings of trees” IEEE Trans. Soft. Eng. 1981
- [Schedl et al. 07] Markus Schedl, Peter Knees, Gerhard Widmer, Klaus Seyerlehner and Tim Pohle, “Browsing the web using stacked three-dimensional Sunbursts to visualize term co-occurrences and multimedia content” poster at INFOVIS 2007
- [Schulz et al. 11] Hans-Jörg Schulz, Steffen Hadlak and Heidrun Schumann, “The design space of implicit hierarchy visualization: A survey”, IEEE TVCG, 2011
- [Shneiderman & Wattenberg 01] Ben Shneiderman and Martin Wattenberg, “Ordered Treemap Layouts” INFOVIS 2001
- [Stasko & Zhang 00] John Stasko and Eugene Zhang, “Focus+context display and navigation techniques for enhancing radial, space-filling hierarchy visualizations”, INFOVIS 2000
- [Walker 90] J. Q. II Walker, “A node-positioning algorithm for general trees”, Software-Practice and Experience 1990
- [Wattenberg 99] Martin Wattenberg, “Visualizing the stock market”, ACM CHI EA '99
- [van der Weele 2010] Vincent van der Weele. “Treemaps with bounded aspect ratio”. MSc thesis, Department of Mathematics and Computer Science – Technische Universiteit Eindhoven, 2010.
- [van Wijk & van de Wetering 99] J.J. van Wijk and H. van de Wetering “Cushion treemaps - visualization of hierarchical information”, INFOVIS 1999

 copyright ©2025 m.patrignani